ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN

MÔN: Xử lý ảnh số và Thị giác máy tính (Digital Image Processing and Computer Vision)

Project 2

Gradient Domain Editing và Ghép ảnh toàn cảnh

Gradient Domain Editing and Panoramic Image Stitching

Giảng viên hướng dẫn: Võ Thanh Hùng

Lớp: L01

Học kỳ: 261

Sinh viên thực hiện

MSSV	Họ và tên
2311002	Trần Gia Hiển
2313053	Nguyễn Chí Tân
2433042	Võ Thanh
2311601	Mai Anh Khoa

TP. Hồ Chí Minh, tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày hai phần của bài tập lớn: **Phần 1** (Gradient Domain Editing) ghép một đối tượng từ ảnh nguồn vào ảnh nền sao cho đường biên và ánh sáng trông tự nhiên; **Phần 2** ghép 3–4 ảnh chụp cùng một cảnh với các góc xoay khác nhau thành một ảnh toàn cảnh. Hai phần gặp nhau ở bước hòa trộn: bộ hòa trộn đa băng tần của Phần 2 dùng lại các toán tử kim tự tháp đã viết ở Phần 1.

Phần 1. Phương pháp Poisson image editing được hiện thực từ đầu: dựng hệ phương trình Poisson rời rạc trên lưới điểm ảnh, giữ gradient của ảnh nguồn bên trong vùng ghép và ép giá trị tại biên theo ảnh nền, rồi giải hệ thưa cho từng kênh màu. Kết quả được so sánh với hai phương pháp đối chứng cùng được hiện thực (dán trực tiếp bằng mặt nạ và Laplacian pyramid blending) trên ba cặp ảnh có mức chênh lệch màu nền khác nhau. Chỉ số đường ghép (đo trên các điểm nền của ảnh nguồn sát biên) giảm 73–86% so với dán trực tiếp và chỉ bằng khoảng 1/5–1/3 so với Laplacian blending. Thí nghiệm có đáp án cho thấy khi ảnh nguồn chỉ lệch so với nền một phép dịch hằng số hoặc tuyến tính, Poisson khôi phục chính xác ảnh gốc (SSIM = 1.0000), trong khi hai phương pháp còn lại chỉ đạt PSNR $\approx$ 21–22 dB. Hạn chế: màu của nền *lan* vào đối tượng, và việc cắt giá trị về $[0, 1]$ làm mất chi tiết khi chênh lệch màu ở biên lớn; gradient trộn (mixed gradients) làm đối tượng đặc bị *mờ ảo* trên nền nhiều kết cấu.

Phần 2. Hệ thống ghép ảnh toàn cảnh dựa trên đặc trưng (*feature-based image stitching*) gồm tiền xử lý, trích xuất và mô tả đặc trưng, so khớp, ước lượng homography (DLT chuẩn hóa trong RANSAC), căn chỉnh bằng warp ngược và hòa trộn đa băng tần; hầu hết các khâu được viết lại từ đầu bằng NumPy. Cùng một pipeline được chạy với ba bộ trích xuất đặc trưng (SIFT và ORB của OpenCV, Harris + MOPS tự cài đặt) rồi so sánh trên năm bộ ảnh thật và trong một nghiên cứu có kiểm soát với đáp án chính xác (khoảng 3×1050 lần thử). Cả 15 tổ hợp (5 bộ ảnh $\times$ 3 bộ trích xuất) đều ghép thành công, với sai số tái chiếu trung bình 0.65 px (SIFT), 0.83 px (Harris + MOPS) và 1.08 px (ORB). Trên ảnh thật, panorama của ba bộ trích xuất gần như không phân biệt được; khác biệt nằm ở độ chính xác hình học và ở giới hạn chịu đựng. SIFT chịu được chồng lấn nhỏ nhất và zoom lớn nhất; ORB nhanh nhất nhưng hẹp nhất; Harris + MOPS chịu ánh sáng yếu và ảnh nhòe tốt nhất nhưng thất bại khi tỉ lệ nằm giữa hai bậc của kim tự tháp. Báo cáo cũng phân tích các trường hợp khó: chữ lặp lại tạo ra cặp ảnh giả, ảnh tối làm các detector có ngưỡng tuyệt đối mất hết điểm đặc trưng, nước và băng chuyển động giữa các lần chụp, ảnh xám lẫn ảnh màu trong cùng một bộ. Hai phép chẩn đoán trên ảnh thật tách được độ khớp thấp do nội dung thay đổi khỏi độ khớp thấp do căn chỉnh lệch, và đo sai số tích lũy khi nối chuỗi homography (0.08–0.26 px trên cảnh phẳng, 0.6–3.7 px RMS trên hai cảnh ba chiều).

Mục lục

Tóm tắt **i**

1 Mô tả bài toán và mục tiêu **1**
1.1 Tổng quan về bài tập lớn . . . 1
1.2 Phần 1: Gradient Domain Editing . . . 1
1.2.1 Bài toán ghép ảnh trong miền gradient . . . 1
1.2.2 Đầu vào và đầu ra . . . 1
1.2.3 Mục tiêu chính . . . 1
1.2.4 Phạm vi và giả định . . . 3
1.3 Phần 2: Ghép ảnh toàn cảnh (panorama) . . . 3
1.3.1 Bài toán và phạm vi . . . 3
1.3.2 Đầu vào và đầu ra . . . 4
1.3.3 Mục tiêu chính . . . 4
1.3.4 Giả định và dữ liệu . . . 4
1.4 Bố cục báo cáo . . . 5

2 Cơ sở lý thuyết **6**
2.1 Gradient Domain Editing và Poisson image editing . . . 6
2.1.1 Ý tưởng . . . 6
2.1.2 Phát biểu bài toán . . . 6
2.1.3 Bài toán biến phân và phương trình Poisson . . . 6
2.1.4 Cách hiểu hình học: nội suy màng . . . 7
2.1.5 Trường dẫn hướng: nhập gradient và gradient trộn . . . 7
2.1.6 Rời rạc hóa thành hệ tuyến tính thưa . . . 8
2.1.7 Tính chất của hệ và cách giải . . . 9
2.2 Hai phương pháp đối chứng . . . 10
2.2.1 Dán trực tiếp bằng mặt nạ (alpha blending) . . . 10
2.2.2 Laplacian pyramid blending . . . 10
2.2.3 So sánh ba cách tiếp cận . . . 10
2.3 Cách đánh giá chất lượng ghép ảnh . . . 10
2.3.1 Tiêu chí định tính . . . 10
2.3.2 Chỉ số đường ghép S . . . 11
2.3.3 Thí nghiệm có đáp án và các độ đo PSNR, SSIM, MAE . . . 12
2.4 Phần 2: Cơ sở hình học, vì sao là bài toán homography . . . 12

2.5 Trích xuất và mô tả đặc trưng . . . 13
2.5.1 Harris + MOPS (tự cài đặt) . . . 14
2.5.2 SIFT (OpenCV) . . . 15
2.5.3 ORB (OpenCV) . . . 15
2.6 So khớp đặc trưng . . . 16
2.7 Ước lượng homography . . . 17
2.7.1 Direct Linear Transform (DLT) . . . 17
2.7.2 Chuẩn hóa Hartley . . . 17
2.7.3 Khi nào H không duy nhất? Các cấu hình suy biến . . . 18
2.7.4 RANSAC . . . 18
2.7.5 Levenberg–Marquardt trên sai số hình học . . . 19
2.7.6 Cặp ảnh nào chồng lấn? Ba phép kiểm tra . . . 19
2.8 Căn chỉnh nhiều ảnh: đồ thị chồng lấn, warp ngược và canvas . . . 20
2.8.1 Đồ thị chồng lấn, cây khung và ảnh tham chiếu . . . 20
2.8.2 Warp ngược và canvas . . . 20
2.9 Bù sáng và hòa trộn . . . 21
2.10 Cách đánh giá chất lượng ghép ảnh toàn cảnh . . . 22

3 Hiện thực 24
3.1 Môi trường và thư viện . . . 24
3.2 Pipeline tổng thể . . . 24
3.3 Tiền xử lý và đặt vị trí . . . 25
3.4 Dán trực tiếp . . . 25
3.5 Laplacian pyramid blending . . . 26
3.6 Poisson editing (Gradient Domain Editing) . . . 27
3.7 Mã nguồn và hướng dẫn chạy . . . 31
3.8 Phần 2: Pipeline ghép ảnh và vai trò của từng bước . . . 31
3.9 Phần nào tự cài đặt, phần nào dùng thư viện . . . 33
3.10 Tiền xử lý . . . 33
3.11 Quy ước tọa độ: tâm của pixel nằm ở đâu? . . . 34
3.12 Mã nguồn chính của Phần 2 . . . 35

4 Kết quả thực nghiệm 40
4.1 Dữ liệu và thiết lập thí nghiệm . . . 40
4.2 Kết quả định tính . . . 41
4.2.1 Cặp `01`: vịt trắng trên nền nước gợn sóng . . . 41
4.2.2 Cặp `02`: chim cánh cụt trên nền băng . . . 42
4.2.3 Cặp `03`: máy bay trên nền trời . . . 42

4.3 Phân tích bằng số liệu . . . 44
4.3.1 Chỉ số đường ghép . . . 44
4.3.2 Kích thước hệ phương trình và thời gian chạy . . . 45
4.4 Ảnh hưởng của tham số . . . 45
4.4.1 Số mức của Laplacian pyramid . . . 45
4.4.2 Gradient trộn so với nhập gradient . . . 46
4.5 Kiểm chứng lý thuyết . . . 46
4.5.1 Thí nghiệm có đáp án . . . 46
4.5.2 Bản đồ hiệu chỉnh và hiện tượng bão hòa . . . 48
4.6 Phần 2: Dữ liệu và thiết lập thí nghiệm . . . 49
4.7 Kiểm chứng từng khâu của pipeline . . . 50
4.7.1 Điểm đặc trưng . . . 50
4.7.2 So khớp: tác dụng của ratio test . . . 50
4.7.3 RANSAC và Levenberg–Marquardt . . . 50
4.7.4 Cặp ảnh giả: bộ `newspaper` . . . 52
4.7.5 Đồ thị chồng lấn . . . 52
4.7.6 Phép warp . . . 52
4.8 Kết quả trên năm bộ ảnh thật . . . 52
4.9 Phân tích từng bộ ảnh: thành công và khó khăn . . . 56
4.10 Nghiên cứu có kiểm soát với đáp án chính xác . . . 58
4.11 Hòa trộn: đường nối và bóng ma . . . 61
4.12 Thời gian chạy . . . 63
4.13 Sai số tích lũy khi nối chuỗi homography . . . 64

5 Nhận xét và thảo luận 66
5.1 Phân tích kết quả . . . 66
5.1.1 Điểm mạnh của Poisson editing . . . 66
5.1.2 Trường hợp làm tốt và chưa tốt . . . 66
5.2 Nguyên nhân của các lỗi . . . 67
5.2.1 Lan màu và bão hòa . . . 67
5.2.2 Cấu trúc thừa của nguồn nằm trong mặt nạ . . . 67
5.2.3 Gradient trộn làm đối tượng đặc bị mờ ảo . . . 67
5.2.4 Laplacian blending chỉ che mờ đường ghép . . . 67
5.3 Ảnh hưởng của tham số . . . 68
5.4 Hạn chế của giải pháp . . . 68
5.5 Phần 2: So sánh các bộ trích xuất đặc trưng . . . 68
5.5.1 SIFT (OpenCV) . . . 69
5.5.2 ORB (OpenCV) . . . 69

5.5.3 Harris + MOPS (tự viết) 70
5.5.4 Bảng tổng hợp và khuyến nghị 70
5.6 Phần 2: Thành công và các trường hợp khó 71
5.6.1 Những gì đã hoạt động 71
5.6.2 Những gì không hoạt động, và vì sao 71

6 Kết luận và hướng phát triển 73
6.1 Tóm tắt kết quả đạt được 73
6.1.1 Phần 1: Gradient Domain Editing 73
6.1.2 Phần 2: Ghép ảnh toàn cảnh 73
6.2 Hướng cải thiện (Phần 1) 74
6.3 Hướng mở rộng 74
6.3.1 Phần 1: Gradient Domain Editing 74
6.3.2 Phần 2: Ghép ảnh toàn cảnh 74

Tài liệu tham khảo 75

A Phân công công việc 78

B Nguồn tham khảo và công cụ hỗ trợ 79

C Cách chạy lại Phần 2 80

D Tham số mặc định của Phần 2 82

Danh mục hình vẽ

1.1 Minh họa bài toán trên cặp ảnh `03` 2

2.1 Minh họa một chiều của nội suy màng 8
2.2 Lưới điểm ảnh quanh vùng ghép Ω 9
2.3 Bộ mô tả MOPS 15

3.1 Pipeline của Phần 1: tiền xử lý dùng chung, ba nhánh tính ảnh ghép, rồi ghép kết quả vào ảnh nền. 25
3.2 Các bước bên trong `poisson_blend` cho một kênh màu (bước 1–3) và bước gộp kênh (bước 4). 27
3.3 Cấu trúc ma trận A cho một lưới nhỏ 29
3.4 Sơ đồ pipeline Phần 2 32

4.1 Ảnh đầu vào của ba cặp thử nghiệm (nguồn, mặt nạ vẽ bằng GIMP, nền). 41
4.2 Kết quả trên cặp `01`. 42
4.3 Kết quả trên cặp `02`. 43
4.4 Kết quả trên cặp `03`. 43
4.5 Laplacian blending trên cặp `03` với số mức khác nhau (phóng to vùng ghép). 46
4.6 Poisson với nhập gradient (trái) và gradient trộn (phải) trên hai cặp có nền khác nhau. 47
4.7 Thí nghiệm có đáp án (vùng ghép phóng to): chỉ Poisson trả về đúng ảnh gốc. 47
4.8 Bản đồ hiệu chỉnh $h = f - g$ trong Ω 48
4.9 Điểm đặc trưng của ba bộ trích xuất 50
4.10 Tác dụng của ngưỡng ratio test 50
4.11 Bố cục panorama bộ `cathedral` 52
4.12 Panorama của năm bộ ảnh (SIFT) 53
4.13 Cặp ảnh 2–3 của bộ `boat` và các nội điểm 54
4.14 Bộ `cathedral` với ba bộ trích xuất 55
4.15 Các độ đo trên năm bộ ảnh và ba bộ trích xuất 55
4.16 Tỉ lệ lần thử đúng theo từng yếu tố khó 58
4.17 Số nội điểm theo hệ số zoom (răng cưa) 61
4.18 Các chế độ hòa trộn khi có sai số căn chỉnh và chênh lệch phơi sáng 62
4.19 Các chế độ hòa trộn tại một đường nối thật 63

Danh mục bảng

1.1 Đầu vào và đầu ra của bài toán ghép ảnh (Phần 1). 2
1.2 Đầu vào và đầu ra của bài toán ghép ảnh toàn cảnh (Phần 2). 4

2.1 Các cách giải hệ Poisson trên lưới 2D với n ẩn [1, 2]. 9
2.2 So sánh về mặt lý thuyết giữa ba phương pháp ghép ảnh. 11
2.3 Ba bộ trích xuất đặc trưng. 16
2.4 Các chế độ hòa trộn. 21

3.1 Thành phần tự cài đặt và thư viện sử dụng. 24
3.2 Vai trò của từng bước và hậu quả khi bước đó thiếu hoặc hỏng. 32
3.3 Phân chia giữa mã tự viết và thư viện trong Phần 2. 33
3.4 Phép thử xoay 180° trên ảnh `boat1`: sai lệch giữa vị trí mong đợi và vị trí tìm được của các điểm khớp (phần dư = 2× độ lệch). 34

4.1 Mô tả ba cặp ảnh thử nghiệm. 40
4.2 Kích thước ảnh, vị trí đặt và kích thước vùng ghép của từng cặp (s: hệ số tỉ lệ; $H_s \times W_s$: vùng ghép sau khi co giãn). 41
4.3 Tổng hợp định tính trên ba cặp ảnh theo các tiêu chí (T1)–(T3) của Mục 2.3. 44
4.4 Chỉ số đường ghép (thang 0–255, càng nhỏ càng tốt). Cột "Ảnh nền gốc" là mức biến thiên tự nhiên để tham chiếu. 44
4.5 Kích thước hệ Poisson và thời gian chạy 45
4.6 Chỉ số đường ghép S của Laplacian blending theo số mức L; cột cuối là Poisson để so sánh. 45
4.7 Kết quả thí nghiệm có đáp án 48
4.8 Khoảng giá trị của $h = f - g$, mức bão hòa và kiểm tra tính điều hòa . . . 49
4.9 Năm bộ ảnh thật (ảnh kiểm thử ghép ảnh của OpenCV). Tỉ lệ chồng lấn là phần diện tích của một ảnh nằm trong ảnh kế bên, tính từ homography của SIFT trên các cặp của cây khung (khoảng giá trị của hai chiều); `scripts/describe_scenes.py` liệt kê mọi cặp. 49
4.10 Ratio test ở $\tau = 0.75$ trên cặp tổng hợp cách nhau 8°. 51
4.11 Số mẫu RANSAC cần thiết và độ bền với ngoại lai (12 lần chạy mỗi hàng, tối đa 3000 mẫu; "đúng" nghĩa là sai số < 1 px). 51
4.12 Hai ảnh đầu của bộ `boat` với ba bộ trích xuất (ratio test 0.75 + lọc một–một, RANSAC ngưỡng 3 px). RMSE là sai số tái chiếu RMS của các nội điểm. 51
4.13 Cặp giả của bộ `newspaper` (ảnh 0 và 2, SIFT). 52

4.14 Kết quả của 15 tổ hợp (5 bộ ảnh × 3 bộ trích xuất). Nội điểm: tổng trên các cặp của cây khung; thời gian là một lần chạy trên máy dùng chung và dao động hàng chục phần trăm giữa các lần chạy, các cột còn lại tái lập được. . . . 53
4.15 Trung bình trên năm bộ ảnh. . . . 53
4.16 Bộ `boat` (SIFT): NCC thông cao trong sáu dải ngang của vùng chồng lấn, từ trên xuống dưới, với ảnh như đã warp (hàng "NCC") và sau phép dịch ảnh thứ hai tốt nhất trong ±12 px (hàng "sau khi dịch"; vector (d_x, d_y) ghi trong ngoặc khi khác 0, nghĩa là ảnh thứ hai được lấy mẫu tại $(x{+}d_x,\ y{+}d_y)$ để so với ảnh thứ nhất tại (x, y)). . . . 57
4.17 Giới hạn của từng bộ trích xuất: mức khó nhất vẫn giải được ít nhất 90% trong 30 lần thử (in đậm: tốt nhất). . . . 59
4.18 Lỗi thầm lặng: cặp được chấp nhận nhưng homography sai từ 3 px trở lên (trên 1050 lần thử mỗi bộ). . . . 59
4.19 Số điểm đặc trưng trong một ảnh có gain cho trước. . . . 60
4.20 Ảnh có gain 1 khớp với ảnh có gain 0.15 (4 lần thử). . . . 60
4.21 Số nội điểm RANSAC (trung vị của 3 lần thử) theo hệ số zoom. . . . 61
4.22 Độ nét của vùng chồng lấn khi căn chỉnh chính xác và khi có sai số 3.6 px. 62
4.23 Thời gian của các bước phụ thuộc bộ trích xuất (mili-giây, trung bình trên mọi bộ ảnh). Đây là thời gian của hiện thực NumPy này chứ không phải của thuật toán, và dao động hàng chục phần trăm giữa các lần chạy. . . . 63
4.24 Độ lệch giữa homography nối chuỗi và homography ước lượng trực tiếp của các cặp không thuộc cây khung (chỉ gồm các cặp được nhận). . . . 64

5.1 Tóm tắt các trường hợp làm tốt và chưa tốt của Poisson editing và số liệu đi kèm. . . . 66
5.2 Tổng hợp so sánh ba bộ trích xuất (in đậm: tốt nhất; ở hàng "xoay" cả ba như nhau). Mọi số liệu lấy từ các bảng của Chương 4. . . . 70

A.1 Phân công công việc và mức đóng góp. . . . 78

C.1 Nguồn của các bảng và hình có số liệu của Phần 2 (các bảng mô tả 3.3, 3.2, 2.4, 5.2 và D.1 không có trong danh sách). . . . 81

D.1 Tham số mặc định của `StitchConfig` (`src/panorama/config.py`) dùng cho mọi kết quả trên ảnh thật. . . . 82

Chương 1 Mô tả bài toán và mục tiêu

1.1 Tổng quan về bài tập lớn

Bài tập lớn gồm hai phần. **Phần 1** (Gradient Domain Editing) ghép một đối tượng từ ảnh nguồn vào ảnh nền sao cho đường biên và ánh sáng trông tự nhiên. **Phần 2** (ghép ảnh toàn cảnh) ghép 3–4 ảnh chụp cùng một cảnh từ cùng một vị trí với các góc xoay khác nhau thành một ảnh có góc nhìn rộng hơn. Hai phần gặp nhau ở bước cuối của pipeline panorama: bộ hòa trộn đa băng tần của Phần 2 tổng quát hóa phép trộn Laplacian hai ảnh của Phần 1 lên K ảnh và dùng lại chính các toán tử kim tự tháp đã viết ở Phần 1 (`build_gaussian_pyramid`, `build_laplacian_pyramid`, `pyr_up`; Mục 2.9). Mục 1.2 trình bày bài toán của Phần 1 và Mục 1.3 trình bày bài toán của Phần 2.

1.2 Phần 1: Gradient Domain Editing

1.2.1 Bài toán ghép ảnh trong miền gradient

Trong thao tác *ghép ảnh* (image compositing), ta cắt một đối tượng từ *ảnh nguồn* rồi đặt vào một *ảnh nền* khác. Nếu chỉ chép nguyên các điểm ảnh của đối tượng sang ảnh nền, đường viền của vùng chép gần như luôn lộ ra: hai ảnh được chụp dưới điều kiện chiếu sáng, nhiệt độ màu và kết cấu nền khác nhau, nên quanh đối tượng xuất hiện một "mảng" có màu khác hẳn phần còn lại (Hình 1.1c).

Một quan sát thường dùng làm động cơ cho giải pháp là thị giác người nhạy với *sự thay đổi đột ngột của cường độ* (gradient) hơn là với giá trị cường độ tuyệt đối [3, 4]. Vì vậy, thay vì cố ép màu của đối tượng khớp với nền, *Gradient Domain Editing* làm việc trực tiếp trên gradient: giữ lại *cấu trúc* (gradient) của đối tượng, còn màu tuyệt đối được suy ra sao cho vừa khít với ảnh nền tại đường biên (Hình 1.1d). Báo cáo này trình bày **Phần 1** của bài tập lớn, tập trung vào bài toán đó.

1.2.2 Đầu vào và đầu ra

Bảng 1.1 liệt kê đầu vào và đầu ra của bài toán ghép ảnh ở Phần 1.

1.2.3 Mục tiêu chính

1. **Hiện thực Gradient Domain Editing từ đầu:** dựng hệ phương trình Poisson rời rạc cho vùng ghép, giải bằng bộ giải thưa và ghép kết quả trở lại ảnh nền, kể cả trường hợp gradient trộn (mixed gradients).



(a) Ảnh nguồn



(b) Ảnh nền



(c) Dán trực tiếp bằng mặt nạ



(d) Gradient Domain Editing (Poisson)

Hình 1.1: Minh họa bài toán trên cặp ảnh `03`: dán trực tiếp để lộ mảng nền trời của ảnh nguồn quanh máy bay; Poisson editing làm vùng nền hòa vào ảnh nền mà vẫn giữ nguyên chi tiết của máy bay.

Bảng 1.1: Đầu vào và đầu ra của bài toán ghép ảnh (Phần 1).

Thành phần	**Mô tả**	**Dạng dữ liệu**
Đầu vào		
Ảnh nguồn g	Ảnh chứa đối tượng cần ghép	RGB 8 bit, $H_s \times W_s \times 3$
Mặt nạ M	Vùng trắng bao quanh đối tượng (có thể chứa một ít nền xung quanh); vùng đen bị bỏ qua	Trắng–đen, $H_s \times W_s$
Ảnh nền f^*	Ảnh nhận đối tượng	RGB 8 bit, $H_t \times W_t \times 3$
Vị trí đặt	Offset (y_0, x_0) của góc trên trái ảnh nguồn trên ảnh nền và hệ số tỉ lệ s	Chọn tương tác bằng chuột
Đầu ra		
Ảnh ghép f	Ảnh nền đã nhận đối tượng; ngoài vùng ghép giữ nguyên ảnh nền	RGB, $H_t \times W_t \times 3$

2. **Hiện thực hai phương pháp đối chứng:** dán trực tiếp bằng mặt nạ (alpha blending) và Laplacian pyramid blending, để thấy rõ Poisson editing khác gì so với các cách xử lý trong miền không gian và miền tần số.
3. **So sánh có phân tích** ba phương pháp trên nhiều cặp ảnh: bằng quan sát trực quan, bằng một chỉ số đo đường ghép, và bằng thí nghiệm *có đáp án* để kiểm chứng lý thuyết.
4. **Đánh giá ảnh hưởng của tham số** (số mức pyramid, gradient trộn) và nêu rõ các trường hợp thành công / thất bại.

1.2.4 Phạm vi và giả định

- Ảnh màu RGB 8 bit, được đưa về $[0, 1]$; mỗi kênh màu được xử lý *độc lập*.
- Mặt nạ được vẽ thủ công bằng GIMP theo hướng dẫn trong README của repo (vùng chọn tô trắng trên nền đen); vị trí và tỉ lệ đặt ảnh nguồn được chọn bằng cửa sổ tương tác.
- Chỉ xét phép đặt *tịnh tiến* và *co giãn đều*. Báo cáo không xử lý sai khác về hình học, phối cảnh, hướng chiếu sáng hay độ phân giải giữa hai ảnh.
- Vùng ghép Ω nằm gọn bên trong vùng đặt (không chạm biên ảnh); lưới điểm ảnh dùng 4 láng giềng.
- Kích thước ảnh vừa phải (hệ phương trình cỡ 10^5 ẩn) để dùng bộ giải trực tiếp cho ma trận thưa.
- Dữ liệu gồm **ba cặp ảnh** `01`, `02`, `03` với mức chênh lệch màu và kết cấu nền khác nhau, mô tả ở Mục 4.1.

1.3 Phần 2: Ghép ảnh toàn cảnh (panorama)

1.3.1 Bài toán và phạm vi

Cho 3–4 ảnh chụp cùng một cảnh từ cùng một vị trí, camera được xoay giữa các lần chụp; cần ghép chúng thành một ảnh có góc nhìn rộng hơn. Bài toán có hai nửa. **Căn chỉnh** (*registration*): đưa mọi ảnh về một hệ tọa độ chung, cần hình học và đặc trưng. **Hợp thành** (*compositing*): hòa trộn các vùng chồng lấn thành một ảnh liền mạch, cần các kỹ thuật hòa trộn ảnh.

Đề bài yêu cầu tối thiểu: tiền xử lý, trích xuất đặc trưng, so khớp, ước lượng homography, căn chỉnh và hòa trộn cơ bản, trong đó **ít nhất hai** bộ trích xuất đặc trưng chạy trên **cùng một** pipeline và được so sánh về độ chính xác căn chỉnh, tính liên tục và chất lượng trực quan. Hệ thống ở đây làm đủ các bước đó và bổ sung: đồ thị chồng lấn (ảnh có thể đưa vào theo thứ tự bất kỳ), các phép kiểm tra loại cặp ảnh giả, bù sáng, bốn chế độ hòa trộn để so sánh, và một bộ đánh giá có đáp án chính xác.

1.3.2 Đầu vào và đầu ra

Bảng 1.2 liệt kê đầu vào và đầu ra của bài toán ghép ảnh toàn cảnh ở Phần 2.

Bảng 1.2: Đầu vào và đầu ra của bài toán ghép ảnh toàn cảnh (Phần 2).

Thành phần	**Mô tả**	**Dạng dữ liệu**
Đầu vào		
Tập ảnh	2–4 ảnh chụp từ cùng một vị trí, camera chỉ xoay giữa các lần chụp (đề bài: 3–4 ảnh); thứ tự đưa vào tùy ý	Ảnh màu hoặc xám
Tham số	Bộ trích xuất đặc trưng (SIFT, ORB hoặc Harris + MOPS), chế độ hòa trộn, bù sáng bật hoặc tắt (Phụ lục D)	Cấu hình `StitchConfig`
Đầu ra		
Ảnh toàn cảnh	Canvas chứa mọi ảnh đã căn chỉnh và hòa trộn; ảnh không chồng lấn bị loại và được báo cáo	Ảnh màu
Thông tin kèm theo	Homography của từng ảnh về ảnh tham chiếu, đồ thị chồng lấn, số nội điểm và sai số tái chiếu của từng cặp	Ma trận 3×3, thống kê

1.3.3 Mục tiêu chính

1. **Hiện thực pipeline ghép ảnh dựa trên đặc trưng** (*feature-based image stitching*): tiền xử lý, trích xuất và mô tả đặc trưng, so khớp, ước lượng homography, căn chỉnh và hòa trộn; phần lớn các khâu được viết lại từ đầu bằng NumPy.
2. **So sánh ba bộ trích xuất đặc trưng** trên cùng một pipeline: SIFT và ORB (OpenCV) và Harris + MOPS (tự cài đặt), về độ chính xác căn chỉnh, tính liên tục của ảnh và chất lượng trực quan.
3. **Đánh giá có đáp án:** dựng góc nhìn tổng hợp bằng camera ảo chỉ xoay để biết homography đúng và đo sai số bằng pixel.
4. **Phân tích các trường hợp thành công và khó khăn** trên ảnh thật và trong các thí nghiệm có kiểm soát.

1.3.4 Giả định và dữ liệu

- Camera chỉ xoay quanh tâm quang học (hoặc cảnh gần như phẳng), nên homography là mô hình hình học duy nhất (Mục 2.4). Thị sai nhỏ được chấp nhận và để lại bóng ma ở vật gần.

- Ảnh được ghép lên *mặt phẳng* của ảnh tham chiếu; không dùng phép chiếu trụ hoặc cầu, không có bundle adjustment hay đóng vòng.
- Bộ ảnh mẫu của môn học chưa có tại thời điểm thực hiện, nên báo cáo dùng **năm bộ ảnh kiểm thử ghép ảnh của dự án OpenCV** (`opencv_extra/testdata/stitching`; nguồn gốc ghi trong `data/panorama/README.md`): `boat` (4 ảnh), `newspaper` (4), `cathedral` (3), `mountain` (2), `aqueduct` (2), mô tả ở Mục 4.6.
- Ảnh của môn học được thêm bằng cách đặt vào `data/panorama/input/<tên bộ>/` rồi chạy lại các script (Phụ lục C); không cần sửa mã.

1.4 Bố cục báo cáo

Chương 2 trình bày cơ sở lý thuyết của cả hai phần: bài toán biến phân Poisson và hai phương pháp đối chứng cho Phần 1, rồi cơ sở hình học, đặc trưng, so khớp, homography, căn chỉnh và hòa trộn cho Phần 2. Chương 3 mô tả pipeline hiện thực và vai trò từng bước. Chương 4 báo cáo kết quả thực nghiệm; Chương 5 phân tích điểm mạnh, các trường hợp thất bại, nguyên nhân, ảnh hưởng của tham số và so sánh các bộ trích xuất đặc trưng; Chương 6 kết luận và đề xuất hướng phát triển. Mỗi chương xếp nội dung của Phần 1 trước, Phần 2 sau.

Chương 2 Cơ sở lý thuyết

Các Mục 2.1–2.3 là cơ sở lý thuyết của Phần 1 (Gradient Domain Editing); các Mục 2.4–2.10 là của Phần 2 (ghép ảnh toàn cảnh).

2.1 Gradient Domain Editing và Poisson image editing

2.1.1 Ý tưởng

Một ảnh có thể được mô tả tương đương bằng giá trị cường độ tại từng điểm, hoặc bằng *trường gradient* cùng một điều kiện biên: từ gradient, ta khôi phục lại ảnh bằng cách giải phương trình Poisson. Chỉnh sửa trong miền gradient tận dụng điều đó: ta chọn (hoặc chỉnh sửa) trường gradient mong muốn $\mathbf{v}$, rồi tìm ảnh mới có gradient *gần* $\mathbf{v}$ *nhất* và vẫn khớp với ảnh nền tại biên của vùng cần sửa. Kỹ thuật này tổng quát hơn việc ghép ảnh: nó còn được dùng cho nén dải động ảnh HDR [5], làm phẳng kết cấu hay đổi màu cục bộ [3]. Báo cáo chỉ dùng nó cho *seamless cloning*.

Vì sao làm việc trong miền gradient thay vì miền cường độ?

Sai khác ánh sáng giữa hai ảnh thường là *thay đổi chậm* (gần như một độ lệch màu cộng thêm), còn thứ làm nên hình dạng và chi tiết của đối tượng là các *cạnh* (gradient lớn). Gradient bỏ qua độ lệch cộng thêm nhưng giữ nguyên các cạnh; vì thế nếu ta bắt buộc "giữ gradient của nguồn" và "khớp cường độ của nền ở biên", độ lệch màu sẽ được hấp thụ, còn chi tiết của đối tượng được bảo toàn. Ngược lại, dán trực tiếp giữ cường độ tuyệt đối của nguồn nên mang theo cả độ lệch màu.

2.1.2 Phát biểu bài toán

Xét một kênh màu. Gọi $S \subset \mathbb{R}^2$ là miền xác định của ảnh nền, $\Omega \subset S$ là vùng ghép với biên $\partial\Omega$. Ký hiệu $f^* : S \to \mathbb{R}$ là ảnh nền, g là ảnh nguồn (đã đặt đúng vị trí, xác định trên Ω), $f : \Omega \to \mathbb{R}$ là ẩn cần tìm (ngoài Ω ảnh kết quả bằng f^*) và $\mathbf{v}$ là trường vector dẫn hướng trên Ω.

2.1.3 Bài toán biến phân và phương trình Poisson

Ảnh kết quả là nghiệm của bài toán cực tiểu hóa độ lệch gradient dưới điều kiện biên Dirichlet [3]:

$$\min_f \iint_\Omega \|\nabla f - \mathbf{v}\|^2 \, dx\, dy \qquad \text{với} \qquad f\big|_{\partial\Omega} = f^*\big|_{\partial\Omega}. \tag{2.1}$$

Tích phân đo độ lệch giữa gradient của ảnh kết quả và gradient mong muốn; điều kiện biên bảo đảm không có bước nhảy cường độ tại $\partial\Omega$. Phương trình Euler–Lagrange của (2.1) là phương trình Poisson

$$\Delta f = \operatorname{div}\mathbf{v} \ \text{ trong } \Omega, \qquad f\big|_{\partial\Omega} = f^*\big|_{\partial\Omega}. \tag{2.2}$$

Thật vậy, gọi f là nghiệm và φ là hàm thử bất kỳ với $\varphi|_{\partial\Omega} = 0$. Khi đó $E(f+\varepsilon\varphi) = E(f) + 2\varepsilon \iint_\Omega \langle \nabla f - \mathbf{v}, \nabla\varphi\rangle + \varepsilon^2 \iint_\Omega \|\nabla\varphi\|^2$, nên điều kiện cực tiểu tại $\varepsilon = 0$ là $\iint_\Omega \langle \nabla f - \mathbf{v}, \nabla\varphi\rangle = 0$ với mọi φ. Tích phân từng phần (công thức Green, số hạng biên triệt tiêu vì $\varphi|_{\partial\Omega} = 0$) cho $-\iint_\Omega \varphi\,(\Delta f - \operatorname{div}\mathbf{v}) = 0$ với mọi φ, suy ra (2.2). Vì phiếm hàm là dạng toàn phương lồi nên nghiệm này là cực tiểu duy nhất.

2.1.4 Cách hiểu hình học: nội suy màng

Với *nhập gradient* $\mathbf{v} = \nabla g$ (xem Mục 2.1.5), (2.2) trở thành $\Delta f = \Delta g$. Đặt $h = f - g$, ta được

$$f = g + h, \qquad \Delta h = 0 \text{ trong } \Omega, \qquad h\big|_{\partial\Omega} = (f^* - g)\big|_{\partial\Omega}. \tag{2.3}$$

Nghĩa là kết quả bằng ảnh nguồn *cộng* một hàm điều hòa h nội suy (như một tấm màng căng) sai lệch $f^* - g$ đo được ở biên vào bên trong Ω (Hình 2.1). Từ đây suy ra bốn điều:

(i) *Cấu trúc được giữ:* $\nabla f = \nabla g + \nabla h$ với ∇h biến thiên chậm, nên các cạnh của nguồn còn nguyên.

(ii) *Khớp biên:* $f = f^*$ trên $\partial\Omega$ nên không còn bước nhảy tại đường ghép.

(iii) *Lan màu và bão hòa:* theo nguyên lý cực đại cho hàm điều hòa, $\min_{\partial\Omega}(f^* - g) \le h \le \max_{\partial\Omega}(f^* - g)$; nếu màu nền ở biên khác xa màu nguồn thì h lớn và toàn bộ vật thể bị kéo theo, có thể đẩy giá trị ra ngoài $[0, 1]$ (Mục 5.2.1).

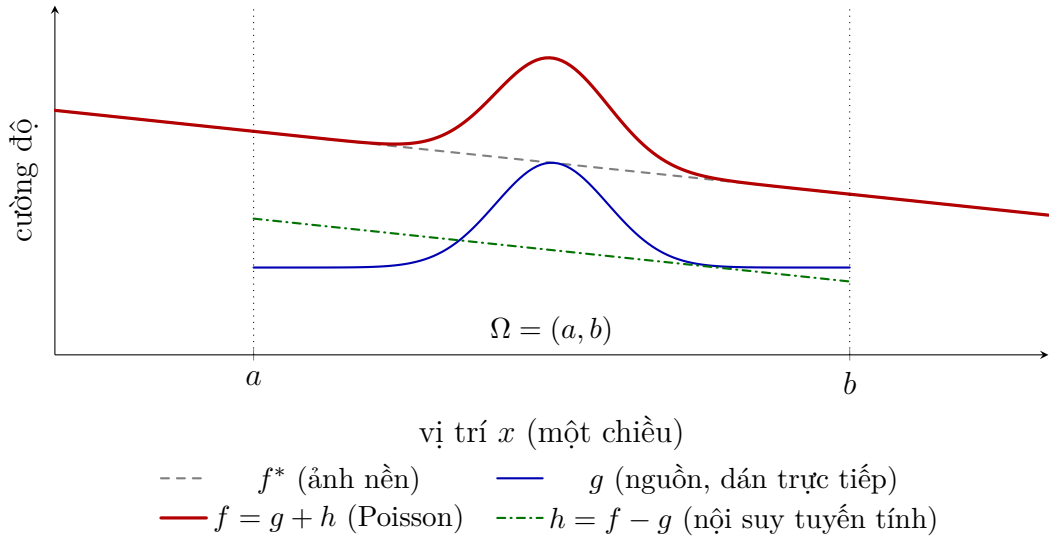
(iv) *Khôi phục chính xác:* nếu $g = f^* + \eta$ với $\Delta\eta = 0$ (độ lệch hằng số hoặc tuyến tính) thì f^* cũng thỏa (2.2) với cùng điều kiện biên, nên theo tính duy nhất $f = f^*$. Đây là cơ sở của thí nghiệm có đáp án ở Mục 2.3.3.

2.1.5 Trường dẫn hướng: nhập gradient và gradient trộn

- **Nhập gradient (importing gradients):** $\mathbf{v} = \nabla g$. Đây là lựa chọn mặc định, phù hợp với đối tượng *đặc*.
- **Gradient trộn (mixed gradients):** tại mỗi điểm giữ gradient *mạnh hơn* trong hai ảnh [3],

$$\mathbf{v}(x) = \begin{cases} \nabla f^*(x), & \|\nabla f^*(x)\| > \|\nabla g(x)\|, \\ \nabla g(x), & \text{ngược lại.} \end{cases} \tag{2.4}$$

  Nhờ vậy các cạnh mạnh của ảnh nền vẫn "xuyên qua" vùng ghép; thích hợp cho đối tượng trong suốt, có lỗ hổng, hoặc khi cần giữ kết cấu của nền.



Hình 2.1: Minh họa một chiều của (2.3): g (xanh) nhảy vọt so với nền f^* tại a và b khi dán trực tiếp. Poisson cộng thêm h, nghiệm điều hòa một chiều, tức đường thẳng nối hai sai lệch ở biên (xanh lá), nên f (đỏ) liền mạch với nền tại a, b còn hình dạng đỉnh của g được giữ nguyên.

2.1.6 Rời rạc hóa thành hệ tuyến tính thưa

Trên lưới điểm ảnh, gọi $\mathcal{N}_p$ là tập bốn láng giềng (trên, dưới, trái, phải) của điểm p, Ω là một tập hữu hạn các điểm ảnh và $\partial\Omega = \{q \notin \Omega : \mathcal{N}_q \cap \Omega \neq \emptyset\}$ là các điểm ngoài Ω kề Ω (Hình 2.2). Bài toán (2.1) trở thành

$$\min_{f|_\Omega} \sum_{\{p,q\}\cap\Omega\neq\emptyset} \big(f_p - f_q - v_{pq}\big)^2, \qquad f_p = f_p^* \;\; \forall p \in \partial\Omega, \tag{2.5}$$

trong đó tổng chạy trên các cặp láng giềng có ít nhất một điểm thuộc Ω, và $v_{pq} = -v_{qp}$ là gradient dẫn hướng trên cạnh pq:

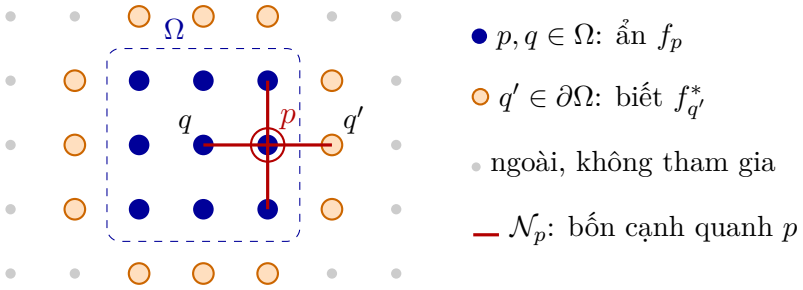
$$\begin{aligned} v_{pq} &= g_p - g_q && \text{(nhập gradient)},\\ v_{pq} &= \begin{cases} g_p - g_q, & |g_p - g_q| > |f_p^* - f_q^*|,\\ f_p^* - f_q^*, & \text{ngược lại} \end{cases} && \text{(gradient trộn)}. \end{aligned} \tag{2.6}$$

Lấy đạo hàm theo f_p ($p \in \Omega$) và cho bằng 0: $\sum_{q\in\mathcal{N}_p}(f_p - f_q - v_{pq}) = 0$. Chuyển các giá trị biên đã biết sang vế phải:

$$|\mathcal{N}_p|\, f_p - \sum_{q\in\mathcal{N}_p\cap\Omega} f_q = \sum_{q\in\mathcal{N}_p\cap\partial\Omega} f_q^* + \sum_{q\in\mathcal{N}_p} v_{pq}, \qquad \forall p \in \Omega. \tag{2.7}$$

Vế trái là $-\Delta f$ theo toán tử Laplace năm điểm; với $v_{pq} = g_p - g_q$ thì $\sum_q v_{pq} = -\Delta g_p$, nên (2.7) chính là $\Delta f = \Delta g$ trên lưới, khớp với dạng liên tục (2.2).

Dạng ma trận dùng trong hiện thực. Để khỏi phải đánh chỉ số lại các điểm của Ω, mọi điểm của vùng đặt (kể cả ngoài Ω) được xem là ẩn: $\mathbf{f} \in \mathbb{R}^n$ với $n = H_s W_s$ và hệ



Hình 2.2: Lưới điểm ảnh quanh vùng ghép Ω. Phương trình (2.7) tại p gồm ẩn f_p, ẩn của các láng giềng thuộc Ω (q) và giá trị biết trước của các láng giềng trên biên (q').

$A\mathbf{f} = \mathbf{b}$, hàng thứ p là

$$\begin{cases} A_{pp} = 4, \quad A_{pq} = -1 \ (q \in \mathcal{N}_p), \quad b_p = \sum_{q \in \mathcal{N}_p} v_{pq}, & p \in \Omega, \\ A_{pp} = 1, \quad A_{pq} = 0 \ (q \neq p), \quad b_p = f_p^*, & p \notin \Omega. \end{cases} \tag{2.8}$$

Các hàng $p \notin \Omega$ cố định $f_p = f_p^*$; thế vào các hàng $p \in \Omega$ ta thu lại đúng (2.7). Ma trận đầy đủ A không đối xứng, nhưng hệ rút gọn theo các ẩn thuộc Ω thì có (Hình 3.3).

2.1.7 Tính chất của hệ và cách giải

Hệ rút gọn cho các ẩn trong Ω có ma trận $A_\Omega = 4I - \mathrm{Adj}_\Omega$ (Adj_Ω là ma trận kề giữa các điểm của Ω). Ma trận này đối xứng, chéo trội (tổng trị tuyệt đối ngoài đường chéo của mỗi hàng không vượt quá 4), chéo trội chặt ở các hàng kề $\partial\Omega$ và bất khả quy nếu mỗi thành phần liên thông của Ω chạm $\partial\Omega$. Do đó A_Ω *xác định dương*, nghiệm tồn tại và duy nhất; ma trận cực thưa (tối đa 5 phần tử khác 0 mỗi hàng). Mỗi kênh màu cho một hệ riêng với cùng A nhưng vế phải $\mathbf{b}$ khác nhau.

Bảng 2.1: Các cách giải hệ Poisson trên lưới 2D với n ẩn [1, 2].

Phương pháp	Chi phí	Nhận xét
Jacobi / Gauss–Seidel	$O(n^2)$	$O(n)$ vòng lặp, mỗi vòng $O(n)$; đơn giản nhưng hội tụ chậm
Conjugate gradient	$O(n^{3/2})$	$O(\sqrt{n})$ vòng lặp; cần A đối xứng xác định dương
LU / Cholesky thưa (trực tiếp)	$O(n^{3/2})$	Với thứ tự khử phù hợp; cho nghiệm chính xác, dùng lại được phân rã cho nhiều vế phải
Multigrid	$O(n)$	Nhanh nhất về lý thuyết, hiện thực phức tạp hơn

Báo cáo dùng bộ giải trực tiếp (LU thưa SuperLU [6] qua hàm `spsolve` của SciPy [7]), vì số ẩn cỡ 10^4–10^5 đủ nhỏ để giải chính xác trong chưa đầy một giây (Bảng 4.5), không phải chọn tiêu chí dừng hay tinh chỉnh bộ tiền điều kiện.

2.2 Hai phương pháp đối chứng

2.2.1 Dán trực tiếp bằng mặt nạ (alpha blending)

Với mặt nạ $M \in [0,1]$ đã đặt đúng vị trí trên ảnh nền,

$$f = M \odot g + (1 - M) \odot f^*. \tag{2.9}$$

Chi phí $O(H_s W_s)$ và không có tham số, nhưng f nhảy một khoảng bằng $g - f^*$ tại biên của mặt nạ: mọi chênh lệch màu giữa hai ảnh đều hiện thành đường viền.

2.2.2 Laplacian pyramid blending

Phương pháp của Burt và Adelson [8, 9] tách mỗi ảnh thành các băng tần rồi trộn từng băng tần với độ mờ của mặt nạ khác nhau. Tháp Gauss được tạo bởi $G_0 = I$ và $G_{k+1} = \downarrow\big(w * G_k\big)$, với $w = \frac{1}{16}[1\ 4\ 6\ 4\ 1]$ áp dụng lần lượt theo hàng và cột, $\downarrow$ giữ lại các hàng, cột chẵn. Phép phóng đại là $\mathrm{EXPAND}(G) = 4\,\big(w * \uparrow G\big)$, với $\uparrow$ chèn số 0 giữa các mẫu. Tháp Laplace gồm các băng tần

$$L_k = G_k - \mathrm{EXPAND}(G_{k+1}) \quad (0 \le k < N), \qquad L_N = G_N. \tag{2.10}$$

Gọi G_k^M là tháp Gauss của mặt nạ; mỗi băng tần được trộn riêng rồi tái tạo từ mức thô nhất:

$$L_k^f = G_k^M \odot L_k^g + \big(1 - G_k^M\big) \odot L_k^{f^*}, \qquad \tilde{G}_N = L_N^f, \quad \tilde{G}_k = L_k^f + \mathrm{EXPAND}(\tilde{G}_{k+1}), \quad f = \tilde{G}_0. \tag{2.11}$$

Băng tần k chứa cấu trúc cỡ 2^k điểm ảnh và mặt nạ ở mức đó cũng bị làm mờ cùng cỡ: chi tiết mịn được trộn với chuyển tiếp hẹp, thành phần màu–độ sáng (tần số thấp) được trộn với chuyển tiếp rộng cỡ 2^N điểm ảnh. Phương pháp *che mờ* đường ghép, nhưng *không thay đổi mức màu bên trong* Ω nên mảng nền của nguồn vẫn giữ màu riêng của nó.

2.2.3 So sánh ba cách tiếp cận

Bảng 2.2 đặt ba phương pháp cạnh nhau theo các tiêu chí sẽ được dùng để đánh giá ở Chương 4 và 5.

2.3 Cách đánh giá chất lượng ghép ảnh

2.3.1 Tiêu chí định tính

Không tồn tại ảnh ghép "đúng" để đối chiếu, nên cách đánh giá chính là *định tính*, theo ba tiêu chí: (T1) *tính liên tục tại biên:* không còn viền hay quầng sáng quanh đối tượng; (T2) *nhất quán màu và ánh sáng* giữa đối tượng và nền; (T3) *bảo toàn chi tiết và kết cấu* của đối tượng (không bị mờ, bão hòa hay trong suốt ngoài ý muốn).

Bảng 2.2: So sánh về mặt lý thuyết giữa ba phương pháp ghép ảnh.

	Dán trực tiếp	**Laplacian blending**	**Poisson (miền gradient)**
Miền xử lý	Điểm ảnh	Đa phân giải (băng tần)	Gradient
Giữ từ ảnh nguồn	Cường độ tuyệt đối	Cường độ, chuyển tiếp mượt theo băng tần	Gradient (cấu trúc); cường độ được suy ra
Màu bên trong Ω	Giữ nguyên	Gần như giữ nguyên	Thay đổi để khớp biên (nội suy điều hòa)
Chênh lệch màu nền	Lộ thành viền	Che mờ, không loại bỏ	Loại bỏ nhờ khớp biên
Chi phí	$O(H_s W_s)$	$O(H_s W_s)$ nhân số mức	Giải hệ thưa $\lvert\Omega\rvert$ ẩn cho mỗi kênh
Rủi ro chính	Viền rõ	Quầng sáng quanh vật	Lan màu, bão hòa, mờ ảo (gradient trộn)

2.3.2 Chỉ số đường ghép S

Để hỗ trợ cho (T1), báo cáo dùng một chỉ số tự định nghĩa. Gọi Ω_b là *dải trong* sát biên của Ω có độ rộng $w = 5$ điểm ảnh (khoảng cách Euclid tới điểm ngoài Ω không quá w), và $Z = \{q : M_q < 0.01\}$ là các điểm nền thuần. Màu "bên ngoài" cục bộ tại p là

$$\hat{f}(p) = \frac{\sum_{q \in Z} G_\sigma(p-q)\, f_q}{\sum_{q \in Z} G_\sigma(p-q)}, \qquad \sigma = 6, \tag{2.12}$$

với G_σ là nhân Gauss. Chỉ số đường ghép (thang 0–255) là

$$S(f) = \frac{255}{3\,|\Omega'_b|} \sum_{p \in \Omega'_b} \left\| f_p - \hat{f}(p) \right\|_1, \tag{2.13}$$

trong đó $\Omega'_b \subset \Omega_b$ chỉ gồm các điểm *nền của ảnh nguồn* (độ lớn gradient của nguồn nhỏ hơn trung vị trên Ω). S nhỏ nghĩa là dải sát biên có màu gần màu nền ngay bên ngoài, tức không có "mảng" khác màu. Việc chỉ xét điểm nền của nguồn là để biên *thật* của đối tượng (ví dụ lông đen của chim cánh cụt sát mặt nạ) không bị tính nhầm là đường ghép; kết quả trên toàn dải, ký hiệu S_{all}, cũng được báo cáo. Giá trị của chính ảnh nền (thay f bằng f^* trong (2.13)) là mức biến thiên tự nhiên để tham chiếu. Đây là chỉ số bổ trợ do báo cáo tự đặt ra, không thay thế quan sát trực quan, và phụ thuộc vào cách vẽ mặt nạ.

2.3.3 Thí nghiệm có đáp án và các độ đo PSNR, SSIM, MAE

Từ tính chất (iv) ở Mục 2.1.4, ta dựng một phép thử *có đáp án*: lấy một ảnh nền f_0 làm đáp án, tạo "ảnh nguồn" $g = f_0 + \eta$ trên một vùng elip Ω với η là sai lệch màu hằng số hoặc tuyến tính, rồi ghép g trở lại f_0 bằng mỗi phương pháp. Phương pháp nào loại bỏ được sai lệch thì sẽ trả về f_0. Kết quả được đo bằng

$$\mathrm{PSNR} = 10\log_{10}\frac{255^2}{\mathrm{MSE}}, \qquad \mathrm{MAE}_\Omega = \frac{1}{3|\Omega|}\sum_{p\in\Omega}\|f_p - f_{0,p}\|_1, \tag{2.14}$$

và chỉ số SSIM [10] (tính bằng scikit-image [11]). PSNR, SSIM đo trên hộp bao của Ω; MAE đo riêng trong Ω. Phép thử này kiểm tra hiện thực và lý thuyết, không đại diện cho chất lượng trên ảnh thật.

2.4 Phần 2: Cơ sở hình học, vì sao là bài toán homography

Vì sao một phép biến đổi toàn cục cho mỗi ảnh là đủ để ghép panorama?

Khi camera chỉ xoay quanh tâm quang học, độ sâu của điểm cảnh bị triệt tiêu trong phép chiếu giữa hai ảnh, nên ánh xạ giữa chúng là cùng một ma trận 3×3 (homography) cho mọi cảnh, phẳng hay không. Vì vậy ước lượng homography từ các cặp điểm tương ứng rồi nối chúng lại là đủ để đưa mọi ảnh về một hệ tọa độ chung.

Camera lỗ kim. Điểm 3D $\mathbf{X} = (X, Y, Z)^\top$ trong hệ tọa độ camera (x sang phải, y xuống dưới, Z dọc trục quang) được chiếu thành điểm ảnh

$$\tilde{\mathbf{x}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K\mathbf{X}, \qquad K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{2.15}$$

trong đó f là tiêu cự tính bằng pixel, (c_x, c_y) là điểm chính, và "$\sim$" nghĩa là bằng nhau sai khác một hệ số khác 0. Thông tin bị mất là độ sâu Z: mọi điểm trên tia $\lambda\mathbf{X}$ ($\lambda > 0$) rơi vào cùng một pixel.

Hai camera cùng tâm quang học. Đặt tâm chung ở gốc tọa độ thế giới, camera j có hướng R_j (từ camera sang thế giới). Điểm $\mathbf{X}_w$ có tọa độ camera $\mathbf{X}_j = R_j^\top \mathbf{X}_w$, nên

$$\tilde{\mathbf{x}}_1 \sim K_1 R_1^\top \mathbf{X}_w, \qquad \tilde{\mathbf{x}}_2 \sim K_2 R_2^\top \mathbf{X}_w.$$

Từ hệ thức thứ nhất, $\mathbf{X}_w = \lambda R_1 K_1^{-1}\tilde{\mathbf{x}}_1$ với $\lambda > 0$ chưa biết. Thế vào hệ thức thứ hai:

$$\tilde{\mathbf{x}}_2 \sim \lambda\, K_2 R_2^\top R_1 K_1^{-1}\,\tilde{\mathbf{x}}_1 \quad\Longrightarrow\quad \boxed{\tilde{\mathbf{x}}_2 \sim H_{12}\,\tilde{\mathbf{x}}_1, \qquad H_{12} = K_2\, R_2^\top R_1\, K_1^{-1}} \tag{2.16}$$

Hệ số λ biến mất vì đã nằm trong dấu "$\sim$": **độ sâu bị triệt tiêu**. Với camera chỉ xoay, ánh xạ giữa hai ảnh là cùng một ma trận 3×3 cho mọi cảnh, phẳng hay không. Đó là lý do một phép biến đổi toàn cục cho mỗi ảnh là đủ để ghép panorama.

Khi camera còn tịnh tiến t thì $\tilde{\mathbf{x}}_2 \sim K_2 R_2^\top (\lambda R_1 K_1^{-1} \tilde{\mathbf{x}}_1 - \mathbf{t})$ và λ không còn triệt tiêu: ánh xạ phụ thuộc độ sâu, đó là **thị sai (parallax)**. Ở xấp xỉ bậc nhất, điểm ở độ sâu Z bị lệch $f\,t/Z$ pixel. Với $f = 1944$ px (ảnh rộng 800 px, góc nhìn ngang khoảng 23°), $t = 1$ cm: độ lệch là 19 px khi $Z = 1$ m nhưng chỉ 1 px khi $Z = 20$ m. Vì vậy ảnh chụp cầm tay cảnh xa vẫn ghép tốt, còn vật ở gần để lại bóng ma. Ánh xạ vẫn **chính xác** nếu cảnh là một mặt phẳng: với mặt phẳng $\mathbf{n}^\top \mathbf{X}_1 = d$ trong hệ camera 1 ta có $\mathbf{n}^\top \mathbf{X}_1 / d = 1$, do đó $\mathbf{X}_2 = R\mathbf{X}_1 + \mathbf{t} = (R + \mathbf{t}\mathbf{n}^\top / d)\mathbf{X}_1$ và $H = K_2 (R + \mathbf{t}\mathbf{n}^\top / d) K_1^{-1}$. Bộ ảnh `newspaper` (tờ báo) thuộc trường hợp này.

Các tính chất mà pipeline dùng đến.

1. H chỉ xác định sai khác một hệ số, nên có **8 bậc tự do**. Mỗi cặp điểm tương ứng cho hai phương trình (u và v), vì vậy cần ít nhất **4 cặp**; dùng nhiều hơn để trung bình hóa nhiễu.
2. Các homography khép kín với phép nhân và nghịch đảo (nhóm $PGL(3)$): $H_{13} = H_{23} H_{12}$ và $H_{21} = H_{12}^{-1}$. Do đó các ước lượng theo từng cặp ảnh có thể được **nối** lại để đưa mọi ảnh về hệ của một ảnh tham chiếu.
3. Đường thẳng được ánh xạ thành đường thẳng; khoảng cách và góc thì không. Hàng thứ ba $\mathbf{h}_3^\top$ của H mô tả phối cảnh: đường $\mathbf{h}_3^\top \tilde{\mathbf{x}} = 0$ bị đẩy ra vô cực.

Kiểm tra bằng số. Module `synthetic.py` dựng các góc nhìn của một ảnh lớn bằng camera ảo chỉ xoay và trả về homography đúng $H_{ij} = K_j R_j^\top R_i K_i^{-1}$. Warp góc nhìn 0 bằng H_{01} rồi so với góc nhìn 1 trong vùng chồng lấn cho sai khác trung bình 0.7 mức xám (so với 30.2 khi không warp; phần còn lại do ảnh bị nội suy song tuyến hai lần), và quy tắc nhân được thỏa mãn đến sai số máy: $\max |H_{02} - H_{12} H_{01}| = 2.8 \cdot 10^{-14}$. Các góc nhìn tổng hợp này cũng là cơ sở của nghiên cứu có kiểm soát ở Mục 4.10, vì chỉ với chúng đáp án mới biết chính xác.

2.5 Trích xuất và mô tả đặc trưng

Cần gì ở một đặc trưng? Các cặp điểm tương ứng là đầu vào duy nhất của khâu hình học, nên chất lượng của chúng quyết định mọi thứ phía sau. Đặc trưng tốt phải *lặp lại được* (cùng một điểm của cảnh được phát hiện ở cả hai ảnh dù góc nhìn, góc xoay, tỉ lệ, độ sáng và nhiễu thay đổi), *phân biệt được* (bộ mô tả cách xa bộ mô tả của mọi điểm khác, nếu không việc so khớp mơ hồ), *định vị chính xác* (sai số ε px ở điểm đặc trưng trở thành sai số cùng cỡ trong H), và *nhiều, phân bố đều* trên vùng chồng lấn (nếu các cặp khớp dồn vào một vùng nhỏ thì H điều kiện kém, xem Mục 2.7). Giữa hai ảnh của một panorama, nội dung bị dịch và xoay (roll), có thể bị phóng hoặc thu (đổi tiêu cự hay khoảng cách) và đổi độ sáng (tự động phơi sáng); cặp detector/descriptor vì vậy nên bất biến với **tịnh tiến, xoay, tỉ lệ và gain/bias của cường độ**.

2.5.1 Harris + MOPS (tự cài đặt)

Một cửa sổ nhỏ quanh (x, y) định vị được tốt đến mức nào [12, 13]? Dịch cửa sổ một đoạn $\Delta = (\Delta x, \Delta y)$ và đo mức thay đổi có trọng số w; khai triển Taylor bậc một $I(x + \Delta) \approx I(x) + \nabla I^\top \Delta$ cho

$$E(\Delta) = \sum_{(x,y)} w(x, y) \left[I(x+\Delta x, y+\Delta y) - I(x, y)\right]^2 \approx \Delta^\top M \, \Delta, \qquad M = \sum w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}.$$

Ma trận cấu trúc M đối xứng, nửa xác định dương; các giá trị riêng $\lambda_1 \geq \lambda_2 \geq 0$ là độ cong của E theo hai hướng chính. Vùng phẳng: $\lambda_1 \approx \lambda_2 \approx 0$. **Cạnh**: $\lambda_1 \gg \lambda_2 \approx 0$ (dịch dọc theo cạnh không làm ảnh đổi, vấn đề khẩu độ). **Góc**: cả λ_2 cũng lớn. Vậy góc là điểm có giá trị riêng *nhỏ* vẫn lớn. Để khỏi tính giá trị riêng ở từng pixel, MOPS dùng trung bình điều hòa

$$f_{HM} = \frac{\det M}{\operatorname{tr} M} = \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2} = \frac{1}{1/\lambda_1 + 1/\lambda_2}, \qquad \tfrac{1}{2}\lambda_2 \leq f_{HM} \leq \lambda_2 \tag{2.17}$$

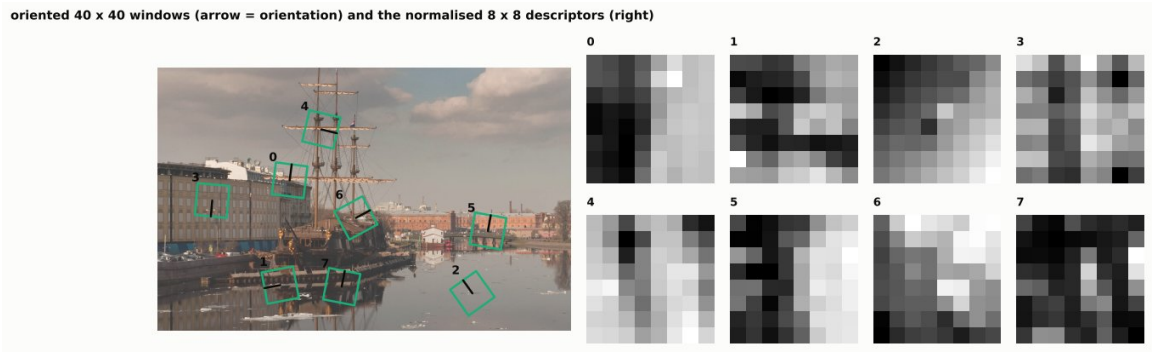
(vì $1/\lambda_1 \leq 1/\lambda_2$ nên $1/\lambda_2 \leq 1/\lambda_1 + 1/\lambda_2 \leq 2/\lambda_2$). Nó nằm trong khoảng hệ số 2 của giá trị riêng nhỏ, không cần phân rã riêng và không cần hằng số k như $\det M - k \operatorname{tr}^2 M$ của Harris–Stephens. Đạo hàm được lấy sau khi làm trơn Gaussian $\sigma_d = 1$; M được tích lũy bằng cửa sổ Gaussian $\sigma_i = 1.5$ (trọng số trơn giữ cho đáp ứng bất biến với phép xoay).

Khi ảnh xoay, M biến thành $R^\top M R$ nên giá trị riêng không đổi: **detector bất biến với phép xoay**. Nó **không bất biến tỉ lệ**: một góc nhìn từ xa trở thành đường cong giống cạnh khi phóng to. Tỉ lệ được xử lý bằng kim tự tháp Gaussian 4 bậc, hệ số 2. Trên mỗi bậc, đáp ứng được tính, các cực đại cục bộ 3×3 trên ngưỡng tương đối 1% của đáp ứng lớn nhất được tinh chỉnh dưới pixel bằng parabola qua ba mẫu quanh cực đại (theo từng trục). Cuối cùng **ANMS** (*adaptive non-maximal suppression*) chọn điểm: điểm i có bán kính triệt tiêu $r_i = \min_j \{\|\mathbf{x}_i - \mathbf{x}_j\| : \; c\, f_j > f_i\}$ với $c = 0.9$, tức khoảng cách đến điểm mạnh hơn rõ rệt gần nhất, và N điểm có bán kính lớn nhất được giữ. Các điểm mạnh nhất của một ảnh dồn vào vài chỗ nhiều kết cấu; ANMS rải đều chúng, đúng điều mà H cần.

Bộ mô tả MOPS. (i) *Hướng*: gradient của ảnh làm mờ mạnh ($\sigma_o = 4.5$) ổn định trước nhiễu và sai lệch định vị; góc $\theta = \operatorname{atan2}(\partial_y \tilde{I}, \partial_x \tilde{I})$ tại điểm đặc trưng là hướng chủ đạo, và xoay lưới lấy mẫu theo θ cho bất biến xoay. (ii) *Lấy mẫu*: lưới 8×8 với bước 5 px (cửa sổ có hướng 40×40 px) được lấy mẫu song tuyến từ bậc kim tự tháp sau khi làm mờ $\sigma = 2$; phép làm mờ là bộ lọc khử răng cưa cho việc lấy mẫu thưa. (iii) *Chuẩn hóa bias/gain*: mỗi patch d được thay bằng $\hat{d} = (d - \bar{d})/s_d$ (s_d là độ lệch chuẩn của 64 mẫu). Với $I \mapsto aI + b$, $a > 0$, trung bình thành $a\bar{I} + b$ và độ lệch chuẩn thành $a\, s_I$, nên $\hat{d}$ **không đổi**. (iv) *Khoảng cách*: với patch đã chuẩn hóa ($\sum_k \hat{d}_k = 0$, $\sum_k \hat{d}_k^2 = n = 64$)

$$\|\hat{d}_a - \hat{d}_b\|^2 = \sum_k \hat{d}_{a,k}^2 + \sum_k \hat{d}_{b,k}^2 - 2\sum_k \hat{d}_{a,k}\hat{d}_{b,k} = 2n\,(1 - \rho_{ab}),$$

với ρ_{ab} là hệ số tương quan chéo chuẩn hóa (NCC) của hai patch: so khớp bằng khoảng cách L_2 chính là so khớp theo NCC. Kiểm tra bằng số: $\hat{d}$ không đổi khi $I \mapsto 0.4I + 0.2$ (sai khác trung vị $1.8 \cdot 10^{-5}$, tối đa $2.3 \cdot 10^{-3}$) và đẳng thức trên đúng với sai lệch tối đa $2.2 \cdot 10^{-2}$ trên khoảng cách bình phương trung bình khoảng 100.



Hình 2.3: Bộ mô tả MOPS: các cửa sổ 40×40 px có hướng (đoạn thẳng là hướng) trên ảnh `boat1`, và các patch 8×8 đã chuẩn hóa tương ứng (bên phải, đánh số theo cửa sổ).

2.5.2 SIFT (OpenCV)

Không gian tỉ lệ $L(x, y, \sigma) = G_\sigma * I$ [14]. Laplacian *chuẩn hóa theo tỉ lệ* $\sigma^2\nabla^2 L$ độc lập với phép phóng đại: nếu ảnh 2 là ảnh 1 phóng a lần thì $L_2(ax, a\sigma) = L_1(x, \sigma)$, suy ra $\nabla^2 L_2 = a^{-2}\nabla^2 L_1$ và $(a\sigma)^2\nabla^2 L_2 = \sigma^2\nabla^2 L_1$. Các cực trị theo (x, y, σ) của đại lượng này (blob) xuất hiện ở vị trí **và tỉ lệ** tương ứng trong cả hai ảnh. Vì Gaussian là nghiệm của phương trình nhiệt, $\partial G/\partial\sigma = \sigma\nabla^2 G$, nên $G_{k\sigma} - G_\sigma \approx (k-1)\sigma^2\nabla^2 G$: hiệu hai Gaussian (DoG) là Laplacian chuẩn hóa nhân với hằng số $(k-1)$, và rẻ hơn nhiều so với tính Laplacian ở từng tỉ lệ. Điểm đặc trưng là cực trị trong lân cận 26 điểm của chồng DoG, được tinh chỉnh bằng xấp xỉ bậc hai theo (x, y, σ); điểm tương phản thấp (ngưỡng **tuyệt đối** `contrastThreshold` $= 0.04$ trên $|D|$) và điểm nằm trên cạnh (tỉ lệ độ cong chính) bị loại. *Hướng* là đỉnh của histogram 36 ngăn hướng gradient. *Bộ mô tả*: gradient trong cửa sổ xoay theo hướng đó và co theo σ, gom vào 4×4 ô × 8 hướng = **128 số**, chuẩn hóa độ dài đơn vị, cắt ở 0.2 rồi chuẩn hóa lại (phép chuẩn hóa loại gain; phép cắt giảm ảnh hưởng của vài gradient quá mạnh do bão hòa hay chiếu sáng phi tuyến).

2.5.3 ORB (OpenCV)

oFAST [15]: pixel p là góc FAST nếu một cung liên tiếp ít nhất 9 trong 16 pixel trên đường tròn bán kính 3 xung quanh đều sáng hơn $I_p + t$ hoặc đều tối hơn $I_p - t$, với $t = 20$ mức xám (`fastThreshold`, ngưỡng **tuyệt đối**). Các góc được xếp hạng bằng độ đo Harris, N điểm tốt nhất được giữ; việc phát hiện chạy trên kim tự tháp 8 bậc, hệ số 1.2. Hướng là hướng từ góc tới **tâm cường độ** của patch tròn, $\theta = \operatorname{atan2}(m_{01}, m_{10})$ với $m_{pq} = \sum x^p y^q I(x, y)$. *rBRIEF*: 256 phép so sánh nhị phân $[\, I(\mathbf{p} + \mathbf{a}_i) < I(\mathbf{p} + \mathbf{b}_i) \,]$ trên patch đã làm trơn, mẫu cặp điểm xoay theo θ. Bộ mô tả là **32 byte**, so sánh bằng

khoảng cách Hamming (XOR + đếm bit), cực rẻ. Phép so sánh $I_a < I_b$ bất biến với mọi biến đổi đơn điệu của cường độ, nhưng *ngưỡng của detector* thì không (Mục 4.10).

Bảng 2.3: Ba bộ trích xuất đặc trưng.

	SIFT (OpenCV)	**ORB (OpenCV)**	**Harris + MOPS (tự viết)**
Bộ phát hiện	cực trị DoG (blob)	góc oFAST	góc Harris (trung bình điều hòa) + ANMS
Tỉ lệ	liên tục, ≈ 3 bậc mỗi octave	8 bậc, hệ số 1.2	4 bậc, hệ số 2
Hướng	hướng gradient chủ đạo	tâm cường độ	hướng gradient của ảnh làm mờ
Bộ mô tả	128 số thực, histogram gradient	256 bit, so sánh cường độ	64 số thực, patch chuẩn hóa
Khoảng cách	L_2	Hamming	L_2
Gain / bias	chuẩn hóa độ dài; ngưỡng tương phản tuyệt đối	các phép so sánh bất biến; ngưỡng FAST tuyệt đối	chính xác (patch chuẩn hóa); ngưỡng tương đối

2.6 So khớp đặc trưng

Láng giềng gần nhất bằng một phép nhân ma trận. Với bộ mô tả $\mathbf{a}_i$ của ảnh 1 và $\mathbf{b}_j$ của ảnh 2,

$$\|\mathbf{a}_i - \mathbf{b}_j\|^2 = \|\mathbf{a}_i\|^2 + \|\mathbf{b}_j\|^2 - 2\,\mathbf{a}_i^\top \mathbf{b}_j,$$

nên toàn bộ ma trận khoảng cách bình phương $N_1 \times N_2$ gồm hai vector chuẩn bình phương và số hạng $-2AB^\top$: một phép nhân ma trận (BLAS) thay vì $N_1 N_2$ phép tính khoảng cách trong Python. Sai số làm tròn có thể làm khoảng cách rất nhỏ hơi âm nên trước khi lấy căn nó được chặn ở 0; các hàng được xử lý theo khối 1024 để giới hạn bộ nhớ; `argpartition` tìm hai phần tử nhỏ nhất của một hàng trong $O(N_2)$, không cần sắp xếp. **Khoảng cách Hamming dùng cùng mẹo:** với vector bit $\mathbf{a}, \mathbf{b} \in \{0,1\}^n$ ta có $|a_k - b_k| = (a_k - b_k)^2$, nên $d_H(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|^2$; các byte ORB được bung thành 256 bit rồi dùng lại đúng đoạn mã trên. Kết quả khớp với tìm kiếm vét cạn: sai khác tối đa 10^{-6} với L_2 và đúng tuyệt đối với Hamming.

Ratio test và lọc một–một. Láng giềng gần nhất là sai mỗi khi điểm không có tương ứng (nằm ngoài vùng chồng lấn, bị che, hoặc cấu trúc lặp lại), và riêng khoảng cách d_1 không phân biệt được điều đó vì bộ mô tả của hai ảnh không so sánh được ở giá trị tuyệt đối. **Ratio test của Lowe** [14] chỉ nhận cặp khớp khi $d_1/d_2 < \tau$, với d_2 là khoảng cách tới láng giềng thứ hai. Với điểm không có tương ứng, d_1 và d_2 là hai mẫu của cùng một phân phối (khoảng cách tới một bộ mô tả không liên quan) nên tỉ số gần 1; với cặp đúng, d_1 nhỏ còn d_2 điển hình nên tỉ số nhỏ. Ngưỡng τ đánh đổi số cặp đúng lấy độ tinh khiết.

Lọc một–một thêm một ràng buộc vật lý: trong cảnh tĩnh, một điểm của ảnh 2 là ảnh của nhiều nhất một điểm của ảnh 1; khi nhiều đặc trưng của ảnh 1 cùng chọn một đối tác, chỉ giữ cặp rõ ràng nhất (tỉ số nhỏ nhất). Thiếu phép lọc này, kết cấu lặp (dòng chữ, cửa sổ, gạch) có thể tạo ra nhiều cặp khớp dồn về cùng một điểm (ví dụ ở Mục 4.7.4).

Tác dụng của ratio test được đo trên một cặp góc nhìn tổng hợp có hình học đã biết ở Mục 4.7.2 (Hình 4.10, Bảng 4.10).

2.7 Ước lượng homography

2.7.1 Direct Linear Transform (DLT)

Một cặp $\mathbf{x} = (x, y, 1)^\top \leftrightarrow \mathbf{x}' = (u, v, 1)^\top$ thỏa $\mathbf{x}' \sim H\mathbf{x}$ [16]. Trong tọa độ thuần nhất, "bằng nhau sai khác một hệ số" nghĩa là *song song*, tức tích có hướng bằng không: $\mathbf{x}' \times H\mathbf{x} = \mathbf{0}$. Với các hàng $\mathbf{h}_k^\top$ của H:

$$\mathbf{x}' \times H\mathbf{x} = \begin{bmatrix} v\,\mathbf{h}_3^\top\mathbf{x} - \mathbf{h}_2^\top\mathbf{x} \\ \mathbf{h}_1^\top\mathbf{x} - u\,\mathbf{h}_3^\top\mathbf{x} \\ u\,\mathbf{h}_2^\top\mathbf{x} - v\,\mathbf{h}_1^\top\mathbf{x} \end{bmatrix} = \mathbf{0}.$$

Ba phương trình này phụ thuộc tuyến tính (u·hàng 1 + v·hàng 2 + hàng 3 = 0), chỉ hai phương trình độc lập, đúng với "hai phương trình cho mỗi cặp" ở Mục 2.4. Viết theo $\mathbf{h} = (\mathbf{h}_1; \mathbf{h}_2; \mathbf{h}_3) \in \mathbb{R}^9$:

$$\begin{bmatrix} 0 & 0 & 0 & -x & -y & -1 & vx & vy & v \\ x & y & 1 & 0 & 0 & 0 & -ux & -uy & -u \end{bmatrix} \mathbf{h} = \mathbf{0}. \tag{2.18}$$

Xếp $N \geq 4$ cặp ta được $A\mathbf{h} = \mathbf{0}$ với $A \in \mathbb{R}^{2N\times 9}$. Với $N = 4$, A có hạng 8 và không gian null của nó là nghiệm chính xác. Với nhiễu và $N > 4$ không có nghiệm chính xác, ta cực tiểu hóa sai số đại số $\min_{\|\mathbf{h}\|=1} \|A\mathbf{h}\|^2$. **Nghiệm là vector kỳ dị phải ứng với giá trị kỳ dị nhỏ nhất.** Thật vậy, đặt $A = U\Sigma V^\top$ và $\mathbf{c} = V^\top\mathbf{h}$; khi đó $\|A\mathbf{h}\|^2 = \sum_k \sigma_k^2 c_k^2$ với $\sum_k c_k^2 = \|\mathbf{h}\|^2 = 1$, là một trung bình có trọng số của các σ_k^2; nó nhỏ nhất, bằng $\sigma_{\min}^2$, khi toàn bộ trọng số dồn vào giá trị kỳ dị nhỏ nhất, $\mathbf{c} = \mathbf{e}_9$, tức $\mathbf{h} = \mathbf{v}_9$, cột cuối của V (`Vt[-1]` trong mã). ■ Kết quả khớp với `cv2.findHomography`: sai khác tối đa 10^{-5} với 4 điểm chính xác và RMS $5 \cdot 10^{-6}$ px với 60 điểm có nhiễu.

2.7.2 Chuẩn hóa Hartley

Các phần tử của A có độ lớn rất khác nhau (1, $x \sim 10^3$, $ux \sim 10^6$), nên A điều kiện xấu và các phương trình có tọa độ lớn lấn át những phương trình khác. Tệ hơn, sai số đại số phụ thuộc vào *hệ tọa độ*: dời gốc hoặc đổi đơn vị làm nghiệm cực tiểu thay đổi. Cách chữa là chuẩn hóa cả hai tập điểm bằng một phép đồng dạng,

$$\hat{\mathbf{x}} = T\mathbf{x}, \quad \hat{\mathbf{x}}' = T'\mathbf{x}', \qquad T: \text{ trọng tâm} \to \text{gốc, khoảng cách trung bình tới gốc} \to \sqrt{2},$$

giải $\hat{H}$ thỏa $\hat{\mathbf{x}}' \sim \hat{H}\hat{\mathbf{x}}$ rồi hoàn nguyên $H = T'^{-1}\hat{H}T$ (vì $T'\mathbf{x}' \sim \hat{H}T\mathbf{x} \iff \mathbf{x}' \sim T'^{-1}\hat{H}T\mathbf{x}$). T và T' do dữ liệu quyết định và biến đổi hiệp biến khi dời gốc hoặc đổi đơn vị, nên ước lượng không còn phụ thuộc hệ tọa độ. Thực nghiệm cho cả ba tác dụng: số điều kiện của A giảm từ $1.1 \cdot 10^8$ (tọa độ pixel thô) xuống $1.1 \cdot 10^3$; dời gốc tọa độ làm H thay đổi $2.4 \cdot 10^{-2}$ px khi không chuẩn hóa và $2.4 \cdot 10^{-13}$ px khi có; và với các cặp khớp dồn trong một dải hẹp (trường hợp hay gặp ở vùng chồng lấn hẹp), sai số của H trên toàn ảnh, trung bình 300 lần thử, là 28.2 px nếu không chuẩn hóa so với 7.9 px nếu có.

2.7.3 Khi nào H không duy nhất? Các cấu hình suy biến

H được xác định duy nhất khi và chỉ khi $\operatorname{rank} A = 8$, tức giá trị kỳ dị σ_8 (lớn thứ hai từ cuối) khác không. Giả sử mọi điểm $\mathbf{x}_i$ nằm trên một đường thẳng $\boldsymbol{\ell}$ ($\boldsymbol{\ell}^\top \mathbf{x}_i = 0$). Với mọi $\mathbf{a} \in \mathbb{R}^3$ và mọi α,

$$(H_0 + \alpha\, \mathbf{a}\boldsymbol{\ell}^\top)\, \mathbf{x}_i = H_0\mathbf{x}_i + \alpha\, \mathbf{a}\, (\boldsymbol{\ell}^\top \mathbf{x}_i) = H_0\mathbf{x}_i,$$

nên cả một họ homography 3 tham số tái tạo chính xác mọi cặp điểm: không gian null của A có chiều ≥ 4 (hạng ≤ 5) và nghiệm mà SVD trả về là tùy ý. Điều tương tự xảy ra khi các điểm của một ảnh co về một điểm. Cả hai tình huống đều có thật: kết cấu lặp (dòng chữ) cho các cặp khớp nằm trên một đường hoặc dồn về một điểm, và RANSAC đếm chúng thành một tập đồng thuận khổng lồ. Tỉ số σ_8/σ_1 của ma trận DLT chuẩn hóa (`dlt_conditioning`) đo điều này: khoảng 0.1 với điểm phân bố tốt (các cặp thật trong báo cáo cho 0.04–0.3) và ≈ 0 khi suy biến. Thí nghiệm: 12 điểm ở vị trí tổng quát cho $\sigma_8/\sigma_1 = 2.2 \cdot 10^{-1}$; 12 điểm thẳng hàng cho $4.8 \cdot 10^{-17}$, và hai nghiệm SVD khác nhau đều khớp 12 điểm thẳng hàng đến 10^{-12} px nhưng sai từ hàng trăm đến hàng nghìn pixel trên các điểm khác.

2.7.4 RANSAC

Các cặp khớp dự kiến luôn lẫn ngoại lai, và một ngoại lai thô đủ làm hỏng phép khớp bình phương tối thiểu (điểm sụp đổ bằng không). RANSAC (Fischler & Bolles, 1981) [17] lặp lại: rút một **mẫu tối thiểu** gồm 4 cặp, tính H chính xác, đếm các cặp có sai số tái chiếu $\|\pi(H\mathbf{x}_i) - \mathbf{x}'_i\|$ dưới ngưỡng t (*tập đồng thuận*), giữ giả thuyết tốt nhất.

Cần bao nhiêu mẫu? Nếu tỉ lệ nội điểm là w, một mẫu 4 cặp sạch với xác suất w^4; xác suất không mẫu nào trong N mẫu độc lập sạch là $(1 - w^4)^N$. Đòi hỏi ít nhất một mẫu sạch với xác suất p:

$$1 - (1 - w^4)^N \geq p \quad \iff \quad N \;\geq\; \frac{\ln(1-p)}{\ln(1-w^4)}. \tag{2.19}$$

N tăng rất nhanh khi w giảm, nên được cập nhật thích nghi theo tỉ lệ nội điểm tốt nhất đã tìm thấy và bị chặn ở 3000. Mẫu suy biến bị bỏ qua: ba điểm gần thẳng hàng ($\sin\angle < 0.02$ ở một trong hai ảnh) hoặc hai ảnh bất đồng về chiều của một tam giác (nghĩa là có phép phản xạ).

Ngưỡng. Nếu sai số của một cặp đúng là Gauss với độ lệch chuẩn σ trên mỗi tọa độ thì $e^2/\sigma^2 \sim \chi^2_2$ và $P(e < t) = 1 - e^{-t^2/2\sigma^2}$: ngưỡng $t = 3$ px giữ lại 98.9% số cặp đúng khi $\sigma = 1$ px. Ngưỡng nhỏ hơn loại bỏ cả cặp tốt; ngưỡng lớn hơn nhận cả các ngoại lai tình cờ nằm gần mô hình.

Sau vòng lặp. Tập đồng thuận tốt nhất được khớp lại bằng DLT trên mọi nội điểm, tập nội điểm được tính lại, và lặp đến khi tập không đổi; sau đó sai số hình học được tinh chỉnh (mục tiếp theo) và nội điểm được đếm lần cuối. Công thức (2.19) và độ bền của RANSAC với ngoại lai được kiểm chứng bằng thực nghiệm ở Mục 4.7.3 (Bảng 4.11).

2.7.5 Levenberg–Marquardt trên sai số hình học

Phần dư của phương trình DLT đầu tiên là $v\,\mathbf{h}_3^\top\mathbf{x} - \mathbf{h}_2^\top\mathbf{x} = w\,(v - v_{\text{pred}})$, với $w = \mathbf{h}_3^\top\mathbf{x}$ là tọa độ thuần nhất của điểm dự đoán: sai số đại số bằng sai số hình học **nhân một trọng số phụ thuộc điểm** w_i. Các trọng số khác nhau khi thành phần phối cảnh của H mạnh, nên nghiệm DLT không phải ước lượng hợp lý cực đại. Với nhiễu Gauss đẳng hướng ở ảnh thứ hai, ước lượng hợp lý cực đại cực tiểu hóa

$$\min_H \sum_i \left\| \pi(H\mathbf{x}_i) - \mathbf{x}'_i \right\|^2, \qquad \pi(p_1, p_2, p_3) = (p_1/p_3,\ p_2/p_3).$$

Đây là bài toán bình phương tối thiểu phi tuyến theo 8 phần tử tự do ($h_{33} = 1$). Với $\mathbf{p} = H\mathbf{x}$, $\hat{u} = p_1/p_3$, $\hat{v} = p_2/p_3$, các hàng của ma trận Jacobi của phần dư $\mathbf{r}_i = (\hat{u} - u,\ \hat{v} - v)$ là $\partial\hat{u}/\partial(h_{11}, h_{12}, h_{13}) = (x, y, 1)/p_3$, $\partial\hat{u}/\partial(h_{31}, h_{32}) = -\hat{u}\,(x, y)/p_3$, và tương tự cho $\hat{v}$ với (h_{21}, h_{22}, h_{23}); các phần tử còn lại bằng 0. Levenberg–Marquardt giải $\big(J^\top J + \lambda\,\mathrm{diag}(J^\top J)\big)\boldsymbol{\delta} = -J^\top\mathbf{r}$: với λ nhỏ đó là bước Gauss–Newton, với λ lớn là bước giảm gradient ngắn; λ chia 10 sau một bước thành công và nhân 10 sau một bước thất bại. Thuật toán chạy trong tọa độ đã chuẩn hóa Hartley, xuất phát từ nghiệm DLT. Ảnh hưởng của bước này trên dữ liệu thật được báo cáo ở Mục 4.7.3 (Bảng 4.12).

2.7.6 Cặp ảnh nào chồng lấn? Ba phép kiểm tra

Không phải cặp ảnh nào của một bộ cũng chồng lấn, mà RANSAC trả về *một* mô hình nào đó cho bất kỳ cặp nào. Một cặp chỉ được chấp nhận là chồng lấn khi:

1. có ít nhất 20 nội điểm;
2. các nội điểm xác định duy nhất H: $\sigma_8/\sigma_1 \geq 10^{-3}$ cho ma trận DLT của chúng (lập luận suy biến ở trên);
3. H cho dấu chân hợp lý (`footprint_ok`): không điểm nào của ảnh bị đẩy ra vô cực (dấu của $\mathbf{h}_3^\top\mathbf{x}$ ở bốn góc như nhau, nên ảnh của hình chữ nhật là một tứ giác lồi), giữ nguyên chiều (diện tích có dấu theo công thức dây giày dương, không có phản xạ) và diện tích thay đổi không quá 25 lần.

Ba phép kiểm tra là lưới an toàn chứ không phải bảo đảm: bộ `newspaper` cho thấy chúng chặn được cặp giả do chữ lặp (Mục 4.7.4), còn nghiên cứu có kiểm soát ở Mục 4.10 đếm các cặp qua được chúng mà hình học vẫn sai từ 3 px trở lên (*lỗi thầm lặng*).

2.8 Căn chỉnh nhiều ảnh: đồ thị chồng lấn, warp ngược và canvas

2.8.1 Đồ thị chồng lấn, cây khung và ảnh tham chiếu

Các nút là các ảnh; một cạnh nối hai ảnh qua được cả ba phép kiểm tra ở Mục 2.7.6, với trọng số là số nội điểm. Nối *tất cả* các homography theo cặp sẽ tích lũy và trộn lẫn sai số. Thay vào đó:

1. **cây khung lớn nhất** (Kruskal, cạnh nặng trước, không tạo chu trình) giữ $N-1$ cặp tin cậy nhất nối mọi ảnh, nên mỗi ảnh có đúng một đường tới ảnh tham chiếu;
2. **ảnh tham chiếu** là tâm của cây (số bước nhảy tối đa nhỏ nhất, hòa thì chọn ảnh có nhiều nội điểm hơn): nếu mỗi bước đóng góp một sai số độc lập có phương sai σ^2, ảnh cách k bước có phương sai $\approx k\sigma^2$, và tâm cực tiểu hóa trường hợp xấu nhất;
3. $H_{k\to\mathrm{ref}} = H_{p\to\mathrm{ref}}\,H_{k\to p}$ dọc theo đường đi, với p là cha của k;
4. ảnh không có liên kết hợp lệ với thành phần chính bị loại và được báo cáo; thứ tự đưa ảnh vào không ảnh hưởng.

Không có đóng chu trình (tối ưu toàn cục trên *mọi* cặp, tức bundle adjustment): các cặp không nằm trong cây chỉ dùng để quyết định cặp nào chồng lấn. Một ví dụ với bộ `cathedral` được vẽ ở Hình 4.11 (Mục 4.7.5).

2.8.2 Warp ngược và canvas

Warp ngược và nội suy song tuyến. Warp *xuôi* đẩy từng pixel nguồn tới $H\mathbf{x}$, để lại lỗ ở nơi ảnh được phóng và gây va chạm ở nơi ảnh bị thu. Warp *ngược* duyệt từng pixel $\mathbf{p}$ của canvas *đích*, tính xem nó đến từ đâu, $\mathbf{x} = H^{-1}\mathbf{p}$ (một phép nghịch đảo 3×3 và một phép chia cho tọa độ thứ ba cho mỗi pixel), rồi **lấy mẫu ảnh nguồn** tại đó; nhờ vậy mỗi pixel đích có đúng một giá trị. Pixel của canvas có vị trí nguồn nằm ngoài $[0, W-1]\times[0, H-1]$ không thuộc ảnh này (mặt nạ = sai). Vị trí nguồn là số thực nên phải nội suy. Với $x_0 = \lfloor x \rfloor$, $a = x - x_0$ (tương tự y_0, b):

$$f = (1-b)\big[(1-a)f_{00} + a\,f_{10}\big] + b\big[(1-a)f_{01} + a\,f_{11}\big],$$

mỗi trong bốn láng giềng được nhân với diện tích hình chữ nhật con *đối diện*; các trọng số không âm, có tổng bằng 1, và mọi hàm $\alpha + \beta x + \gamma y + \delta xy$ được tái tạo chính xác.

Canvas. Bốn góc của mỗi ảnh được ánh xạ vào hệ tham chiếu; hộp bao của tất cả chúng là canvas. Phép tịnh tiến $T = \begin{bmatrix} 1 & 0 & -x_{\min} \\ 0 & 1 & -y_{\min} \\ 0 & 0 & 1 \end{bmatrix}$ đưa nó về tọa độ không âm, và ánh xạ ảnh → canvas cuối cùng là TH_k. Nếu canvas lớn phi lý (phép chiếu đã "nổ") thì pipeline báo kết quả suy biến thay vì cấp phát bộ nhớ. Phép warp được đối chiếu với OpenCV ở Mục 4.7.6.

2.9 Bù sáng và hòa trộn

Sau khi warp, vùng chồng lấn chứa hai (hoặc nhiều) ước lượng cho mỗi pixel. Chúng khác nhau vì (i) sai số căn chỉnh còn lại (sai số dưới pixel của H, méo ống kính, thị sai) hiện ra thành **bóng ma (ghosting)**, và (ii) khác biệt phơi sáng và cân bằng trắng giữa các lần chụp hiện ra thành **đường nối** nhìn thấy được. Bảng 2.4 liệt kê các chế độ hòa trộn đã hiện thực.

Bảng 2.4: Các chế độ hòa trộn.

Chế độ	Quy tắc	Điểm yếu
overwrite	ảnh sau đè lên ảnh trước	đường nối cứng (mốc so sánh "ngây thơ", như ở Phần 1)
average	trung bình các ảnh phủ pixel	bóng ma; trọng số nhảy bậc ở biên dấu chân ảnh
feather	trọng số = khoảng cách tới biên ảnh, chuẩn hóa	một độ rộng chuyển tiếp cho mọi tần số: hẹp thì lộ đường nối phơi sáng, rộng thì chi tiết bị bóng ma
multi-band	trộn kim tự tháp Laplace (Burt & Adelson, 1983) [8]	cần bộ máy kim tự tháp; thỏa hiệp tốt nhất

Hòa trộn đa băng tần cho K ảnh: nơi hai phần của dự án gặp nhau. Phần 1 trộn *hai* ảnh bằng một mặt nạ (Mục 2.2.2). Ở đây các toán tử của Phần 1 (`build_gaussian_pyramid`, `build_laplacian_pyramid`, `pyr_up` trong `laplacian_blend.py`) được dùng lại cho K ảnh:

1. **Phân vùng.** Mỗi pixel của canvas thuộc về ảnh có khoảng cách tới biên của chính nó lớn nhất (mặt nạ cứng $M_k \in \{0,1\}$, $\sum_k M_k = 1$ trên vùng được phủ).
2. **Kim tự tháp.** Kim tự tháp Gaussian của mỗi mặt nạ, $G_l(M_k)$, và kim tự tháp Laplace của mỗi ảnh, $L_l(I_k) = G_l(I_k) - \mathrm{up}\left(G_{l+1}(I_k)\right)$ (các ảnh băng thông; bậc cuối là phần dư thông thấp).
3. **Trộn từng băng với mặt nạ cùng bậc**, chuẩn hóa bằng tổng trọng số:

$$B_l = \frac{\sum_k G_l(M_k)\, L_l(I_k)}{\sum_k G_l(M_k)}. \tag{2.20}$$

4. **Thu gọn**: $I = B_0 + \mathrm{up}\left(B_1 + \mathrm{up}(B_2 + \cdots)\right)$.

Vì sao hiệu quả. Các bậc Laplace cho khai triển lồng nhau $I = L_0 + \mathrm{up}\left(L_1 + \mathrm{up}(L_2 + \cdots)\right)$, vì vậy trộn các *băng* rồi thu gọn cũng là trộn ảnh. Mặt nạ ở bậc l đã bị làm mờ l lần nên vùng chuyển tiếp của nó rộng cỡ 2^l pixel: chi tiết mịn (bậc 0) chuyển từ ảnh này sang ảnh kia trong vài pixel (không có bóng ma), còn tần số thấp (độ sáng, màu) được trộn trên một vùng rộng, che được đường nối. Nếu dùng một mặt nạ duy nhất cho mọi bậc thì phương pháp quay về feather. Một số chi tiết hiện thực quan trọng: canvas

được đệm tới bội của 2^L để mọi bậc giảm đúng một nửa; các pixel mà một ảnh không phủ được điền bằng màu hợp lệ gần nhất trong một lề (nếu không, nền đen lem vào các tần số thấp đã làm mờ và tạo quầng tối); và phép chia cho $\sum_k G_l(M_k)$ quan trọng ở gần biên canvas, nơi các mặt nạ đã làm mờ không cộng lại thành 1.

Bù sáng (Brown & Lowe, 2007) [18]. Phơi sáng khác nhau được mô hình hóa bằng một hệ số khuếch đại (gain) cho mỗi ảnh, $I_k^{\text{quan sát}} = g_k I_k$. Gọi $\bar{I}_{ij}$ là cường độ trung bình của ảnh i trong vùng chồng với ảnh j và N_{ij} là số pixel của vùng chồng. Các gain cực tiểu hóa

$$e(\mathbf{g}) = \frac{1}{2}\sum_{i,j} N_{ij}\left[\frac{(g_i\bar{I}_{ij} - g_j\bar{I}_{ji})^2}{\sigma_N^2} + \frac{(1-g_i)^2}{\sigma_g^2}\right], \tag{2.21}$$

trong đó số hạng thứ nhất buộc các ảnh chồng lấn khớp nhau và số hạng thứ hai giữ gain gần 1 (thiếu nó thì $\mathbf{g} = \mathbf{0}$ là một cực tiểu). e là bậc hai theo $\mathbf{g}$; đặt $\partial e/\partial g_k = 0$ cho hệ tuyến tính nhỏ

$$\sum_{j\neq k} N_{kj}\left[\left(\frac{2\bar{I}_{kj}^2}{\sigma_N^2} + \frac{1}{\sigma_g^2}\right)g_k - \frac{2\bar{I}_{kj}\bar{I}_{jk}}{\sigma_N^2}\,g_j\right] = \sum_{j\neq k}\frac{N_{kj}}{\sigma_g^2} \qquad (k = 1,\ldots,K),$$

với $\sigma_N = 10/255$ và $\sigma_g = 0.1$ cho ảnh trong $[0,1]$; đó là điều `compute_gains` giải. Kiểm tra: cực tiểu hóa e bằng số (SciPy) cho cùng các gain, sai khác $2.6\cdot10^{-8}$ (bộ `boat`: 0.919; 1.034; 0.995; 1.032). Bù sáng là bước *tùy chọn* (`gain_compensation=True`, cờ `--gain`); các thí nghiệm trên ảnh thật ở Mục 4.8 chạy với giá trị mặc định (tắt) để chỉ so sánh các bộ trích xuất.

2.10 Cách đánh giá chất lượng ghép ảnh toàn cảnh

Hai nhóm thí nghiệm trả lời hai câu hỏi khác nhau.

1. **Ảnh thật** (5 bộ, 2–4 ảnh mỗi bộ, 3 bộ trích xuất, tham số mặc định). Không có đáp án, nên hình học theo từng cặp được đánh giá bằng thống kê của RANSAC, còn căn chỉnh cuối cùng bằng độ khớp của các ảnh đã warp bên trong vùng chồng lấn của chúng.
2. **Nghiên cứu có kiểm soát với đáp án chính xác** (`synthetic.py`, `scripts/run_synthetic.py`). Hai góc nhìn của một bức ảnh được dựng bằng cách xoay một camera ảo; mỗi lần chỉ thay đổi *một* yếu tố khó trong khi các yếu tố khác giữ ở mức dễ. Nhờ biết homography đúng, sai số hình học đo được trực tiếp bằng pixel.

Các độ đo cho ảnh thật.

- **Tỉ lệ nội điểm** = số nội điểm RANSAC / số cặp khớp tạm thời: độ sạch của tập cặp khớp.

- **RMSE** = sai số tái chiếu RMS của các nội điểm (px): mức nhất quán của các điểm đặc trưng với mô hình, tức độ chính xác định vị.
- **NCC thông cao** của vùng chồng lấn: hệ số tương quan chéo chuẩn hóa của độ chói hai ảnh đã warp sau khi lọc thông cao (ảnh trừ ảnh làm mờ Gauss $\sigma = 3$), trên vùng chồng lấn đã co lại 9 px. Bằng 1 khi hai ảnh trùng khít cấu trúc; nhạy với bóng ma và sai lệch căn chỉnh, không nhạy với gain/bias.
- **MAE**: sai khác tuyệt đối trung bình của độ chói (mức xám) trong vùng chồng lấn; nhạy cả với khác biệt phơi sáng và thị sai nên là tính chất của cảnh chứ không chỉ của bộ trích xuất.

Hạn chế của các độ đo: không có đáp án nên RMSE chỉ cho biết các điểm khớp *nhất quán* với mô hình đã khớp, chưa phải sai số tuyệt đối; vì vậy kết luận về độ chính xác tuyệt đối được rút ra từ nghiên cứu có kiểm soát (Mục 4.10).

Tiêu chí thành công trong nghiên cứu có kiểm soát. Một lần thử **thành công** nếu cặp được chấp nhận bởi ba phép kiểm tra (Mục 2.7.6) **và** homography ước lượng cách homography đúng dưới 3 px (RMS trên vùng chồng lấn). *Giới hạn* của một bộ trích xuất theo một yếu tố là mức khó nhất mà nó còn giải được ít nhất 90% số lần thử.

Chương 3 Hiện thực

Các Mục 3.1–3.7 mô tả hiện thực Phần 1; các Mục 3.8–3.12 mô tả hiện thực Phần 2.

3.1 Môi trường và thư viện

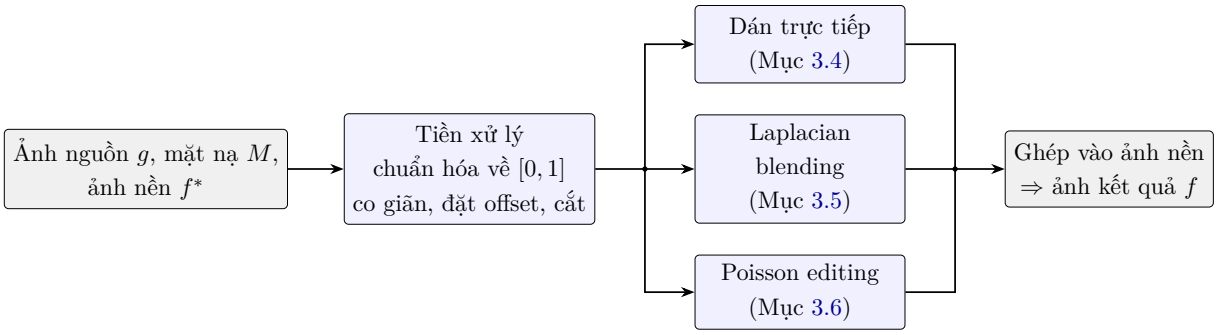
Hiện thực bằng Python 3.12. Bảng 3.1 phân biệt rõ phần *tự cài đặt* (theo yêu cầu "from scratch" của repo) với phần *dùng thư viện*.

Bảng 3.1: Thành phần tự cài đặt và thư viện sử dụng.

Thành phần	Tự cài đặt	Thư viện / phiên bản
Ma trận Laplace thưa, trường dẫn hướng (nhập gradient, gradient trộn)	✓	NumPy [19] 2.4.4; `scipy.sparse.diags` để lưu CSR
Giải hệ tuyến tính thưa		SciPy [7] 1.17.1: `spsolve` (SuperLU)
Tích chập tách được, REDUCE, EXPAND, tháp Gauss và Laplace	✓	NumPy; `cv2.resize` để khớp kích thước
Tái tạo từ tháp Laplace		OpenCV [20] 4.13.0: `cv2.pyrUp`, `cv2.resize`
Đọc ảnh, chuẩn hóa, co giãn, cửa sổ chọn vị trí		OpenCV
Hiển thị, lưu hình		Matplotlib [21]

3.2 Pipeline tổng thể

Ba phương pháp dùng *chung* bước tiền xử lý và cùng đầu vào, chỉ khác bước tính ảnh ghép (Hình 3.1); nhờ vậy sự khác biệt của kết quả hoàn toàn do phương pháp chứ không do cách chuẩn bị dữ liệu.



Hình 3.1: Pipeline của Phần 1: tiền xử lý dùng chung, ba nhánh tính ảnh ghép, rồi ghép kết quả vào ảnh nền.

Vai trò của từng bước

- **Chuẩn hóa về $[0,1]$:** đưa các ảnh về cùng thang số thực để phép trừ gradient và giải hệ không bị tràn số nguyên.
- **Co giãn và đặt offset:** quyết định đối tượng nằm ở đâu và to bao nhiêu; vùng đặt $[y_0, y_0+H_s) \times [x_0, x_0+W_s)$ là miền mà cả ba phương pháp cùng làm việc.
- **Mặt nạ M:** xác định $\Omega = \{M > 0.99\}$ với Poisson (các điểm này là ẩn; điểm còn lại cố định theo ảnh nền) và là hệ số trộn với hai phương pháp kia.
- **Bước tính ảnh ghép:** nơi ba phương pháp khác nhau; xem các mục dưới đây.
- **Ghép lại và cắt giá trị về $[0,1]$:** chép vùng đã tính vào đúng chỗ của ảnh nền; ngoài vùng đặt ảnh nền được giữ nguyên.

3.3 Tiền xử lý và đặt vị trí

- `load_img`: đọc ảnh nguồn, mặt nạ, ảnh nền bằng OpenCV, chuẩn hóa *min–max* về $[0,1]$ (`cv2.normalize`), rồi co giãn nguồn và mặt nạ theo hệ số s bằng nội suy song tuyến tính nếu $s \neq 1$.
- `select_offset`: cửa sổ tương tác để chọn vị trí và tỉ lệ. Di chuột hoặc nhấp để chọn offset, cuộn chuột hoặc phím `+/-` đổi tỉ lệ $\pm 5\%$ mỗi bước, `Enter` để xác nhận. Giá trị đã chọn có thể ghi vào `MANUAL_OFFSETS` trong `config.py` để chạy lại không cần cửa sổ.
- `preprocess_images`: xử lý offset âm (cuộn mảng và gán 0 phần tràn), cắt phần vượt khỏi ảnh nền, và trả về `dims`$= (H_{\min}, H_{\max}, W_{\min}, W_{\max})$ của vùng đặt.

3.4 Dán trực tiếp

Hiện thực đúng (2.9): trên vùng đặt, ảnh kết quả là tổ hợp của nguồn và nền theo hệ số mặt nạ (Mã nguồn 3.1).

```python
# Trích từ src/image_blending/naive_blend.py (ngắt dòng để vừa trang)
def naive_blend(image_data: dict) -> np.ndarray:
    source = image_data['source']
    mask = image_data['mask']
    target = image_data['target'].copy()
    dims = image_data['dims']

    # f = M * g + (1 - M) * f*  trên vùng đặt ảnh nguồn
    target[dims[0]:dims[1], dims[2]:dims[3], :] = \
        target[dims[0]:dims[1], dims[2]:dims[3], :] * (1 - mask) + source * mask
    return target
```

Mã nguồn 3.1: Dán trực tiếp bằng mặt nạ.

3.5 Laplacian pyramid blending

Các bước (Mã nguồn 3.2):

1. *Ghép thô:* nguồn được thay bằng $g \cdot [M > 0.5] + f^* \cdot [M \leq 0.5]$ để phần ngoài mặt nạ của tháp nguồn trùng với nền (tránh kéo nội dung ngoài mặt nạ của ảnh nguồn vào kết quả).
2. *Dựng tháp:* tháp Gauss cho nguồn đã ghép thô, ảnh nền và mặt nạ với $L =$ `NUM_LEVELS` $= 3$ mức mặc định; tháp Laplace cho hai ảnh theo (2.10).
3. *Trộn* từng băng tần theo (2.11) rồi *tái tạo* từ mức thô nhất và cắt về $[0, 1]$.

REDUCE, EXPAND và tích chập tách được (padding phản xạ) được tự cài đặt bằng NumPy; bước tái tạo dùng `cv2.pyrUp`, cùng nhân Gauss 5 điểm với hệ số khuếch đại 4.

```python
# Trích từ src/image_blending/laplacian_blend.py (rút gọn)
_GAUSS5 = np.array([1, 4, 6, 4, 1], dtype=np.float64) / 16.0   # nhân Gauss 5 điểm

def pyr_down(img):                    # REDUCE: làm mờ rồi lấy mẫu một nửa
    blurred = _conv(img, _GAUSS5)   # _conv: tích chập tách được, padding phản xạ
    return blurred[::2, ::2].astype(img.dtype)

def pyr_up(img):                      # EXPAND: chèn 0, làm mờ, nhân 4
    H, W, C = img.shape
    up = np.zeros((H * 2, W * 2, C), dtype=np.float64)
    up[::2, ::2] = img.astype(np.float64)
    return (_conv(up, _GAUSS5) * 4.0).astype(img.dtype)

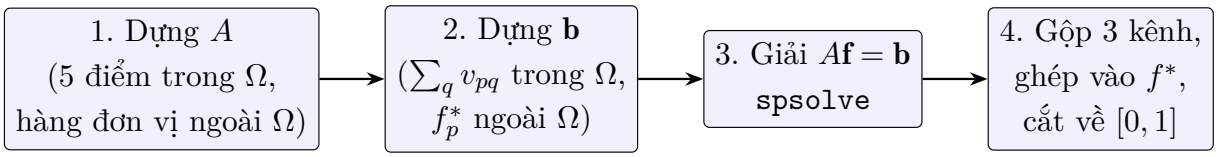
def build_laplacian_pyramid(gp, levels):   # L_k = G_k - EXPAND(G_{k+1}); L_N = G_N
    lp = [gp[levels].astype(np.float32)]   # chỉ số 0 = mức thô nhất
    for i in range(levels, 0, -1):
        up = cv2.resize(pyr_up(gp[i]), (gp[i-1].shape[1], gp[i-1].shape[0]))
        lp.append(gp[i-1].astype(np.float32) - up.astype(np.float32))
    return lp

# ---- bên trong laplacian_blend(): trộn từng băng tần rồi tái tạo ----
for i, (lap_s, lap_t) in enumerate(zip(lp_src, lp_tgt)):
    m = gp_mask[num_levels - i].astype(np.float32) / 255.0   # mặt nạ cùng mức
    blended_pyramid.append(lap_s * m + lap_t * (1.0 - m))
result = blended_pyramid[0]
for i in range(1, num_levels + 1):
    result = cv2.pyrUp(result)
    result = cv2.resize(result, (blended_pyramid[i].shape[1],
                                 blended_pyramid[i].shape[0]))
    result += blended_pyramid[i]
```

Mã nguồn 3.2: Laplacian pyramid blending (rút gọn): REDUCE, EXPAND, tháp Laplace, trộn và tái tạo.

3.6 Poisson editing (Gradient Domain Editing)

Với mỗi kênh màu độc lập, hàm `poisson_blend` thực hiện bốn bước (Hình 3.2):



Hình 3.2: Các bước bên trong `poisson_blend` cho một kênh màu (bước 1–3) và bước gộp kênh (bước 4).

Bước 1 – dựng ma trận A. Theo (2.8), đường chéo chính bằng 4 tại các điểm của Ω và bằng 1 ở ngoài; bốn đường chéo phụ (độ dời ± 1 và $\pm W_s$) bằng -1 tại các hàng thuộc Ω. Phép gán `valid_left[Ws-1::Ws]=False` (và tương tự cho `valid_right`) ngắt các liên kết "vòng" từ cuối hàng này sang đầu hàng kia. Ma trận được lưu ở dạng CSR; Hình 3.3 vẽ cấu trúc thực của A do chính hàm này sinh ra cho một lưới nhỏ.

```
# Trích từ src/image_blending/poisson_blend.py :: poisson_solver (rút gọn)
# Ma trận Laplace thưa A cho một kênh; Hs, Ws: kích thước vùng đặt
mask_bool = mask.flatten(order='C') > 0.99       # Omega
num_pixels = Hs * Ws

# Đường chéo: 4 trong Omega; 1 ngoài Omega (hàng đơn vị: f_p = f*_p)
diag_main = np.where(mask_bool, 4.0, 1.0)

# Bốn đường chéo phụ: -1 cho láng giềng trên/dưới/trái/phải của p thuộc Omega
diag_up   = np.where(mask_bool[Ws:], -1.0, 0.0)
diag_down = np.where(mask_bool[:num_pixels - Ws], -1.0, 0.0)

valid_left = mask_bool[1:].copy()
valid_left[Ws-1::Ws] = False                     # không nối qua mép hàng
valid_right = mask_bool[:num_pixels - 1].copy()
valid_right[Ws-1::Ws] = False
diag_left  = np.where(valid_left, -1.0, 0.0)
diag_right = np.where(valid_right, -1.0, 0.0)

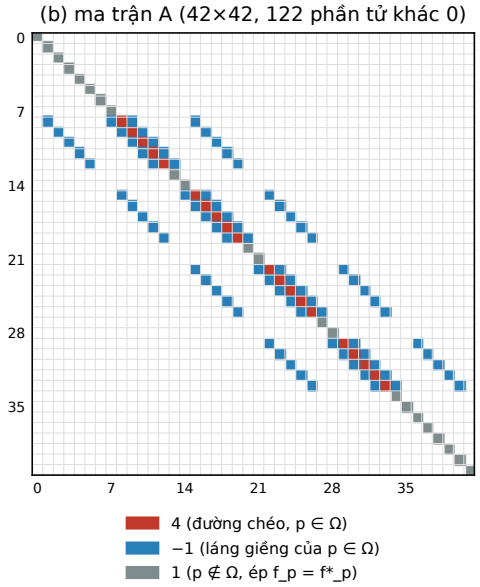
laplacian = diags([diag_main, diag_up, diag_down, diag_left, diag_right],
                  [0, -Ws, Ws, -1, 1],
                  shape=(num_pixels, num_pixels), format='csr')
```

Mã nguồn 3.3: Dựng ma trận Laplace thưa A (rút gọn).

Bước 2 – dựng vế phải b. Gradient của nguồn theo bốn hướng được tính bằng hiệu của mảng với bản `np.roll` của nó (đúng $g_p - g_q$ của (2.6)); ở mép lưới, giá trị láng giềng được xem là 0. Với gradient trộn, hàm `_compare` chọn, trên từng hướng, gradient có độ lớn lớn hơn giữa nguồn và nền. Tổng bốn hướng cho b_p trong Ω; ngoài Ω, $b_p = f^*_p$.

(a) lưới 6×7: ô xanh = Ω, ô xám = ngoài Ω

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41



Hình 3.3: Cấu trúc ma trận A do `poisson_solver` sinh ra cho lưới 6×7 với Ω là khối 4×5 ở giữa: hàng thuộc Ω có đường chéo 4 và bốn phần tử −1; hàng ngoài Ω chỉ có phần tử 1 trên đường chéo.

```
# Trích từ src/image_blending/poisson_blend.py :: poisson_solver (rút gọn)
# Vế phải b cho một kênh
src_2d = source.reshape(Hs, Ws)
tgt_2d = target_subimg.reshape(Hs, Ws)

# Gradient của nguồn theo từng hướng: g_p - g_q (np.roll lấy láng giềng)
grad_up_s = src_2d - np.roll(src_2d, shift=1, axis=0)
grad_up_s[0, :] = src_2d[0, :]                    # mép lưới
grad_down_s = src_2d - np.roll(src_2d, shift=-1, axis=0)
grad_down_s[-1, :] = src_2d[-1, :]
# ... grad_left_s, grad_right_s tương tự theo axis=1
# ... nếu grad_mix=True: tính thêm grad_*_t từ tgt_2d theo cùng cách

if grad_mix:                      # gradient trộn: giữ gradient có độ lớn lớn hơn
    _compare = lambda x, y: np.where(np.abs(x) > np.abs(y), x, y)
    g_up = _compare(grad_up_s, grad_up_t)         # ... tương tự cho 3 hướng còn lại
else:                             # nhập gradient
    g_up = grad_up_s                              # ... tương tự cho 3 hướng còn lại

# b_p = tổng gradient 4 hướng trong Omega; b_p = f*_p ngoài Omega
div = (g_up + g_down + g_left + g_right).flatten(order='C')
guidance_field = np.where(mask_bool, div, target_subimg)
```

Mã nguồn 3.4: Dựng vế phải **b** (rút gọn): gradient của nguồn, gradient trộn và điều kiện biên.

Bước 3 và 4 – giải hệ, gộp kênh. Mỗi kênh giải bằng `spsolve` trên ma trận dạng CSC, kết quả được định hình lại thành ảnh, ba kênh được ghép lại, chép vào đúng vùng

của ảnh nền và cắt về $[0, 1]$.

```
# Trích từ src/image_blending/poisson_blend.py (rút gọn)
def linear_solver(A, b, dims):
    x = linalg.spsolve(A.tocsc(), b)             # LU thưa (SuperLU)
    return np.reshape(x, (dims[0], dims[1]))

def poisson_blend(image_data, grad_mix=False):
    equation_param = []
    for ch in range(3):                          # mỗi kênh màu là một hệ riêng
        ch_data = {'source': image_data['source'][:, :, ch],
                   'mask': image_data['mask'][:, :, ch],
                   'target': image_data['target'][:, :, ch],
                   'dims': image_data['dims']}
        equation_param.append(poisson_solver(ch_data, grad_mix))   # [A, b]

    Hs, Ws = image_data['source'].shape[:2]
    image_result = np.empty_like(image_data['source'])
    for i in range(3):
        A, b = equation_param[i]
        image_result[:, :, i] = linear_solver(A, b, (Hs, Ws))
    image_result = stitch_img(image_result, image_data['target'].copy(),
                              image_data['dims'])
    return np.clip(image_result, 0, 1)           # đưa về [0, 1]
```

Mã nguồn 3.5: Giải hệ cho từng kênh và ghép kết quả vào ảnh nền (rút gọn).

Vì sao lấy cả vùng đặt làm ẩn thay vì chỉ các điểm trong Ω?

Cách đặt hàng đơn vị cho điểm ngoài Ω (công thức (2.8)) cho phép dựng A bằng vài phép `numpy.where` trên cả lưới và `scipy.sparse.diags`, không cần đánh chỉ số lại các điểm của Ω hay vòng lặp từng điểm ảnh. Cái giá là hệ lớn hơn cần thiết: với cặp ảnh `02`, $n = 153\,600$ trong khi chỉ có $|\Omega| = 8\,971$ ẩn thực sự, nhưng các hàng đơn vị được bộ giải xử lý rất rẻ. Với dữ liệu cỡ này sự đơn giản đáng giá hơn tối ưu bộ nhớ.

Chi phí. Mỗi ảnh phải dựng và giải *ba* hệ riêng cho ba kênh (Bảng 4.5). Vì mặt nạ giống nhau ở cả ba kênh nên ma trận A giống hệt nhau; chỉ vế phải $\mathbf{b}$ khác. Hiện thực hiện tại vẫn dựng lại A và phân rã LU ba lần; phân rã một lần rồi dùng lại cho ba vế phải là một cải tiến đơn giản (Mục 6.2).

3.7 Mã nguồn và hướng dẫn chạy

Mã nguồn đầy đủ: https://github.com/laveanfield/Computer-Vision-2. Cấu trúc phần mã của Phần 1 trong repo:

```
Computer-Vision-2/
  src/image_blending/
    config.py            # đường dẫn và tham số chạy (NUM_LEVELS, GRAD_MIX, ...)
    utils.py             # load_img, preprocess_images, select_offset, ...
    naive_blend.py       # dán trực tiếp
    laplacian_blend.py   # Laplacian pyramid blending
    poisson_blend.py     # Gradient Domain Editing (Poisson)
  data/image_blending/{source,mask,target,result}/
  notebooks/demo_image_blending.ipynb   # notebook minh họa
  pyproject.toml
```

Cài gói (kèm thư viện phụ thuộc) và chạy notebook minh họa, hoặc gọi trực tiếp các hàm trong `src/image_blending/`:

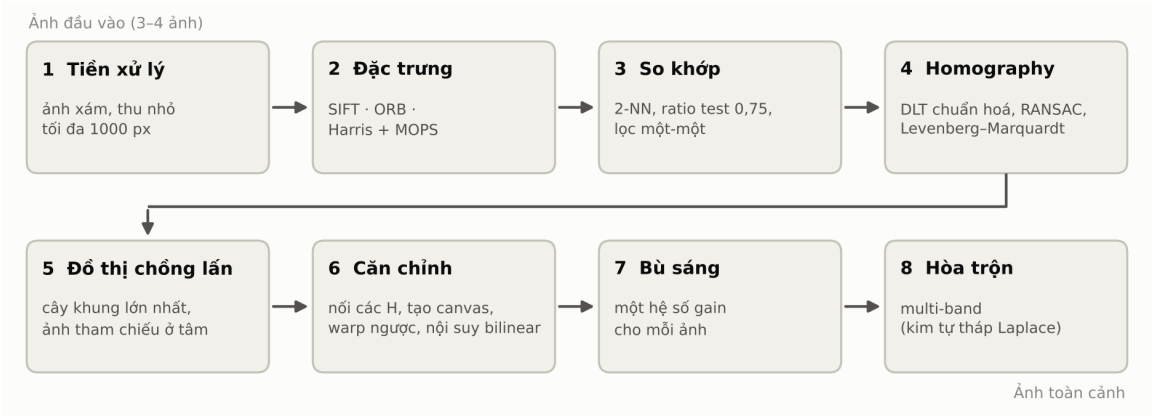
```
pip install -e ".[dev]"
jupyter notebook notebooks/demo_image_blending.ipynb
```

Để tái tạo toàn bộ hình và số liệu Phần 1 của Chương 4 (không cần cửa sổ chọn vị trí vì offset và tỉ lệ đã được cố định trong script `make_figures_part1.py` đi kèm báo cáo):

```
pip install scikit-image
python scripts/make_figures_part1.py --repo <đường dẫn repo> --out figures/part1
↪  --tables tables
```

3.8 Phần 2: Pipeline ghép ảnh và vai trò của từng bước

Hình 3.4 tóm tắt pipeline; Bảng 3.2 nêu vai trò của từng bước và hậu quả khi nó thiếu hoặc hỏng. **Chỉ bước 2 thay đổi giữa ba bộ trích xuất đặc trưng**; mọi bước còn lại dùng cùng một đoạn mã và cùng tham số (Phụ lục D), nên phép so sánh là công bằng. Ảnh được đưa vào theo thứ tự bất kỳ: các cặp ảnh chồng lấn được tìm ra từ chính các cặp khớp.



Hình 3.4: Sơ đồ pipeline ghép ảnh toàn cảnh. Bước 7 (bù sáng) là tùy chọn.

Bảng 3.2: Vai trò của từng bước và hậu quả khi bước đó thiếu hoặc hỏng.

Bước	Vai trò	Nếu thiếu hoặc hỏng	Tham số chính
1. Tiền xử lý	chuyển xám (bộ mô tả chỉ cần độ sáng); thu nhỏ ảnh tới tối đa 1000 px để giảm chi phí mọi bước sau (ảnh 10 Mpx của bộ `boat` giảm khoảng 15 lần số điểm ảnh)	ảnh lớn làm mọi bước chậm; ngưỡng RANSAC 3 px phải đổi theo kích thước ảnh	`max_side`
2. Đặc trưng	phát hiện các điểm có thể tìm lại ở góc nhìn khác (góc, blob) và mô tả lân cận bằng vector *bất biến* với phép xoay, tỉ lệ, đổi sáng	**mọi bước sau chỉ dùng được những gì đã được phát hiện**: vùng không có điểm đặc trưng thì không có ràng buộc (thí nghiệm ảnh tối); bộ mô tả không bất biến ⇒ cặp đúng trông khác nhau (răng cưa theo zoom); không đủ phân biệt ⇒ cặp sai trông giống nhau (chữ lặp)	số điểm, ngưỡng của detector
3. So khớp	với mỗi điểm tìm bộ mô tả gần nhất ở ảnh kia; ratio test loại cặp mơ hồ; lọc một–một loại cặp tranh chấp	không có ratio test thì khoảng một nửa số cặp khớp là sai và RANSAC cần nhiều lần lặp hơn nhiều (Bảng 4.11)	`ratio` = 0.75
4. Homography	DLT chuẩn hóa trong RANSAC, tinh chỉnh LM trên nội điểm; ba phép kiểm tra chấp nhận cặp ảnh	ngoại lai làm hỏng bình phương tối thiểu; mẫu suy biến cho cả một họ nghiệm; cặp sai không được vào panorama	ngưỡng 3 px, `min_inliers` = 20
5. Đồ thị chồng lấn	cạnh = số nội điểm; cây khung lớn nhất nối mọi ảnh bằng các cặp tin cậy nhất; ảnh tham chiếu ở tâm cây	sai số tích lũy dọc chuỗi homography (tâm cây làm chuỗi dài nhất ngắn nhất); ảnh không chồng lấn bị loại thay vì làm hỏng kết quả	—
6. Căn chỉnh	warp ngược + nội suy song tuyến lên canvas chứa mọi dấu chân	warp xuôi để lại lỗ; lấy mẫu láng giềng gần nhất cho cạnh răng cưa	`max_canvas_side`

Bảng 3.2 (tiếp theo)

Bước	Vai trò	Nếu thiếu hoặc hỏng	Tham số chính
7. Bù sáng	một gain cho mỗi ảnh để các vùng chồng lấn cùng độ sáng	khác phơi sáng ⇒ đường nối nhìn thấy được	`gain_compensation`
8. Hòa trộn	multi-band: trộn tần số thấp trên vùng rộng, chi tiết trên vùng hẹp	đường nối (phơi sáng) và bóng ma (sai số căn chỉnh, thị sai, vật chuyển động)	`num_levels`

3.9 Phần nào tự cài đặt, phần nào dùng thư viện

Bảng 3.3: Phân chia giữa mã tự viết và thư viện trong Phần 2.

Khâu	Tự cài đặt (NumPy / SciPy)	Dùng thư viện
Tiền xử lý	—	OpenCV: đọc ảnh, đổi cỡ, chuyển ảnh xám
Đặc trưng	**Harris + MOPS** hoàn chỉnh (`harris_mops.py`)	SIFT và ORB của OpenCV
So khớp	2-NN brute-force bằng một phép nhân ma trận, ratio test, lọc một–một	—
Homography	DLT chuẩn hóa, RANSAC thích nghi, Levenberg–Marquardt, kiểm tra suy biến	—
Căn chỉnh	nội suy song tuyến, warp ngược, tính canvas, đồ thị chồng lấn, cây khung	—
Hòa trộn	overwrite, average, feather, **multi-band cho K ảnh**, bù sáng	lọc Gaussian, biến đổi khoảng cách (OpenCV, SciPy)
Đánh giá	ảnh tổng hợp có đáp án (xoay camera ảo), các độ đo	—

Mỗi thành phần tự viết được đối chiếu với một tham chiếu độc lập (`cv2.BFMatcher`, `cv2.remap`, `cv2.warpPerspective`, `cv2.findHomography`, homography và góc xoay đã biết trước) trong 41 bài kiểm thử tự động (`tests/test_panorama.py`).

3.10 Tiền xử lý

- **Thu nhỏ** để cạnh dài nhất không quá 1000 px, bằng `cv2.INTER_AREA` (trung bình theo diện tích phủ bởi mỗi pixel đầu ra; lấy mẫu thưa đơn thuần sẽ gây răng cưa). Chi phí của mọi bước sau (đặc trưng, so khớp, warp, hòa trộn) tỉ lệ với số pixel, và ngưỡng RANSAC 3 px được tính theo pixel của ảnh đã thu nhỏ. Ví dụ: ảnh `boat` 3888×2592 trở thành 1000×667.
- **Ảnh xám**: các bộ phát hiện làm việc trên một kênh, $Y = 0.299R + 0.587G + 0.114B$. Màu được giữ lại cho bước warp và hòa trộn. Ảnh đầu vào đã là ảnh xám được

nhân thành ba kênh bằng nhau.

3.11 Quy ước tọa độ: tâm của pixel nằm ở đâu?

Homography ánh xạ *tọa độ*, nên mọi khâu phải dùng chung một quy ước. Ở đây (như `cv2.warpPerspective`) tâm của pixel góc trên trái là $(0, 0)$. SIFT và ORB của OpenCV không theo đúng quy ước đó. Bậc đầu của SIFT là ảnh phóng 2 lần: chỉ số u của ảnh ấy nằm ở $u/2 - 1/4$ trong ảnh gốc nhưng OpenCV báo $u/2$, nên **mọi** điểm SIFT bị lệch $+\frac{1}{4}$ px. ORB làm việc trên các bậc có kích thước được làm tròn và báo $x_l s_l$ thay vì $(x_l + \frac{1}{2})W/W_l - \frac{1}{2}$, nên độ lệch phụ thuộc vào bậc. Độ lệch vô hình khi hai ảnh được xử lý giống hệt nhau (nó triệt tiêu), nhưng **không triệt tiêu** khi một ảnh bị xoay hoặc co so với ảnh kia. Phép xoay 180° chính xác làm nó lộ ra: điểm (x, y) phải xuất hiện lại ở $(W - 1 - x,\ H - 1 - y)$, và nếu mỗi điểm lệch **b** thì phần dư là 2**b**. Bảng 3.4 cho kết quả: độ lệch khoảng 0.25 px (SIFT) và cỡ 0.4 px (ORB) bị loại bởi `fix_coords=True`; phần dư 0.25 px của SIFT là nhiễu định vị.

Bảng 3.4: Phép thử xoay 180° trên ảnh `boat1`: sai lệch giữa vị trí mong đợi và vị trí tìm được của các điểm khớp (phần dư = 2× độ lệch).

Bộ trích xuất	Tọa độ	Số cặp khớp	dx trung bình (px)	dy trung bình (px)	RMS (px)
SIFT	thô (OpenCV)	1631	+0.505	+0.496	0.751
SIFT	đã hiệu chỉnh	1631	+0.005	−0.004	0.251
ORB	thô (OpenCV)	3954	−0.842	−0.767	1.687
ORB	đã hiệu chỉnh	3954	0.000	0.000	0.000
Harris + MOPS	mã tự viết	2260	+0.000	+0.005	0.313

3.12 Mã nguồn chính của Phần 2

Mã nguồn Phần 2 nằm cùng repo với Phần 1 (https://github.com/laveanfield/Computer-Vision-2), trong package `src/panorama/`:

```
src/panorama/
  config.py        # StitchConfig: mọi tham số mặc định (Phụ lục D)
  features.py      # tiền xử lý, SIFT/ORB (OpenCV), sửa lệch nửa pixel
  harris_mops.py   # Harris (trung bình điều hòa), ANMS, bộ mô tả MOPS
  matching.py      # 2-NN bằng phép nhân ma trận, ratio test, lọc một-một
  homography.py    # DLT chuẩn hóa, RANSAC thích nghi, Levenberg-Marquardt
  warp.py          # warp ngược, nội suy song tuyến, canvas
  blend.py         # overwrite, average, feather, multi-band, bù sáng
  stitcher.py      # PanoramaStitcher: đồ thị chồng lấn, cây khung, ghép
  metrics.py       # RMSE, NCC thông cao, MAE của vùng chồng lấn
  synthetic.py     # góc nhìn tổng hợp có homography đúng (nghiên cứu có kiểm soát)
  viz.py           # vẽ hình
data/panorama/{input,result}/
```

Bốn đoạn trích dưới đây là các khâu cốt lõi được tự cài đặt (đã rút gọn phần kiểm tra lỗi và chú thích).

```python
# Trích từ src/panorama/homography.py (rút gọn)
def normalize_points(pts):
    # Chuẩn hóa Hartley: trọng tâm -> gốc, khoảng cách trung bình -> sqrt(2)
    c = pts.mean(axis=0)
    d = np.linalg.norm(pts - c, axis=1).mean()
    s = np.sqrt(2.0) / d if d > 1e-12 else 1.0
    T = np.array([[s, 0.0, -s * c[0]],
                  [0.0, s, -s * c[1]],
                  [0.0, 0.0, 1.0]])
    return (pts - c) * s, T

def dlt_homography(src, dst, normalize=True):
    n = len(src)
    if n < 4:
        return None
    if normalize:
        src_n, T1 = normalize_points(src)
        dst_n, T2 = normalize_points(dst)
    else:
        src_n, dst_n, T1, T2 = src, dst, np.eye(3), np.eye(3)

    x, y = src_n[:, 0], src_n[:, 1]
    u, v = dst_n[:, 0], dst_n[:, 1]
    zero, one = np.zeros(n), np.ones(n)
    A = np.zeros((2 * n, 9))                    # hai phương trình cho mỗi cặp
    A[0::2] = np.stack([zero, zero, zero, -x, -y, -one, v*x, v*y, v], axis=1)
    A[1::2] = np.stack([x, y, one, zero, zero, zero, -u*x, -u*y, -u], axis=1)
    if A.shape[0] < 9:                          # đúng 4 điểm: đệm cho đủ 9 hàng
        A = np.vstack([A, np.zeros((9 - A.shape[0], 9))])

    _, _, Vt = np.linalg.svd(A, full_matrices=False)
    # h = vector kỳ dị phải ứng với giá trị kỳ dị nhỏ nhất, rồi hoàn nguyên
    H = np.linalg.inv(T2) @ Vt[-1].reshape(3, 3) @ T1
    return normalize_homography(H)              # chia để h33 = 1
```

Mã nguồn 3.6: DLT chuẩn hóa (`homography.py`): dựng ma trận A từ hai phương trình mỗi cặp điểm, lấy vector kỳ dị phải nhỏ nhất, hoàn nguyên phép chuẩn hóa.

```python
# Trích từ src/panorama/matching.py (rút gọn)
def knn2(desc1, desc2, binary, chunk=1024):
    # ORB: bung 32 byte thành 256 bit -> Hamming = bình phương khoảng cách L2
    a, b = _prepare(desc1, binary), _prepare(desc2, binary)
    b2 = (b * b).sum(axis=1)
    idx = np.empty((len(a), 2), dtype=np.int64)
    dist = np.empty((len(a), 2), dtype=np.float32)
    for s in range(0, len(a), chunk):           # xử lý theo khối để giới hạn RAM
        blk = a[s:s + chunk]
        # ||a - b||^2 = ||a||^2 + ||b||^2 - 2 a.b  : một phép nhân ma trận
        d2 = (blk * blk).sum(axis=1)[:, None] + b2[None, :] - 2.0 * (blk @ b.T)
        np.maximum(d2, 0.0, out=d2)             # chặn giá trị âm do làm tròn
        part = np.argpartition(d2, 1, axis=1)[:, :2]   # hai láng giềng gần nhất
        vals = np.take_along_axis(d2, part, axis=1)
        order = np.argsort(vals, axis=1)
        idx[s:s + chunk] = np.take_along_axis(part, order, axis=1)
        vals = np.take_along_axis(vals, order, axis=1)
        dist[s:s + chunk] = np.rint(vals) if binary else np.sqrt(vals)
    return idx, dist

def match_features(f1, f2, ratio=0.75, mutual=False, unique=True):
    idx, dist = knn2(f1.desc, f2.desc, f1.binary)
    r = dist[:, 0] / dist[:, 1]
    keep = np.isfinite(r) & (r < ratio)         # ratio test của Lowe
    if mutual:                                  # kiểm tra chéo (mặc định tắt)
        back, _ = knn2(f2.desc, f1.desc, f1.binary)
        keep &= back[idx[:, 0], 0] == np.arange(len(f1))
    q = np.flatnonzero(keep)
    if unique and len(q) > 1:                   # lọc một-một
        order = np.argsort(r[q], kind="stable")          # rõ ràng nhất trước
        _, first = np.unique(idx[q[order], 0], return_index=True)
        q = np.sort(q[order[first]])            # mỗi đối tác giữ cặp tỉ số nhỏ nhất
    return Matches(q, idx[q, 0], dist[q, 0], r[q], len(f1))
```

Mã nguồn 3.7: So khớp (`matching.py`): khoảng cách bằng một phép nhân ma trận, ratio test và lọc một–một.

```python
# Trích từ src/panorama/homography.py (rút gọn)
def required_iterations(inlier_ratio, confidence, sample_size=4):
    # N >= log(1 - p) / log(1 - w^4)
    ws = inlier_ratio ** sample_size
    if ws <= 1e-14:
        return np.inf
    if ws >= 1.0 - 1e-14:
        return 0.0
    return float(np.ceil(np.log(1.0 - confidence) / np.log(1.0 - ws)))

def ransac_homography(src, dst, thresh=3.0, confidence=0.995,
                      max_iters=3000, seed=0, refine=True):
    n = len(src)
    rng = np.random.default_rng(seed)
    best_inl, best_cnt, best_err_sum = np.zeros(n, dtype=bool), 0, np.inf
    needed, it = float(max_iters), 0
    while it < min(needed, max_iters):
        it += 1
        idx = rng.choice(n, size=4, replace=False)       # mẫu tối thiểu
        if not _is_good_sample(src[idx], dst[idx]):      # bỏ mẫu suy biến
            continue
        H = dlt_homography(src[idx], dst[idx])
        if H is None:
            continue
        err = reprojection_error(H, src, dst)
        inl = err < thresh                               # tập đồng thuận
        cnt, err_sum = int(inl.sum()), float(err[inl].sum())
        if cnt > best_cnt or (cnt == best_cnt and cnt > 0
                              and err_sum < best_err_sum):
            best_inl, best_cnt, best_err_sum = inl, cnt, err_sum
            # thích nghi: cập nhật số vòng cần thiết theo tỉ lệ nội điểm tốt nhất
            needed = min(needed, required_iterations(cnt / n, confidence))
    if best_cnt < 4:
        return None, np.zeros(n, dtype=bool), {}

    # khớp lại bằng DLT trên mọi nội điểm, lặp đến khi tập nội điểm không đổi
    inl, H = best_inl, dlt_homography(src[best_inl], dst[best_inl])
    for _ in range(5):
        new_inl = reprojection_error(H, src, dst) < thresh
        if new_inl.sum() < 4 or np.array_equal(new_inl, inl):
            break
        inl, H = new_inl, dlt_homography(src[new_inl], dst[new_inl])
    if refine and inl.sum() >= 5:                        # Levenberg-Marquardt
        H = refine_homography(H, src[inl], dst[inl])
    inl = reprojection_error(H, src, dst) < thresh       # đếm nội điểm lần cuối
    return H, inl, {"iterations": it, "n_inliers": int(inl.sum())}
```

Mã nguồn 3.8: Vòng lặp RANSAC thích nghi (`homography.py`).

```python
# Trích từ src/panorama/blend.py (rút gọn)
from image_blending.laplacian_blend import (     # dùng lại toán tử của Phần 1
    build_gaussian_pyramid, build_laplacian_pyramid, pyr_up)

def blend_multiband(images, masks, num_levels=5):
    K, (h, w) = len(images), masks[0].shape
    # 1. Phân vùng: mỗi pixel thuộc ảnh có khoảng cách tới biên lớn nhất
    dist = np.stack(distance_weights(masks))
    label, covered = dist.argmax(axis=0), dist.max(axis=0) > 0
    hard = [((label == k) & covered).astype(np.float32) for k in range(K)]

    mult = 2 ** num_levels                       # đệm để mỗi bậc giảm đúng 1/2
    pad = ((0, (-h) % mult), (0, (-w) % mult))
    margin = 2 ** (num_levels + 2)
    blended, weight_sum = None, None
    for k in range(K):
        # điền màu hợp lệ gần nhất để nền đen không lem vào tần số thấp
        img = _fill_invalid(images[k], masks[k], margin)
        img = np.pad(img, pad + ((0, 0),), mode="edge").astype(np.float32)
        lp = build_laplacian_pyramid(build_gaussian_pyramid(img, num_levels),
                                     num_levels)  # 2. tháp Laplace của ảnh
        # tháp Gauss của mặt nạ
        gm = build_gaussian_pyramid(np.pad(hard[k], pad), num_levels)
        if blended is None:
            blended = [np.zeros_like(l) for l in lp]
            weight_sum = [np.zeros(l.shape[:2], np.float32) for l in lp]
        for lev in range(num_levels + 1):        # 3. trộn từng băng với mặt nạ cùng
        ↪ bậc
            wk = gm[num_levels - lev].astype(np.float32)
            blended[lev] += lp[lev] * wk[..., None]
            weight_sum[lev] += wk
    for lev in range(num_levels + 1):            # chuẩn hóa bằng tổng trọng số
        blended[lev] /= np.maximum(weight_sum[lev], 1e-6)[..., None]

    result = blended[0]                          # 4. thu gọn từ bậc thô nhất
    for lev in range(1, num_levels + 1):
        H_, W_ = blended[lev].shape[:2]
        result = pyr_up(result)[:H_, :W_] + blended[lev]
    return np.clip(result[:h, :w], 0.0, 1.0).astype(np.float32)
```

Mã nguồn 3.9: Hòa trộn đa băng tần cho K ảnh (`blend.py`, đã bỏ phần đệm canvas): dùng lại các toán tử kim tự tháp của Phần 1.

Chương 4 Kết quả thực nghiệm

Các Mục 4.1–4.5 trình bày kết quả của Phần 1; các Mục 4.6–4.13 trình bày kết quả của Phần 2.

4.1 Dữ liệu và thiết lập thí nghiệm

Ba cặp ảnh được chọn để phủ các tình huống khác nhau về mức chênh lệch màu và kết cấu giữa nguồn và nền (Bảng 4.1; ảnh đầu vào ở Hình 4.1).

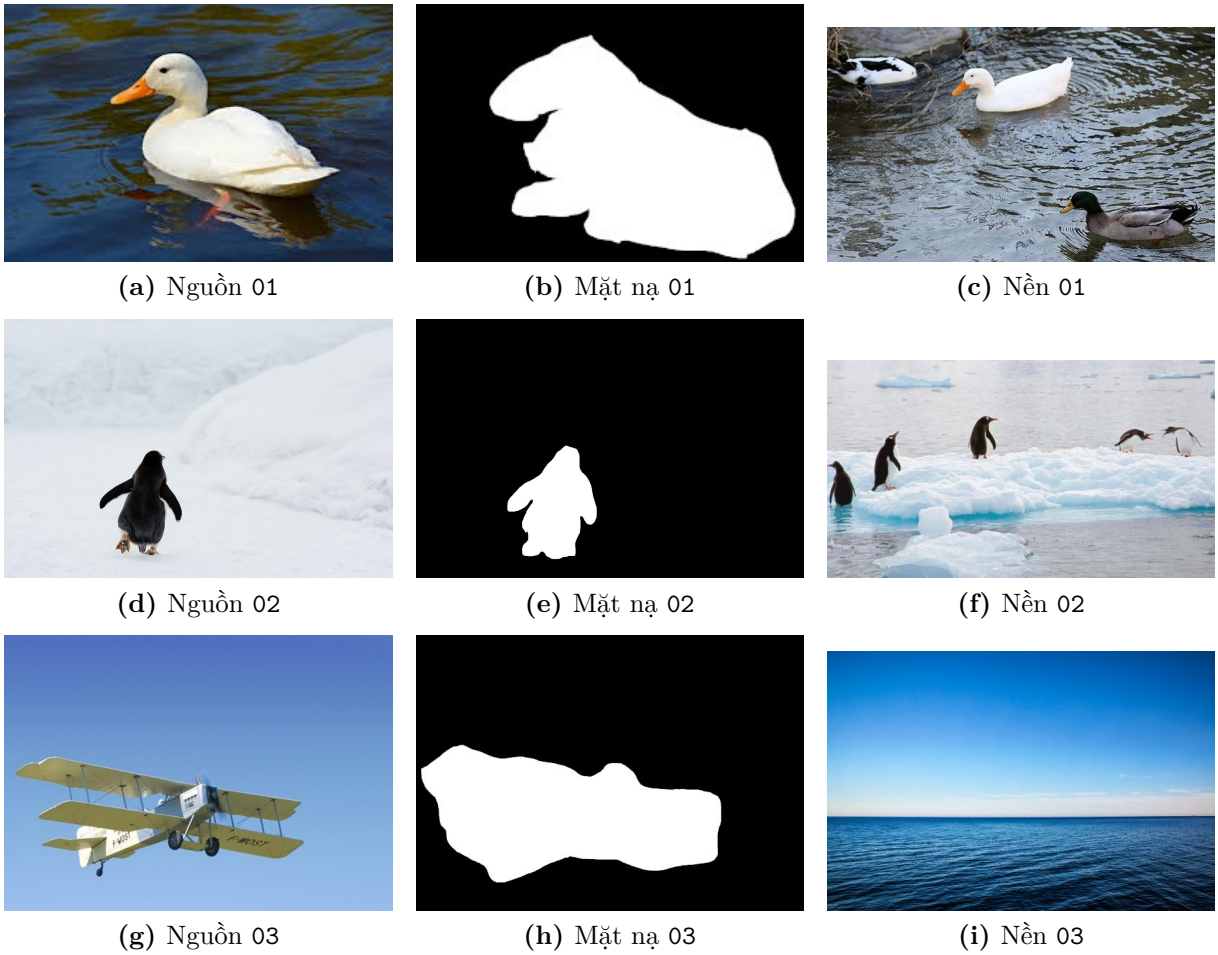
Bảng 4.1: Mô tả ba cặp ảnh thử nghiệm.

Cặp	Nội dung	Đặc điểm	Nguồn ảnh
01	Vịt trắng trên mặt nước xanh sẫm → nền nước xám bạc gợn sóng	Chênh lệch màu nước lớn; nền nhiều kết cấu; mặt nạ chứa cả nước và bóng phản chiếu	Internet
02	Chim cánh cụt trên tuyết trắng → nền băng và nước xanh ngọc	Vật thể tối trên nền sáng; mặt nạ ôm sát vật thể; nền có ranh giới băng/nước ngay cạnh vùng ghép	Internet
03	Máy bay hai tầng cánh trên trời xanh tím → nền trời xanh lam và mặt biển	Nền mượt, ít kết cấu; chênh lệch màu trời vừa phải; mặt nạ chứa nhiều bầu trời	Internet

Thiết lập chung: Laplacian blending dùng $L = 3$ mức; Poisson dùng nhập gradient (gradient trộn chỉ dùng ở Mục 4.4.2). Vị trí đặt (offset, tỉ lệ) được chọn bằng cửa sổ tương tác rồi cố định trong script sinh hình; Bảng 4.2 liệt kê chúng cùng kích thước vùng ghép $\Omega = \{M > 0.99\}$. Các ảnh kết quả trong chương này được tạo bằng chính code của repo. Với cặp 01 và 02, chúng trùng khớp với ảnh đã lưu trong `data/image_blending/result` (sai khác bằng 0 sau cùng một lần nén JPEG chất lượng 95); ảnh đã lưu của cặp 03 được tạo với một vị trí đặt khác nên không so trực tiếp được. Thời gian chạy được đo trên một máy Linux x86-64 với Python 3.12.3, NumPy 2.4.4, SciPy 1.17.1 và OpenCV 4.13.0; các con số chỉ có ý nghĩa về bậc độ lớn và sẽ khác trên máy khác, còn các số liệu còn lại (chỉ số S, PSNR, SSIM…) không phụ thuộc máy.

Bảng 4.2: Kích thước ảnh, vị trí đặt và kích thước vùng ghép của từng cặp (s: hệ số tỉ lệ; $H_s \times W_s$: vùng ghép sau khi co giãn).

Cặp ảnh	Nguồn gốc	s	Vùng ghép	Nền	Offset (y_0, x_0)	$\lvert\Omega\rvert$ (%)
01	183×275	1.0	183×275	369×609	(130, 29)	19 036 (37.8)
02	800×1200	0.4	320×480	326×580	(0, 0)	8 971 (5.8)
03	425×600	0.5	212×300	408×612	(43, 54)	15 603 (24.5)



(a) Nguồn 01 **(b)** Mặt nạ 01 **(c)** Nền 01

(d) Nguồn 02 **(e)** Mặt nạ 02 **(f)** Nền 02

(g) Nguồn 03 **(h)** Mặt nạ 03 **(i)** Nền 03

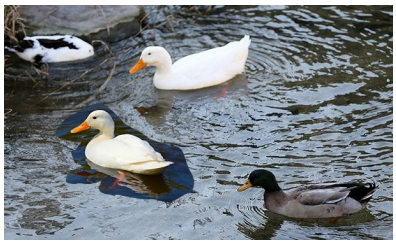
Hình 4.1: Ảnh đầu vào của ba cặp thử nghiệm (nguồn, mặt nạ vẽ bằng GIMP, nền).

4.2 Kết quả định tính

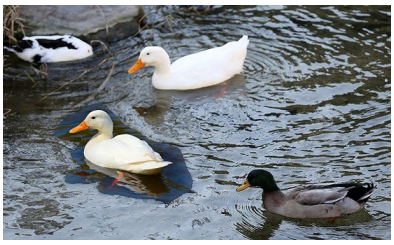
Với mỗi cặp ảnh, hàng trên của hình là toàn ảnh, hàng dưới là vùng ghép được phóng to 2× để quan sát đường biên. Nhận xét theo ba tiêu chí (T1)–(T3) ở Mục 2.3.

4.2.1 Cặp 01: vịt trắng trên nền nước gợn sóng

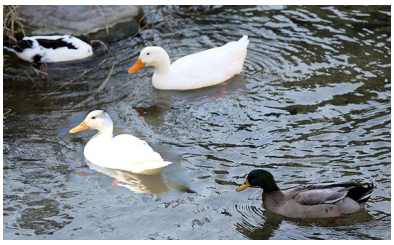
- *Dán trực tiếp:* mảng nước xanh sẫm của ảnh nguồn nổi bật giữa mặt nước xám bạc (T1 và T2 đều kém).



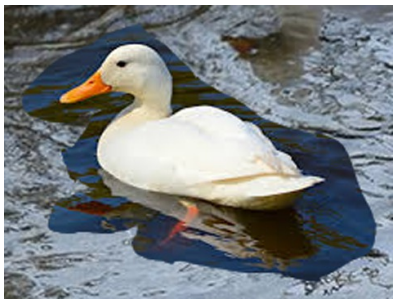
(a) Dán trực tiếp



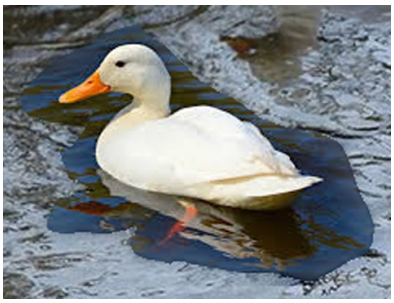
(b) Laplacian (L=3)



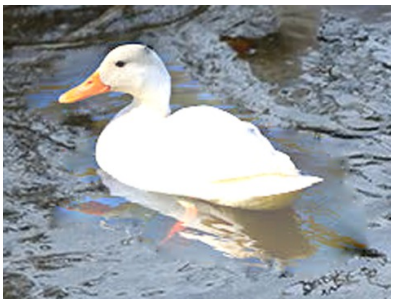
(c) Poisson



(d) Dán trực tiếp (phóng to)



(e) Laplacian (phóng to)



(f) Poisson (phóng to)

Hình 4.2: Kết quả trên cặp `01`.

- *Laplacian:* biên được làm mờ hơn nhưng mảng nước tối vẫn còn nguyên, vì phương pháp không đổi màu bên trong mặt nạ.
- *Poisson:* mảng tối biến mất, nước trong vùng ghép mang màu của nền và con vịt liền mạch với nền (T1, T2 tốt). Tuy nhiên thân vịt trở nên rất sáng, gần như trắng phẳng, mất chi tiết lông (T3 kém đi); còn sót vệt xanh nhạt ở ngực vịt và vùng bóng phản chiếu bên dưới, là gradient của mặt nước nguồn được mang sang.

4.2.2 Cặp `02`: chim cánh cụt trên nền băng

- *Dán trực tiếp:* quầng tuyết trắng bao quanh chim cánh cụt, khác hẳn màu băng và nước xanh của nền.
- *Laplacian:* quầng trắng mềm hơn nhưng vẫn thấy rõ quanh cánh và thân.
- *Poisson:* quầng trắng biến mất, chim cánh cụt hòa vào nền (T1, T2 tốt). Đổi lại thân chim tối hơn và ngả xanh đen, ít thấy vân lông hơn so với ảnh nguồn (T3 kém đi).

4.2.3 Cặp `03`: máy bay trên nền trời

- *Dán trực tiếp:* mảng trời xanh tím của ảnh nguồn bao quanh máy bay, lộ rõ trên nền trời xanh lam.
- *Laplacian:* mảng trời nhạt hơn nhưng vẫn còn thành một quầng sáng quanh máy bay.
- *Poisson:* bầu trời liền mạch, không thấy đường ghép (T1, T2 tốt); chi tiết dây giằng và cánh vẫn rõ. Màu của máy bay ngả nhẹ sang vàng lục so với ảnh nguồn do màu trời nền lan vào.



(a) Dán trực tiếp



(b) Laplacian (L=3)



(c) Poisson



(d) Dán trực tiếp (phóng to)



(e) Laplacian (phóng to)



(f) Poisson (phóng to)

Hình 4.3: Kết quả trên cặp 02.



(a) Dán trực tiếp



(b) Laplacian (L=3)



(c) Poisson



(d) Dán trực tiếp (phóng to)



(e) Laplacian (phóng to)



(f) Poisson (phóng to)

Hình 4.4: Kết quả trên cặp 03.

Bảng 4.3: Tổng hợp định tính trên ba cặp ảnh theo các tiêu chí (T1)–(T3) của Mục 2.3.

Phương pháp	**(T1) Liên tục tại biên**	**(T2) Nhất quán màu**	**(T3) Bảo toàn chi tiết**
Dán trực tiếp	Kém: mảng/quầng nền hiện rõ ở cả ba cặp	Kém: giữ màu nền của nguồn	Tốt: giữ nguyên chi tiết nguồn
Laplacian (L=3)	Trung bình: biên mờ hơn nhưng vẫn thấy quầng	Kém đến trung bình: vẫn giữ màu nền của nguồn	Tốt
Poisson	Tốt: không còn đường ghép ở cả ba cặp	Tốt, nhưng màu nền lan vào vật thể	Trung bình: mất chi tiết khi bão hòa (01, 02); ổn ở 03

4.3 Phân tích bằng số liệu

4.3.1 Chỉ số đường ghép

Bảng 4.4 báo cáo chỉ số S (định nghĩa ở (2.13); càng nhỏ càng ít lộ đường ghép) cùng giá trị tham chiếu của chính ảnh nền.

Bảng 4.4: Chỉ số đường ghép (thang 0–255, càng nhỏ càng tốt). Cột "Ảnh nền gốc" là mức biến thiên tự nhiên để tham chiếu.

Cặp ảnh	**Ảnh nền gốc**	**Dán thẳng**	**Laplacian** (L=3)	**Poisson**	**Poisson-mix**
S: chỉ tính trên các điểm nền của ảnh nguồn					
01	26.2	83.5	57.8	**19.2**	26.6
02	12.0	29.2	23.0	**8.0**	12.7
03	3.9	29.5	19.6	**4.1**	4.5
S_{all}: tính trên toàn bộ dải trong sát biên (kể cả điểm thuộc vật thể)					
01	26.5	80.1	56.5	**23.9**	30.3
02	12.4	48.0	42.6	**33.0**	36.8
03	4.1	33.9	23.4	**9.1**	9.5

- Trên điểm nền của nguồn (S), Poisson giảm chỉ số xuống còn 19.2, 8.0, 4.1 so với 83.5, 29.2, 29.5 của dán trực tiếp, tức giảm 77%, 73% và 86%; so với Laplacian (57.8, 23.0, 19.6) chỉ số của Poisson chỉ còn khoảng 1/5–1/3.
- Giá trị của Poisson nằm ở mức biến thiên tự nhiên của ảnh nền (26.2, 12.0, 3.9) hoặc thấp hơn: dải sát biên không còn khác nền một cách bất thường.

- Nếu tính trên toàn dải (S_{all}), Poisson của cặp `02` là 33.0 (so với 48.0 của dán trực tiếp), cao hơn nhiều so với 8.0: mặt nạ của cặp này ôm sát chim cánh cụt nên dải trong chứa cả lông đen của chính con chim. Đó là biên thật của vật thể chứ không phải đường ghép, và là lý do S chỉ xét điểm nền của nguồn.

4.3.2 Kích thước hệ phương trình và thời gian chạy

Bảng 4.5: Kích thước hệ Poisson cho một kênh màu và thời gian chạy (trung vị của 5 lần đo, 1 lần chạy khởi động bỏ đi). n: số ẩn của ma trận; nnz: số phần tử khác 0.

	Hệ phương trình (mỗi kênh)				Thời gian chạy (ms)			
Cặp ảnh	n	$\lvert\Omega\rvert$	nnz	$\lVert A\mathbf{f}-\mathbf{b}\rVert_\infty$	Dán	Laplacian	Poisson	Poisson-mix
`01`	50 325	19 036	126 469	$\lesssim 10^{-14}$	0.5	45	230	240
`02`	153 600	8 971	189 484	$\lesssim 10^{-14}$	1.1	117	213	245
`03`	63 600	15 603	126 012	$\lesssim 10^{-14}$	0.6	55	196	214

Các hệ được giải chính xác đến sai số làm tròn ($\lVert A\mathbf{f}-\mathbf{b}\rVert_\infty \lesssim 10^{-14}$). Trong môi trường đo, Poisson chậm hơn Laplacian blending vài lần và chậm hơn dán trực tiếp hàng trăm lần, nhưng vẫn dưới một giây với các ảnh cỡ này. Thời gian không tỉ lệ đơn giản với n hay $\lvert\Omega\rvert$ (cặp `02` có n lớn nhất nhưng $\lvert\Omega\rvert$ nhỏ nhất).

4.4 Ảnh hưởng của tham số

4.4.1 Số mức của Laplacian pyramid

Bảng 4.6: Chỉ số đường ghép S của Laplacian blending theo số mức L; cột cuối là Poisson để so sánh.

Cặp ảnh	L=0	L=1	L=2	L=3	L=4	L=5	**Poisson**
`01`	83.6	77.3	69.4	57.8	49.6	46.5	**19.2**
`02`	29.1	26.0	23.1	23.0	26.3	29.8	**8.0**
`03`	29.2	27.1	23.8	19.6	16.4	15.1	**4.1**

Tăng L làm quầng sáng quanh vật thể mờ dần và rộng ra (Hình 4.5). Chỉ số S giảm đều ở cặp `01` (83.6 → 46.5) và `03` (29.2 → 15.1) nhưng đến $L = 5$ vẫn cao gấp khoảng 2.4 và 3.7 lần Poisson. Ở cặp `02`, S giảm đến $L = 2$–3 (23.0) rồi tăng lại (29.8 ở $L = 5$); khi quan sát thấy ở $L = 5$ quầng sáng mờ lan rộng hơn nên một nguyên nhân có thể là vùng chuyển tiếp quá rộng làm màu tuyết của nguồn lan ra xa hơn vào nền (đây là giả thuyết, chưa được kiểm chứng riêng).



(a) $L = 0$ (tương đương dán trực tiếp)



(b) $L = 1$



(c) $L = 3$ (mặc định)



(d) $L = 5$

Hình 4.5: Laplacian blending trên cặp `03` với số mức khác nhau (phóng to vùng ghép).

4.4.2 Gradient trộn so với nhập gradient

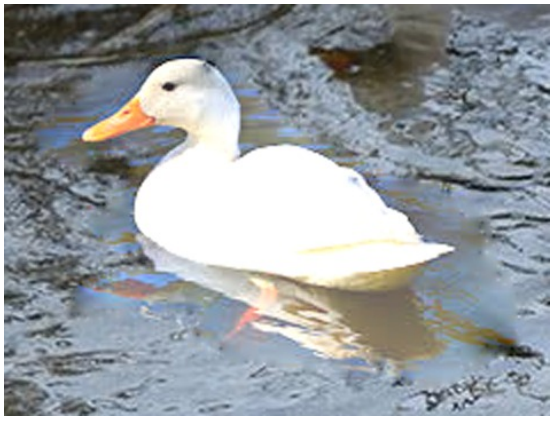
Trên cặp `01` (nền nước nhiều gợn sóng), gradient trộn làm các gợn sóng của nền xuyên qua thân vịt khiến con vịt mờ ảo và có vẻ trong suốt; chỉ số S tăng từ 19.2 lên 26.6. Trên cặp `03` (nền trời mượt, ít gradient), hai kết quả gần như trùng nhau (4.1 so với 4.5). Vậy gradient trộn chỉ khác biệt khi nền có gradient đủ mạnh để thắng gradient của nguồn.

4.5 Kiểm chứng lý thuyết

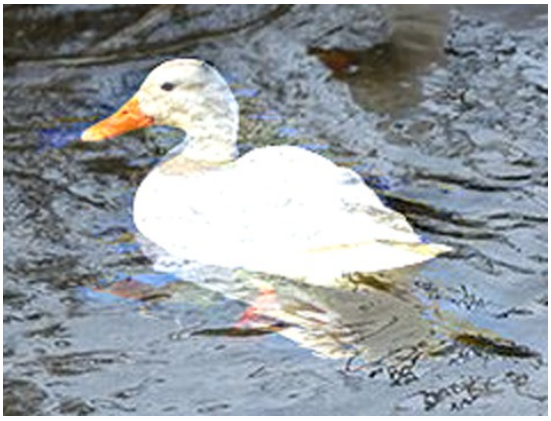
4.5.1 Thí nghiệm có đáp án

Theo Mục 2.3.3, ảnh nền `01` (nén dải động về $[0.25, 0.75]$ để phép dịch không gây bão hòa) được dùng làm đáp án f_0; trên một vùng elip Ω cỡ 140×190 điểm ảnh, ảnh nguồn lệch so với f_0 một độ dịch màu hằng số $(+0.12, -0.08, +0.10)$ theo thứ tự B, G, R, hoặc độ dịch đó cộng thêm một dốc tuyến tính ± 0.08 theo phương ngang. Cả hai độ lệch là hàm điều hòa.

Dán trực tiếp và Laplacian chỉ đạt PSNR khoảng 21–22 dB và sai số trung bình khoảng 25 mức xám trong Ω: chúng mang theo toàn bộ độ lệch màu của nguồn (Laplacian chỉ làm mờ đường biên, SSIM cao hơn một chút). Poisson khôi phục chính xác ảnh gốc cho cả hai loại độ lệch (SSIM = 1.0000, MAE = 0), đúng như tính chất (iv) của Mục 2.1.4. Đây cũng là một phép kiểm tra hiện thực: nếu việc dựng A, $\mathbf{b}$ hay giải hệ có lỗi, kết quả sẽ không trùng ảnh gốc.



(a) `01`: nhập gradient



(b) `01`: gradient trộn



(c) `03`: nhập gradient



(d) `03`: gradient trộn

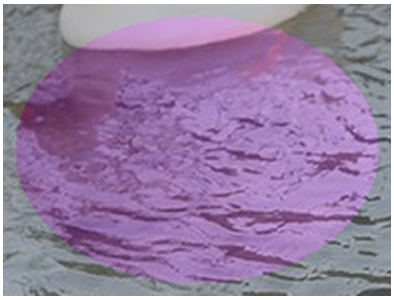
Hình 4.6: Poisson với nhập gradient (trái) và gradient trộn (phải) trên hai cặp có nền khác nhau.



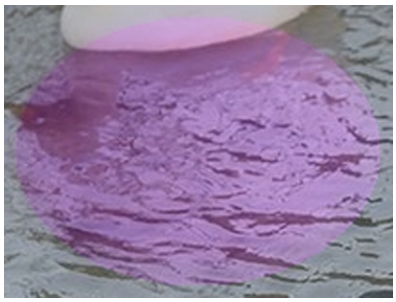
(a) Đáp án f_0



(b) Nguồn lệch g (hằng số + dốc)



(c) Dán trực tiếp



(d) Laplacian (L=3)



(e) Poisson

Hình 4.7: Thí nghiệm có đáp án (vùng ghép phóng to): chỉ Poisson trả về đúng ảnh gốc.

Bảng 4.7: Kết quả thí nghiệm có đáp án. PSNR và SSIM đo trên hộp bao của Ω; MAE đo trong Ω (thang 0–255). ∞: hai ảnh trùng nhau từng điểm ảnh sau khi lượng tử hóa 8 bit.

Sai lệch của nguồn	Phương pháp	PSNR (dB) ↑	SSIM ↑	MAE trong Ω ↓
Hằng số	Dán thẳng	21.6	0.9518	25.50
	Laplacian (L=3)	21.8	0.9648	24.74
	Poisson	∞	**1.0000**	**0.00**
Hằng số + dốc tuyến tính	Dán thẳng	21.0	0.9509	25.50
	Laplacian (L=3)	21.3	0.9624	24.70
	Poisson	∞	**1.0000**	**0.00**

4.5.2 Bản đồ hiệu chỉnh và hiện tượng bão hòa

Theo (2.3), độ chênh $h = f - g$ phải là một hàm trơn, không chứa cấu trúc của đối tượng. Hình 4.8 vẽ h (trước khi cắt về $[0, 1]$) cho ba cặp ảnh, và Bảng 4.8 nêu khoảng giá trị của nó.



(a) Cặp `01` **(b)** Cặp `02` **(c)** Cặp `03`

Hình 4.8: Bản đồ hiệu chỉnh $h = f - g$ trong Ω, tô màu $0.5 + h/2$ (xám trung tính nghĩa là không đổi; ngoài Ω để trắng).

Cả ba bản đồ đều là các trường màu biến thiên chậm, không còn hình dạng của vật thể; Laplace rời rạc của h trên Ω bằng 0 đến mức sai số làm tròn `float32` ($\lesssim 10^{-6}$), tức h điều hòa đúng như lý thuyết. Độ lớn của h phản ánh mức chênh lệch màu ở biên: lớn nhất ở cặp `01` (+0.68) và `02` (−0.78), nhỏ ở cặp `03`. Với hai cặp đầu, khoảng 30% giá trị của f rơi ra ngoài $[0, 1]$ và bị cắt. Sự bão hòa tập trung đúng vào những vùng bị ảnh hưởng ở Mục 4.2: ở cặp `01`, 97% số điểm sáng của nguồn (thân vịt) bị đẩy vượt 1 nên trở thành trắng phẳng; ở cặp `02`, 75% số điểm tối (thân chim cánh cụt) bị đẩy xuống dưới 0 nên đen phẳng và mất vân lông.

Bảng 4.8: Khoảng giá trị của $h = f - g$ trong Ω (thang $[0,1]$, gộp ba kênh); tỉ lệ giá trị của f nằm ngoài $[0,1]$ trước khi cắt (tính trên điểm ảnh và kênh); tỉ lệ điểm sáng (độ sáng trung bình của nguồn > 0.75) có ít nhất một kênh vượt 1 và điểm tối (< 0.25) có ít nhất một kênh xuống dưới 0; và kiểm tra tính điều hòa của h.

Cặp ảnh	$\min h$	$\max h$	**Giá trị ngoài** $[0,1]$	**Điểm sáng vượt** 1	**Điểm tối xuống** < 0	$\max \lvert\Delta h\rvert$
01	-0.30	$+0.68$	29.7%	96.9%	0.0%	$\lesssim 10^{-6}$
02	-0.78	$+0.09$	30.3%	0.0%	75.0%	$\lesssim 10^{-6}$
03	-0.04	$+0.27$	3.6%	69.3%	0.0%	$\lesssim 10^{-6}$

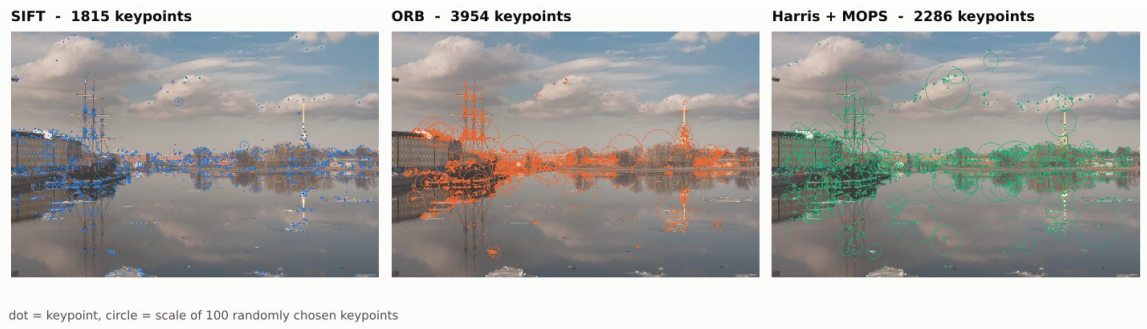
4.6 Phần 2: Dữ liệu và thiết lập thí nghiệm

Bảng 4.9 mô tả năm bộ ảnh thật (ảnh kiểm thử ghép ảnh của OpenCV, Mục 1.3.4).

Bảng 4.9: Năm bộ ảnh thật (ảnh kiểm thử ghép ảnh của OpenCV). Tỉ lệ chồng lấn là phần diện tích của một ảnh nằm trong ảnh kế bên, tính từ homography của SIFT trên các cặp của cây khung (khoảng giá trị của hai chiều); `scripts/describe_scenes.py` liệt kê mọi cặp.

Bộ ảnh	**Số ảnh**	**Kích thước gốc (rộng × cao)**	**Sau khi thu nhỏ**	**Chồng lấn**	**Đặc điểm**
`aqueduct`	2	1246×700, 1385×700	1000×562, 1000×505	60–62%	ngoài trời, kết cấu dày (đá, cây, nước)
`boat`	4	3888×2592	1000×667	48–66%	sông, bầu trời, bóng phản chiếu, băng trôi; trường nhìn rộng
`cathedral`	3	600×768	giữ nguyên	68–70%	trong nhà, tối; **ảnh đầu là ảnh xám**
`mountain`	2	800×566	giữ nguyên	45%	núi tuyết, ít kết cấu; **ảnh đầu là ảnh xám**
`newspaper`	4	818×1125	727×1000	44–73%	tờ báo (mặt phẳng), chữ lặp lại

Tất cả chạy với cấu hình mặc định (Phụ lục D): cùng ngân sách 4000 điểm đặc trưng cho mỗi ảnh, ratio test 0.75, RANSAC ngưỡng 3 px, hòa trộn đa băng tần 5 bậc, không bù sáng. Các độ đo cho ảnh thật và tiêu chí thành công của nghiên cứu có kiểm soát được định nghĩa ở Mục 2.10.



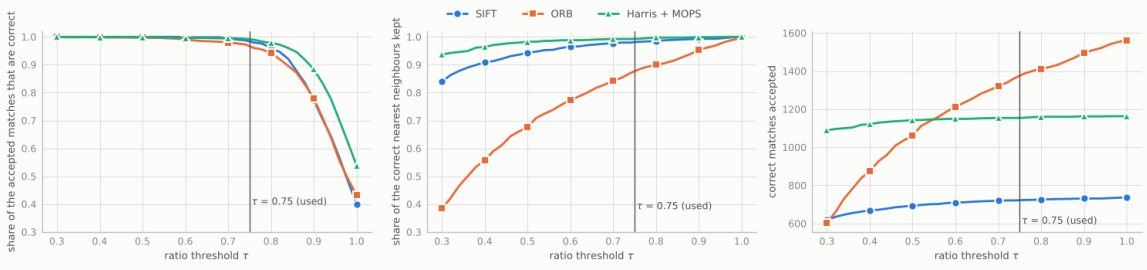
Hình 4.9: Điểm đặc trưng của ba bộ trích xuất trên ảnh `boat1` (1815 điểm SIFT, 3954 ORB, 2286 Harris + MOPS). Mỗi chấm là một điểm đặc trưng; vòng tròn là kích thước (tỉ lệ) của 100 điểm chọn ngẫu nhiên. Cả ba tập trung ở tòa nhà, con tàu, đường chân trời và bóng phản chiếu; bầu trời và mặt nước gần như không có điểm.

4.7 Kiểm chứng từng khâu của pipeline

4.7.1 Điểm đặc trưng

4.7.2 So khớp: tác dụng của ratio test

Với một cặp góc nhìn tổng hợp có hình học đã biết (hai góc nhìn cách nhau 8°, ảnh 800×600), một cặp khớp là *đúng* nếu homography thật đưa điểm của ảnh 1 đến trong vòng 3 px của điểm khớp ở ảnh 2. Hình 4.10 và Bảng 4.10 cho thấy ratio test làm được gì: khi không có nó ($\tau = 1$) chỉ khoảng 40–54% số cặp được nhận là đúng; ở $\tau = 0.75$ độ tinh khiết đạt 97–99% mà vẫn giữ lại 87–99% số láng giềng gần nhất đúng.



Hình 4.10: Tác dụng của ngưỡng ratio test τ trên một cặp tổng hợp cách nhau 8°. Trái: tỉ lệ cặp khớp được nhận mà đúng (độ tinh khiết). Giữa: tỉ lệ các láng giềng gần nhất đúng còn được giữ (độ phủ). Phải: số cặp khớp đúng được nhận. Vạch đứng: $\tau = 0.75$ đang dùng.

4.7.3 RANSAC và Levenberg–Marquardt

Bảng 4.11 kiểm tra công thức (2.19) và độ bền: với $w = 0.5$ thì $N = 83$ và 99.6% trong 1500 thí nghiệm có ít nhất một mẫu sạch (mục tiêu 99.5%); số mẫu thực dùng lớn hơn N của công thức một chút vì cận thích nghi chỉ giảm sau khi đã tìm được một giả thuyết tốt và vì các mẫu suy biến bị bỏ cũng được tính. Ở 80% ngoại lai công thức đòi 3309 mẫu nhưng trần 3000 vẫn thành công (xác suất có một mẫu sạch là $1-(1-0.2^4)^{3000} = 99.2\%$);

Bảng 4.10: Ratio test ở $\tau = 0.75$ trên cặp tổng hợp cách nhau 8°.

Bộ trích xuất	Điểm đặc trưng (ảnh 1)	Cặp được nhận	Trong đó đúng	Độ tinh khiết	Độ phủ
SIFT	1838	730	721	98.8%	98.0%
ORB	3915	1403	1365	97.3%	87.0%
Harris + MOPS	2160	1162	1154	99.3%	99.2%

ở 90% trần thấp hơn nhiều so với 52981 mẫu cần có và RANSAC hỏng trong 8 trên 12 lần chạy. Trần này đánh đổi độ bền lấy thời gian và vô hại ở đây, vì khâu so khớp cho tỉ lệ nội điểm 0.8–1.0 trên các cặp thật (Mục 4.8); ngược lại, bình phương tối thiểu hỏng ngay ở 10% ngoại lai (sai số trung vị 99 px).

Bảng 4.11: Số mẫu RANSAC cần thiết và độ bền với ngoại lai (12 lần chạy mỗi hàng, tối đa 3000 mẫu; "đúng" nghĩa là sai số < 1 px).

Tỉ lệ ngoại lai	N theo công thức ($p = 0.995$)	Số mẫu thực dùng (trung vị)	RANSAC đúng	DLT thường: sai số trung vị (px)
10%	5	10	12/12	99
30%	20	23	12/12	373
50%	83	115	12/12	2862
70%	652	1200	12/12	7459
80%	3309	3000	12/12	14110
90%	52981	3000	4/12	12613

Trên dữ liệu thật, LM thay đổi RMSE chưa đến 0.01 px (Bảng 4.12): với phối cảnh nhẹ các trọng số w_i gần như hằng, và các nội điểm đã được chọn từ trước, nên DLT chuẩn hóa đã gần cực tiểu của sai số hình học. LM vẫn được giữ vì chỉ tốn vài mili-giây và bảo đảm kết quả là cực tiểu của tiêu chí *đúng* khi phối cảnh mạnh hoặc có ít cặp khớp. Độ chính xác thực sự đến từ **định vị điểm đặc trưng** (so RMSE của ba bộ trích xuất ở Bảng 4.12) và từ **việc chọn nội điểm**.

Bảng 4.12: Hai ảnh đầu của bộ `boat` với ba bộ trích xuất (ratio test 0.75 + lọc một–một, RANSAC ngưỡng 3 px). RMSE là sai số tái chiếu RMS của các nội điểm.

Bộ trích xuất	Điểm đặc trưng (ảnh 1 / ảnh 2)	Cặp khớp	Nội điểm	Tỉ lệ	Số mẫu RANSAC	RMSE trước LM (px)	RMSE sau LM (px)
SIFT	1815 / 1328	543	513	0.94	8	0.465	0.465
ORB	3954 / 3823	741	699	0.94	5	1.124	1.124
Harris + MOPS	2286 / 1786	821	775	0.94	4	0.658	0.658

4.7.4 Cặp ảnh giả: bộ `newspaper`

Bộ `newspaper` là ca khó vì các dòng chữ lặp lại. Ảnh 0 và ảnh 2 gần như không chồng lấn, nhưng SIFT vẫn tìm được các cặp khớp giữa những chữ cái giống nhau (Bảng 4.13). Với ratio test thường, RANSAC tìm được 42 "nội điểm", **tất cả đều ánh xạ vào cùng một điểm đặc trưng của ảnh** 2. Chỉ riêng số đếm ($42 \geq 20$) đã đủ để nhận cặp; phép kiểm tra hạng thấy $\sigma_8/\sigma_1 = 0$ và loại nó. Với lọc một–một, các cặp dồn về một điểm biến mất và chỉ còn 7 nội điểm, cũng bị loại.

Bảng 4.13: Cặp giả của bộ `newspaper` (ảnh 0 và 2, SIFT).

Cách so khớp	Số cặp khớp	Đối tác khác nhau ở ảnh 2	Nội điểm RANSAC	Điểm khác nhau của ảnh 2 trong số nội điểm	σ_8/σ_1	Kết luận (đếm + hạng)
ratio test thường	194	126	42	**1**	0	bị loại
thêm lọc một–một	126	126	7	7	$1.5 \cdot 10^{-4}$	bị loại

4.7.5 Đồ thị chồng lấn

Hình 4.11 minh họa cây khung với bộ `cathedral`: ảnh 1 ở giữa là ảnh tham chiếu và hai cạnh của cây có 1025 và 949 nội điểm.



Hình 4.11: Bộ `cathedral` (SIFT). Trái: dấu chân của ba ảnh trên canvas và cây khung (vòng đậm là ảnh tham chiếu; cạnh 1–2 có 1025 nội điểm, cạnh 0–1 có 949). Phải: ba ảnh đã warp lên canvas, trước khi hòa trộn (ảnh 0 là ảnh xám).

4.7.6 Phép warp

`bilinear_sample` so với `cv2.remap`: sai khác tối đa $2.4 \cdot 10^{-3}$ mức xám trên 20000 mẫu. `warp_perspective` so với `cv2.warpPerspective`: sai khác trung bình 0.287 mức xám, 99.9% số pixel dưới 0.5 (OpenCV nội suy bằng số nguyên cố định).

4.8 Kết quả trên năm bộ ảnh thật

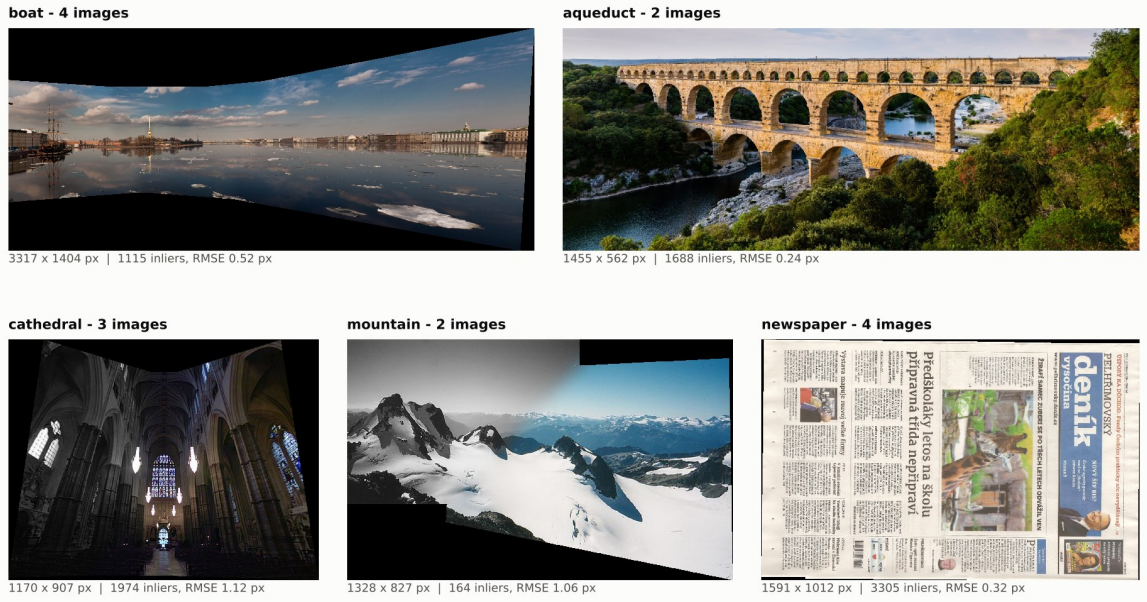
Hình học. SIFT có sai số tái chiếu nhỏ nhất ở cả năm bộ ảnh (trung bình 0.65 px, so với 0.83 px của Harris + MOPS và 1.08 px của ORB): chênh lệch lớn ở các bộ ảnh sắc nét, độ phân giải cao (`aqueduct`, `boat`, `newspaper`: 0.2–0.3 px so với Harris + MOPS và 0.6–0.7 px so với ORB) và nhỏ ở các ảnh nhỏ, mềm (`cathedral`: 0.04–0.08 px; `mountain`:

Bảng 4.14: Kết quả của 15 tổ hợp (5 bộ ảnh × 3 bộ trích xuất). Nội điểm: tổng trên các cặp của cây khung; thời gian là một lần chạy trên máy dùng chung và dao động hàng chục phần trăm giữa các lần chạy, các cột còn lại tái lập được.

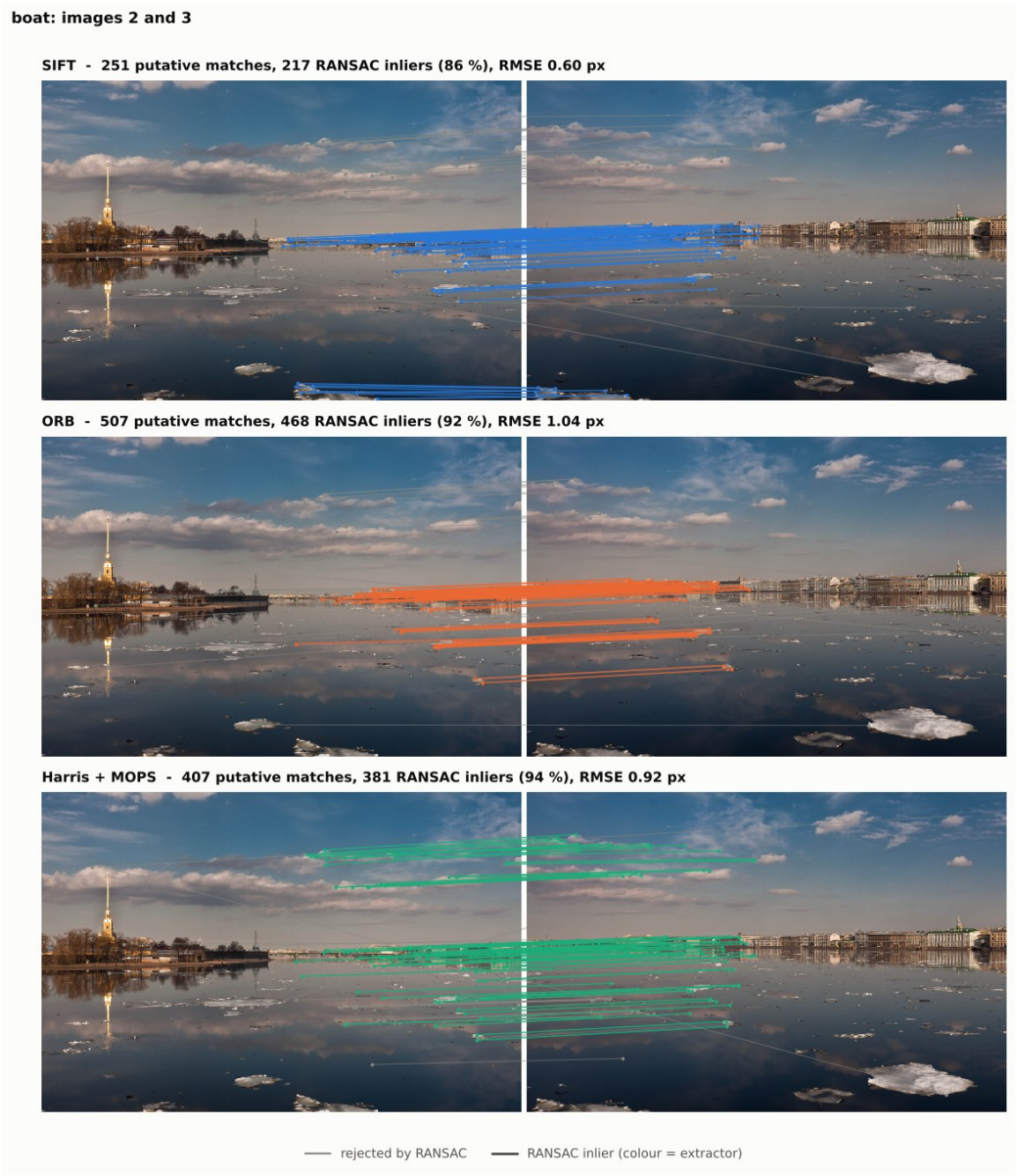
Bộ ảnh	Bộ trích xuất	Điểm / ảnh	Nội điểm	Tỉ lệ	RMSE (px)	NCC thông cao	MAE	Thời gian (s)
aqueduct	SIFT	4000	1688	0.99	**0.24**	0.950	4.4	1.9
aqueduct	ORB	4000	1062	0.95	0.97	0.947	4.5	1.5
aqueduct	Harris + MOPS	4000	1472	0.99	0.55	0.950	4.4	2.7
boat	SIFT	1436	1115	0.88	**0.52**	0.763	10.6	13.1
boat	ORB	3833	2044	0.89	1.09	0.743	10.4	13.3
boat	Harris + MOPS	1889	1828	0.90	0.77	0.735	10.2	12.7
cathedral	SIFT	3778	1974	0.89	**1.12**	0.739	8.7	3.0
cathedral	ORB	3828	2520	0.96	1.15	0.776	9.5	2.8
cathedral	Harris + MOPS	1635	1324	0.92	1.20	0.734	8.9	2.5
mountain	SIFT	1924	164	0.82	**1.06**	0.788	19.9	1.8
mountain	ORB	3840	307	0.89	1.29	0.761	20.0	1.9
mountain	Harris + MOPS	1454	269	0.92	1.15	0.779	19.9	1.6
newspaper	SIFT	4000	3305	0.89	**0.32**	0.927	7.4	6.3
newspaper	ORB	4000	2485	0.90	0.89	0.905	7.9	5.4
newspaper	Harris + MOPS	4000	4310	0.98	0.49	0.925	7.4	8.2

Bảng 4.15: Trung bình trên năm bộ ảnh.

Bộ trích xuất	Điểm / ảnh	Tỉ lệ nội điểm	RMSE (px)	NCC thông cao	MAE	Thời gian (s)
SIFT	3028	0.89	**0.65**	0.833	10.2	5.2
ORB	3900	0.92	1.08	0.826	10.5	5.0
Harris + MOPS	2596	**0.94**	0.83	0.825	10.2	5.6



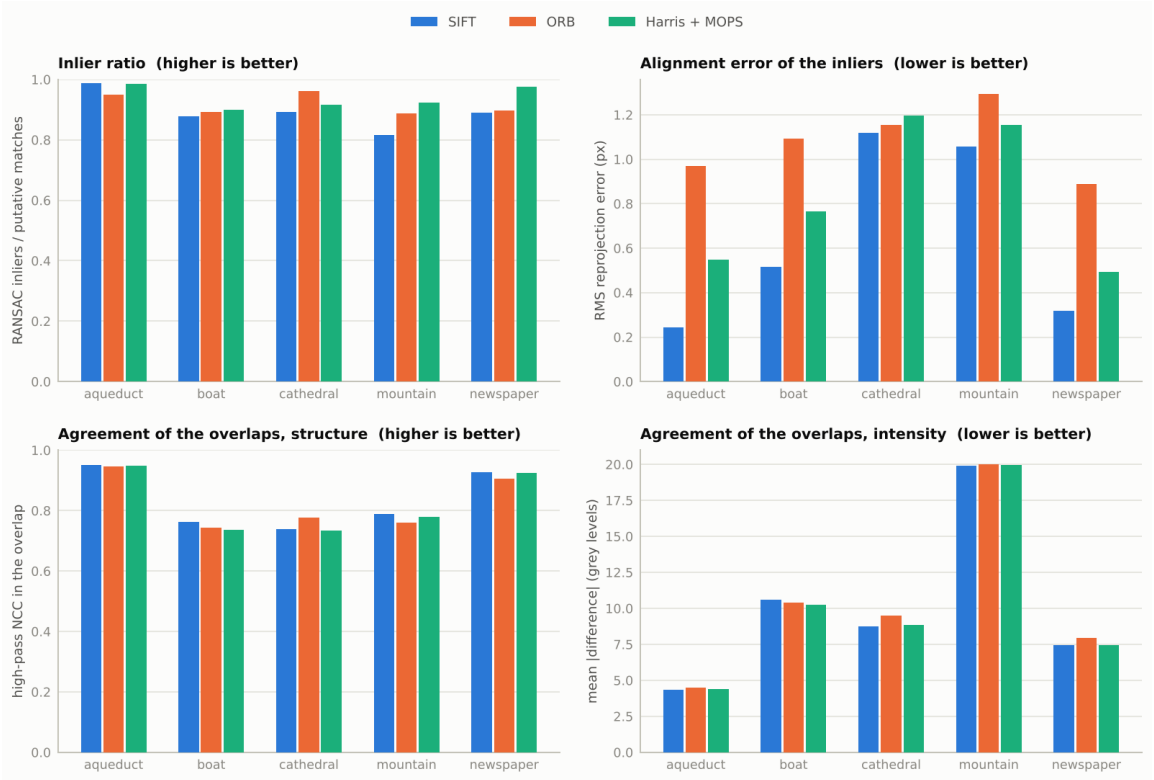
Hình 4.12: Panorama của năm bộ ảnh (SIFT, hòa trộn đa băng tần, tham số mặc định). Vùng đen là phần canvas không có ảnh; dưới mỗi ảnh là kích thước panorama, số nội điểm trên cây khung và RMSE.



Hình 4.13: Cặp ảnh 2–3 của bộ `boat` (cặp có chồng lấn nhỏ nhất của cây khung SIFT, khoảng 49%) được khớp bởi ba bộ trích xuất. Đường màu: nội điểm RANSAC; đường xám: cặp bị RANSAC loại. Nội điểm nằm chủ yếu ở đường bờ và bóng phản chiếu của nó, cùng một ít ở mây và mép băng; mặt nước trống hầu như không có nội điểm.



Hình 4.14: Bộ `cathedral` với ba bộ trích xuất. Ba panorama gần như giống hệt nhau; dưới mỗi hình là số nội điểm trên cây khung (tỉ lệ), RMSE và NCC thông cao của vùng chồng lấn.



Hình 4.15: Năm bộ ảnh × ba bộ trích xuất: tỉ lệ nội điểm, sai số tái chiếu RMS của nội điểm, NCC thông cao và sai khác độ chói trung bình trong vùng chồng lấn.

0.10–0.24 px). Điều này khớp với cách các điểm được định vị: SIFT khớp một hàm bậc hai vào không gian tỉ lệ DoG (dưới pixel theo x, y và tỉ lệ); Harris + MOPS khớp một parabola vào đáp ứng góc của bậc kim tự tháp; ORB báo các góc FAST ở vị trí *nguyên* của bậc kim tự tháp, nên sai số tăng theo bậc. *Tỉ lệ nội điểm* (độ sạch của các cặp khớp) xếp hạng khác đi: Harris + MOPS cao nhất ở ba bộ (`boat`, `mountain`, `newspaper`) và cao nhất về trung bình (0.94, so với 0.92 của ORB và 0.89 của SIFT); một tập cặp khớp sạch chưa chắc có vị trí chính xác.

Độ khớp của vùng chồng lấn. Sau khi hòa trộn, ba bộ trích xuất gần như không phân biệt được: NCC thông cao của vùng chồng lấn chênh nhau không quá 0.05 trong mỗi cảnh và MAE chênh dưới một mức xám. Hai con số này chủ yếu đo *cảnh*, chứ ít đo bộ trích xuất; Mục 4.9 phân tích từng bộ ảnh.

4.9 Phân tích từng bộ ảnh: thành công và khó khăn

- `aqueduct` — **trường hợp dễ nhất.** Hai ảnh, chồng lấn 60%, kết cấu dày (đá, cây, nước); 95–99% cặp khớp là nội điểm, NCC 0.95 cho cả ba bộ trích xuất và đường nối không thấy được trong panorama. SIFT chính xác nhất (0.24 px), ORB kém nhất (0.97 px) nhưng không gây khác biệt nhìn thấy được.
- `newspaper` — **mặt phẳng, và chữ lặp lại.** Với tờ báo, homography đúng cho mọi chuyển động của camera (Mục 2.4), nên đây là bộ có NCC cao ở mọi dải của vùng chồng lấn (0.86–0.97). Các cặp 0–2 và 0–3 chồng lấn (theo chuỗi homography) chỉ 7% và 0% diện tích và bị cả ba bộ trích xuất loại (không quá 7 nội điểm, Mục 4.7.4), còn ba cặp liền kề tạo thành chuỗi; cặp 1–3 (35% diện tích) được nhận nhưng không nằm trong cây khung và khớp với chuỗi đến 0.08–0.26 px (Mục 4.13). Chữ lặp là nguồn của cặp giả ở Bảng 4.13. SIFT chính xác nhất (0.32 px); ORB 0.89 px.
- `boat` — **trường nhìn rộng, nước chuyển động.** Bốn ảnh 3888×2592 cho panorama 3317×1404 px. NCC chỉ 0.74–0.76, và con số trung bình này che hai hiện tượng khác hẳn nhau. Để tách chúng, mỗi dải của vùng chồng lấn được so lại sau khi cho phép dịch ảnh thứ hai tới ±12 px (Bảng 4.16; `scripts/analyse_overlap_bands.py`): nếu một phép dịch nhỏ khôi phục được tương quan thì căn chỉnh đang lệch, nếu không thì nội dung đã đổi. *Nội dung thay đổi*: ở các cặp 0–1 và 1–2, bốn dải trên (mây, đường chân trời, bóng phản chiếu của bờ) tương quan 0.79–0.95 và phép dịch tối ưu bằng 0: căn chỉnh chính xác; hai dải dưới (mặt nước có băng trôi) tương quan gần 0 và không phép dịch nào nâng được quá 0.46: băng và sóng nước đã đổi chỗ giữa các lần bấm máy. *Căn chỉnh lệch ở nơi không có nội điểm*: cặp 2–3 thì ngược lại: các dải nước tương quan tốt (0.75–0.89) còn hai dải trên (bầu trời) gần 0 (-0.07 và -0.04); dịch ảnh $(-7, -4)$ px ở dải 2 và $(-8, -9)$ px ở dải 1 khôi phục được một phần (NCC lên 0.52 và 0.13). Đó không phải do bầu trời quá trơn, vì dải trên cùng của cặp 1–2 có độ lệch chuẩn thông cao tương đương (1.9 so với 1.8 mức xám) mà vẫn tương quan 0.79. Điều khác biệt

là chỗ có nội điểm: 95% nội điểm của cặp 2–3 nằm ở nửa dưới của vùng chồng lấn, từ 48% chiều cao trở xuống (Hình 4.13), nên bầu trời nằm xa mọi ràng buộc của homography. Số liệu phù hợp với một homography bị ngoại suy lệch ở vùng không có nội điểm, nhưng không loại trừ hẳn khả năng mây trôi giữa hai lần chụp. Panorama bị uốn cong (vùng đen phía trên và dưới có dạng đường cong): đó là hệ quả của phép chiếu lên một mặt phẳng khi trường nhìn rộng (Mục 5.6.2), không phải lỗi của homography. Cặp 0–3 không chồng lấn (0% theo chuỗi homography) nên cả ba bộ trích xuất loại nó là đúng; cặp 1–3 chỉ chung 14% diện tích, SIFT và Harris + MOPS nhận (30 và 31 nội điểm) còn ORB loại (10 nội điểm), và cặp này không nằm trong cây khung.

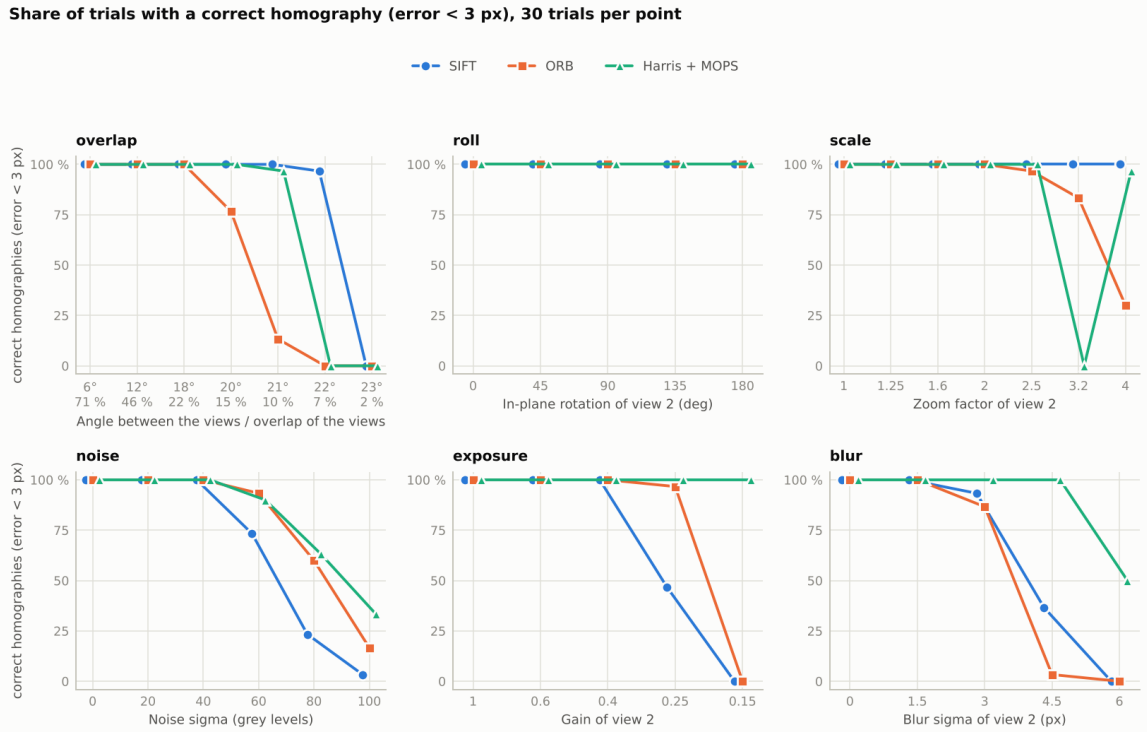
- `cathedral` — **trong nhà, tối, ảnh đầu là ảnh xám.** Cả ba bộ trích xuất cho panorama giống nhau (Hình 4.14); ORB lấy được nhiều nội điểm nhất (2520) trong khi sai số RMS của ba bộ gần nhau (1.12–1.20 px). Mép trên của panorama lõm hình chữ V vì hai ảnh bên bị kéo cao hơn ảnh giữa: đó là hệ quả của phép chiếu lên mặt phẳng của ảnh tham chiếu (Mục 5.6.2), không phải lỗi căn chỉnh; cột và vòm ở các chỗ nối liền nét.
- `mountain` — **ít nội điểm nhất, ảnh xám lẫn ảnh màu.** Chỉ 164–307 nội điểm mặc dù chồng lấn tới 45%: ảnh chủ yếu là tuyết nên rất ít kết cấu. MAE khoảng 20 mức xám ở cả ba bộ trích xuất là tính chất của *dữ liệu*: ảnh thứ nhất là ảnh xám với bầu trời sáng hơn nhiều so với bầu trời xanh của ảnh thứ hai, và một hệ số gain cho mỗi ảnh chỉ có thể co giãn cường độ: nó giảm MAE từ 19.9 xuống 17.1 mức xám (gain 1.035 và 0.963) chứ không xóa được khác biệt. Cả hai panorama `mountain` và `cathedral` đều có đoạn chuyển từ xám sang màu (Hình 4.12, Hình 4.14); các bộ trích xuất chỉ nhìn độ chói nên hình học vẫn được ước lượng bình thường.

Bảng 4.16: Bộ `boat` (SIFT): NCC thông cao trong sáu dải ngang của vùng chồng lấn, từ trên xuống dưới, với ảnh như đã warp (hàng "NCC") và sau phép dịch ảnh thứ hai tốt nhất trong ±12 px (hàng "sau khi dịch"; vector (d_x, d_y) ghi trong ngoặc khi khác 0, nghĩa là ảnh thứ hai được lấy mẫu tại $(x + d_x,\ y + d_y)$ để so với ảnh thứ nhất tại (x, y)).

Cặp		Dải 1	Dải 2	Dải 3	Dải 4	Dải 5	Dải 6
0–1	NCC	0.86	0.90	0.95	0.91	0.28	0.00
	sau khi dịch	0.86	0.90	0.95	0.91	0.32 $(0, +1)$	0.12 $(+7, -7)$
1–2	NCC	0.79	0.91	0.95	0.90	0.30	−0.05
	sau khi dịch	0.79	0.91	0.95	0.90	0.46 $(-7, +1)$	0.22 $(-12, +5)$
2–3	NCC	−0.07	−0.04	0.85	0.87	0.75	0.89
	sau khi dịch	0.13 $(-8, -9)$	0.52 $(-7, -4)$	0.88 $(-1, 0)$	0.87	0.75	0.89

4.10 Nghiên cứu có kiểm soát với đáp án chính xác

Thiết lập. Ba ảnh của bộ `boat` (`boat1`, `boat2`, `boat4`), mỗi ảnh thu về rộng 1944 px (góc nhìn ngang 53° với tiêu cự 1944 px), lần lượt đóng vai "thế giới"; mỗi góc nhìn 800×600 px được dựng bằng camera ảo chỉ xoay, và homography đúng $H_{ij} = K_j R_j^\top R_i K_i^{-1}$ được biết chính xác. Mặc định hai góc nhìn cách nhau 8° (chồng lấn khoảng 66%); mỗi lần chỉ một yếu tố được đẩy dần lên mức khó: **chồng lấn** (góc giữa hai góc nhìn tăng), **xoay trong mặt phẳng ảnh** (roll), **zoom**, **nhiễu Gauss**, **phơi sáng** (gain nhỏ dần) và **nhòe**. Mỗi mức có 30 lần thử (hệ camera được nhiễu loạn ngẫu nhiên nên mỗi lần thấy nội dung khác nhau, luân phiên ba ảnh "thế giới"); tổng cộng 35 mức × 30 = 1050 lần thử, mỗi lần thử được chạy với cả ba bộ trích xuất (≈ 3 × 1050). Tiêu chí thành công và định nghĩa "giới hạn" ở Mục 2.10.



Hình 4.16: Tỉ lệ lần thử có homography đúng (sai số < 3 px) theo từng yếu tố khó, 30 lần thử mỗi điểm. Các trục: góc giữa hai góc nhìn và tỉ lệ chồng lấn tương ứng; góc xoay trong mặt phẳng; hệ số zoom; độ lệch chuẩn nhiễu (mức xám); gain của ảnh 2 (nhỏ hơn = tối hơn); độ lệch chuẩn nhòe (px).

Với "roll" cả ba bộ giải được mọi góc xoay đến 180° (30/30 ở mọi mức), và "exposure" của Harris + MOPS cũng giải được mọi mức; trong hai trường hợp đó bảng ghi mức khó nhất đã thử.

Đọc kết quả (Hình 4.16, Bảng 4.17).

- **Độ chính xác.** Ở các mức dễ (không nhiễu, không nhòe), sai số trung vị của các homography đúng là khoảng 0.03 px với SIFT, 0.03–0.04 px với Harris + MOPS

Bảng 4.17: Giới hạn của từng bộ trích xuất: mức khó nhất vẫn giải được ít nhất 90% trong 30 lần thử (in đậm: tốt nhất).

Yếu tố	SIFT	ORB	Harris + MOPS
chồng lấn của hai góc nhìn	**22°** (**7**%)	18° (22%)	21° (10%)
xoay trong mặt phẳng	180°	180°	180°
zoom của ảnh 2	**4**×	2.5×	2.5×
nhiễu σ (mức xám)	40	**60**	**60**
gain của ảnh 2	0.4	0.25	**0.15**
nhòe σ (px)	3	1.5	**4.5**

và 0.11–0.15 px với ORB, tức ORB kém hơn 3–5 lần. Thứ tự đảo ngược khi nhiễu mạnh: ở $\sigma = 40$ sai số trung vị của SIFT (0.77 px) lớn hơn của ORB (0.52 px) và của Harris + MOPS (0.47 px).

- **Chồng lấn.** SIFT còn giải được khi hai góc nhìn chỉ chung 7% diện tích (22°), Harris + MOPS 10% (21°), ORB dừng ở 22% (18°); ở 23° (2%) không bộ nào giải được.
- **Xoay.** Cả ba bất biến với phép xoay thật sự: 30/30 ở mọi góc, sai số không đổi theo góc.
- **Zoom.** SIFT giải được mọi mức đến 4×, ORB dừng ở 2.5× (25/30 ở 3.2× và 9/30 ở 4×); Harris + MOPS hoàn hảo đến 2.5×, thất bại hoàn toàn ở 3.2× (0/30) rồi lại giải được ở 4× (29/30); hiện tượng được giải thích ở dưới.
- **Nhiễu.** SIFT chịu nhiễu kém nhất (22/30 ở $\sigma = 60$, so với 28/30 của ORB và 27/30 của Harris + MOPS).
- **Phơi sáng và nhòe.** Harris + MOPS giải được mọi mức phơi sáng, kể cả gain 0.15, và chịu nhòe đến $\sigma = 4.5$ px; SIFT dừng ở gain 0.4 và nhòe 3 px; ORB ở gain 0.25 và nhòe 1.5 px.

Bảng 4.18: Lỗi thầm lặng: cặp được chấp nhận nhưng homography sai từ 3 px trở lên (trên 1050 lần thử mỗi bộ).

Bộ trích xuất	Cặp được chấp nhận	Trong đó đúng (< 3 px)	Lỗi thầm lặng	Thuộc nhóm thí nghiệm nhiễu
SIFT	887	862	25	16
ORB	874	797	77	39
Harris + MOPS	955	909	46	30

Ba phép kiểm tra chấp nhận cặp ảnh chặn được cặp giả của bộ `newspaper` nhưng không phải là bảo đảm: Bảng 4.18 đếm các cặp qua được chúng mà homography vẫn sai, nhiều nhất ở ORB, và phần lớn nằm ở rìa dải chịu đựng của mỗi bộ (nhiễu mạnh).

Vì sao SIFT và ORB hỏng ở ảnh tối còn Harris + MOPS thì không? Trong thí nghiệm phơi sáng, ảnh thứ hai được nhân với một gain. SIFT và ORB mất mọi cặp khớp ở gain 0.15; Harris + MOPS không bị ảnh hưởng. Có hai cách giải thích: *bộ mô tả* nhạy với gain, hoặc *bộ phát hiện* không phát hiện được điểm nào. `contrastThreshold` (0.04) của SIFT và `fastThreshold` (20 mức xám) của ORB là các giới hạn **tuyệt đối** về độ tương phản, còn ngưỡng góc của Harris là tương đối so với đáp ứng mạnh nhất (1%). Nếu bộ phát hiện là thủ phạm thì hạ ngưỡng theo tỉ lệ gain phải chữa được. Kiểm tra: đếm số điểm đặc trưng (Bảng 4.19), rồi khớp một ảnh bình thường với ảnh gain 0.15 và so homography với đáp án (Bảng 4.20).

Bảng 4.19: Số điểm đặc trưng trong một ảnh có gain cho trước.

Gain	1.0	0.6	0.4	0.25	0.15
độ lệch chuẩn mức xám của ảnh	28.4	17.0	11.4	7.1	4.2
SIFT, ngưỡng mặc định	1453	724	282	60	0
SIFT, ngưỡng × gain	1453	1437	1468	1454	1451
ORB, ngưỡng mặc định	3874	2568	1012	225	14
ORB, ngưỡng × gain	3874	3830	3763	3659	3488
Harris + MOPS	1948	1958	1959	1974	1960

Bảng 4.20: Ảnh có gain 1 khớp với ảnh có gain 0.15 (4 lần thử).

Cách xử lý ảnh tối	**Điểm đặc trưng (trung vị)**	**Nội điểm RANSAC (trung vị)**	**Homography đúng**	**Sai số trung vị (px)**
SIFT, ngưỡng mặc định	0	0	0/4	—
SIFT, ngưỡng × gain	1357	658	4/4	0.10
ORB, ngưỡng mặc định	6	0	0/4	—
ORB, ngưỡng × gain	2860	799	4/4	0.45
Harris + MOPS	2040	1018	4/4	0.16

Hạ ngưỡng theo gain khôi phục hoàn toàn cả hai bộ phát hiện: ở gain 0.15 SIFT có khoảng 1400 điểm đặc trưng và 660 nội điểm (sai số trung vị 0.10 px), ORB khoảng 2900 điểm và 800 nội điểm (0.45 px), nhiều như ở ảnh phơi sáng bình thường. **Việc SIFT và ORB hỏng ở ảnh tối là hiện tượng ngưỡng của bộ phát hiện, không phải điểm yếu của bộ mô tả** (bộ mô tả SIFT đã được chuẩn hóa, các phép so sánh của ORB đơn điệu). Bài học thực hành: ngưỡng tuyệt đối đòi hỏi ảnh phải được chuẩn hóa độ phơi sáng trước, hoặc ngưỡng phải thích nghi theo độ tương phản của ảnh; ngưỡng tương đối của Harris + MOPS cùng patch chuẩn hóa cho tính bất biến này miễn phí.

Vì sao Harris + MOPS hỏng ở zoom 3.2× mà lại giải được ở 4×? Kim tự tháp hệ số 2 chỉ lấy mẫu tỉ lệ một lần mỗi octave. Một patch của ảnh 1 ở bậc l chỉ có thể so với patch của ảnh 2 ở bậc l' với $s_2/s_1 \approx 2^{l'-l}$; nếu zoom thật là z thì cặp bậc tốt nhất vẫn

lệch $|\log_2 z - \text{round}(\log_2 z)|$ octave, tối đa nửa octave, tức một hệ số $\sqrt{2}$ về kích thước. Độ lệch bằng 0 ở $z = 1, 2, 4, \ldots$ và lớn nhất ở $z = \sqrt{2}, 2\sqrt{2}, \ldots$. Vậy số nội điểm theo $\log_2 z$ phải có dạng **răng cưa** chu kỳ một octave, cực tiểu tại các điểm nửa octave. Hình 4.17 và Bảng 4.21 (quét mịn hơn, 3 lần thử mỗi mức zoom, trung vị) xác nhận: Harris + MOPS đạt đỉnh ở các octave nguyên (1645 nội điểm ở zoom 1, 636 ở zoom 2, 139 ở zoom 4) và gần như biến mất ở các điểm nửa octave (12 ở 1.41; 4 ở 2.83). Các đỉnh giảm dần theo octave vì ảnh 2, chỉ phủ $1/z^2$ diện tích ảnh 1, chứa chi tiết mà ảnh 1 không phân giải được. SIFT không có răng cưa: không gian tỉ lệ của nó có ba khoảng mỗi octave (độ lệch xấu nhất là một phần sáu octave, hệ số 1.12) và số nội điểm giảm trơn, 1018 → 192. Kim tự tháp của ORB có 8 bậc hệ số 1.2 nên phủ thay đổi tỉ lệ đến $1.2^7 \approx 3.6$ và mất cặp ảnh ngoài khoảng đó (15 nội điểm ở zoom 4).



Hình 4.17: Số nội điểm RANSAC (trung vị của 3 lần thử; cả hai trục theo thang log) theo hệ số zoom của ảnh 2. Vạch xám: các octave nguyên. Harris + MOPS có dạng răng cưa; SIFT giảm trơn; ORB giảm nhanh khi vượt quá khoảng tỉ lệ của kim tự tháp.

Bảng 4.21: Số nội điểm RANSAC (trung vị của 3 lần thử) theo hệ số zoom.

Zoom	1	1.19	1.41	1.68	2	2.38	2.83	3.36	4
SIFT	1018	822	663	574	505	428	286	237	192
ORB	2410	1710	1247	806	513	264	137	65	15
Harris + MOPS	1645	524	12	182	636	167	4	35	139

4.11 Hòa trộn: đường nối và bóng ma

Thí nghiệm có kiểm soát cho phần hòa trộn: hai góc nhìn của thế giới tổng hợp (cách nhau 10°), góc nhìn thứ hai (bên phải) tối hơn 30% (gain 0.7), và homography giữa chúng bị nhiễu loạn **cố ý** bằng một phép tịnh tiến $(3, 2)$ px: sai số căn chỉnh 3.6 px, lớn hơn mức pipeline tạo ra nhưng điển hình cho thị sai hay méo ống kính ở rìa ảnh. Cùng hai góc nhìn được hòa trộn bằng mọi chế độ. *Độ nét* là độ lớn gradient trung bình của độ

chói trong lõi vùng chồng lấn (mức xám trên pixel): bóng ma trộn hai bản sao lệch nhau của mỗi cạnh nên làm giảm độ nét, vì vậy mức giảm độ nét do sai số căn chỉnh đo bộ trộn che giấu sai số đó tốt đến đâu. (Giá trị tuyệt đối còn phụ thuộc độ sáng của các ảnh đóng góp, vì gain cũng co giãn gradient, nên so mỗi bộ trộn với chính nó, cột cuối.)

Bảng 4.22: Độ nét của vùng chồng lấn khi căn chỉnh chính xác và khi có sai số 3.6 px.

Bộ trộn	Độ nét, căn chỉnh chính xác	Độ nét, sai số 3.6 px	Thay đổi
overwrite	3.29	3.29	+0.1%
average	2.67	2.05	−23.1%
feather	2.69	2.16	−19.5%
multi-band	2.75	2.71	−1.6%
multi-band + bù sáng	2.58	2.56	−0.6%

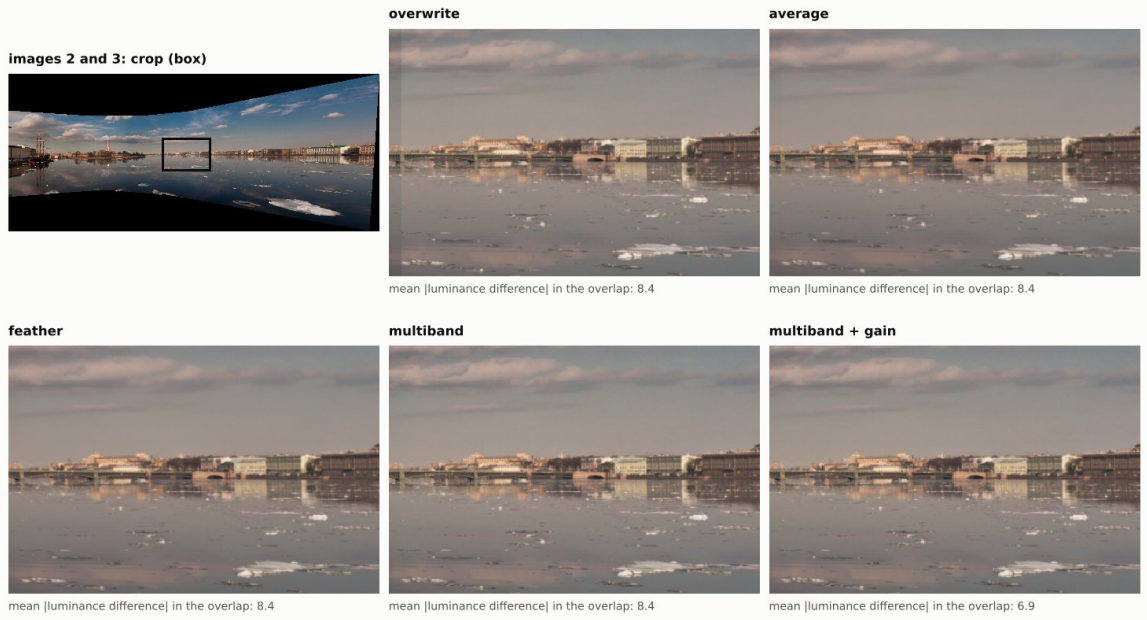


Hình 4.18: Các chế độ hòa trộn khi có sai số căn chỉnh 3.6 px và chênh lệch phơi sáng 30% (vùng cắt 320×200 px ở giữa vùng chồng lấn, quanh cấu trúc ngang mạnh nhất; ô đầu là canvas của chế độ feather và khung vùng cắt). Average và feather cho cây hai lần (bóng ma); overwrite lấy cả vùng cắt từ một ảnh nên sắc nét, đường nối của nó nằm ở biên vùng chồng lấn, ngoài vùng cắt; multi-band giữ nét nhưng còn một vùng tối mờ ở nửa phải, nơi ảnh bị tối đi 30%, và vùng này biến mất khi bù sáng.

Đọc kết quả. Khi có sai số căn chỉnh, average và feather mất từ một phần năm đến một phần tư độ nét (−23%, −20%): mỗi cạnh xuất hiện hai lần, như các cành cây đôi trong Hình 4.18. Overwrite giữ độ nét vì mỗi pixel đến từ một ảnh, nhưng độ sáng nhảy một bậc ở chỗ hai ảnh gặp nhau (đường nối này nằm ở biên của vùng chồng lấn nên không có trong vùng cắt của Hình 4.18; trên ảnh thật nó hiện rõ ở Hình 4.19). Multi-band chỉ mất dưới 2% vì các băng mịn của nó chuyển từ ảnh này sang ảnh kia trong vài pixel, nên mỗi cạnh đến từ *một* ảnh. Cái giá có thể thấy trong ô multi-band khi chưa bù sáng: băng tần thấp trộn hai mức phơi sáng trên một vùng rộng, tức che được *đường nối* nhưng không che được *khác biệt*, nên còn một vùng tối mờ ở phía góc nhìn bị làm tối. Bù sáng

xóa nó: sai khác độ chói trung bình trong vùng chồng lấn giảm từ 37 xuống 7 mức xám (xuống 10 khi có sai số căn chỉnh; không về 0 vì tiên nghiệm σ_g giữ các gain gần 1: gain ước lượng 0.84 và 1.12 chênh nhau hệ số 1.33 trong khi tỉ số thật là $1/0.7 = 1.43$), và độ nét hầu như không mất (-0.6%). Vì vậy pipeline cung cấp `blend="multiband"` cùng `gain_compensation=True`.

Trên cảnh thật (Hình 4.19), khác biệt khó thấy hơn vì sai số căn chỉnh và chênh lệch phơi sáng đều nhỏ: overwrite để lộ một đường nối thẳng ở mép trái của vùng cắt, còn average, feather và multi-band che được nó; con số dưới mỗi ô là sai khác độ chói trung bình của hai ảnh đã warp, chỉ bù sáng mới làm nó thay đổi (8.4 → 6.9 mức xám) vì các chế độ trộn còn lại không cân bằng độ sáng mà chỉ pha trộn.



Hình 4.19: Các chế độ hòa trộn tại một đường nối thật (ảnh 2 và 3 của bộ `boat`, SIFT; ô đầu tiên là panorama của riêng hai ảnh này, với khung vùng cắt). Dưới mỗi ô: sai khác độ chói trung bình giữa hai ảnh đã warp trong vùng chồng lấn.

4.12 Thời gian chạy

Bảng 4.23: Thời gian của các bước phụ thuộc bộ trích xuất (mili-giây, trung bình trên mọi bộ ảnh). Đây là thời gian của hiện thực NumPy này chứ không phải của thuật toán, và dao động hàng chục phần trăm giữa các lần chạy.

	SIFT	ORB	Harris + MOPS
trích xuất đặc trưng, mỗi ảnh	144	**18**	297
so khớp, mỗi cặp ảnh	91	151	70
so khớp, mỗi triệu cặp bộ mô tả	9.1	9.9	8.3
RANSAC, mỗi cặp ảnh	43	29	30

ORB trích xuất nhanh hơn SIFT khoảng 8 lần và nhanh hơn Harris + MOPS (NumPy thuần) khoảng 16 lần. Thời gian so khớp mỗi cặp của ORB lớn nhất chỉ vì ORB giữ nhiều điểm nhất (3900 mỗi ảnh); tính trên một triệu cặp bộ mô tả thì ba loại bộ mô tả tốn như nhau, 8–10 ms (bộ so khớp Hamming dùng popcount sẽ rẻ hơn nhiều cho ORB). Toàn bộ một bộ ảnh mất 1.5–13 s (Bảng 4.14), trong đó bộ `boat` bốn ảnh 3317×1404 px mất khoảng 13 s với bất kỳ bộ trích xuất nào, 10 s trong số đó là hòa trộn đa băng tần trên canvas lớn: các phép tích chập của kim tự tháp Phần 1 được viết để dễ hiểu chứ không để nhanh.

4.13 Sai số tích lũy khi nối chuỗi homography

Pipeline chỉ ước lượng homography cho các cạnh của cây khung rồi nối chuỗi (Mục 2.8.1). Một cặp ảnh chồng lấn nhưng không thuộc cây khung là phép kiểm tra độc lập: homography H_{ij} ước lượng riêng từ các cặp khớp của nó và chuỗi $H_j^{-1}H_i$ đi qua cây phải trùng nhau. Độ lệch giữa hai cách đưa ảnh i sang ảnh j là sai số mà chuỗi đã tích lũy, cộng với sai số của chính ước lượng trực tiếp. Bảng 4.24 (`scripts/analyse_chain_drift.py`) đo độ lệch đó, RMS và lớn nhất, trên lưới các điểm của ảnh i rơi vào trong ảnh j; mọi cặp trong bảng cách nhau hai cạnh của cây.

Bảng 4.24: Độ lệch giữa homography nối chuỗi và homography ước lượng trực tiếp của các cặp không thuộc cây khung (chỉ gồm các cặp được nhận).

Bộ ảnh	Bộ trích xuất	Cặp	Phần ảnh thứ nhất nằm trong ảnh thứ hai	Nội điểm của cặp	RMS (px)	Lớn nhất (px)
newspaper	SIFT	1–3	35%	587	0.08	0.18
newspaper	ORB	1–3	35%	579	0.26	0.69
newspaper	Harris + MOPS	1–3	35%	714	0.10	0.28
boat	SIFT	0–2	32%	256	0.65	1.23
boat	ORB	0–2	31%	270	1.88	5.52
boat	Harris + MOPS	0–2	31%	350	0.69	1.83
boat	SIFT	1–3	14%	30	12.85	28.09
boat	Harris + MOPS	1–3	14%	31	1.84	3.76
cathedral	SIFT	0–2	45%	585	2.83	5.94
cathedral	ORB	0–2	45%	1183	0.62	2.13
cathedral	Harris + MOPS	0–2	44%	373	3.72	12.19

- **Cảnh phẳng:** với tờ báo (nơi homography là chính xác về mặt lý thuyết cho mọi chuyển động của camera) độ lệch chỉ 0.08–0.26 px RMS, cỡ nhiễu định vị: chuỗi hai bước hầu như không tích lũy gì.
- **Cảnh ba chiều:** ở `boat` và `cathedral`, khi cặp trực tiếp có nhiều nội điểm (từ 256 nội điểm, từ 31% diện tích) độ lệch là 0.6–3.7 px RMS, cực đại 5.5 px (`boat`, ORB) và 12.2 px (`cathedral`, Harris + MOPS) ở các góc ảnh. Nguồn có thể là méo ống kính và thị sai nhẹ (mô hình homography không còn đúng ở rìa ảnh) cùng sai số ước lượng; chuỗi không có cơ chế nào sửa chúng.
- **Cặp yếu:** cặp 1–3 của `boat` chỉ chung 14% diện tích với 30–31 nội điểm; SIFT cho 12.9 px (cực đại 28 px), Harris + MOPS chỉ 1.8 px. Chuỗi của cả hai bộ trích xuất

đều dựa trên các cặp 1–2 và 2–3 với hàng trăm nội điểm, còn cặp trực tiếp chỉ có 30–31 nội điểm trên 14% diện tích, nên có vẻ ước lượng trực tiếp của SIFT mới là bên kém tin cậy; con số này không nên đọc là sai số của chuỗi.

Với vài ảnh liền nhau, sai số tích lũy cỡ vài pixel bị hòa trộn đa băng tần che đi (Mục 4.11); với chuỗi dài hoặc vòng 360° nó sẽ lớn dần, và cần hiệu chỉnh đồng thời mọi ảnh (bundle adjustment) cùng đóng vòng (Mục 6.3.2).

Chương 5 Nhận xét và thảo luận

Các Mục 5.1–5.4 thảo luận Phần 1; các Mục 5.5–5.6 thảo luận Phần 2.

5.1 Phân tích kết quả

5.1.1 Điểm mạnh của Poisson editing

- **Loại bỏ chênh lệch màu nền thay vì che nó.** Chỉ số đường ghép giảm 73–86% so với dán trực tiếp (Bảng 4.4) và, trong thí nghiệm có đáp án, Poisson khôi phục chính xác ảnh gốc (Bảng 4.7). Hai kết quả cùng chiều vì $f = g + h$ với h điều hòa ép khớp biên (công thức (2.3)).
- **Giữ cấu trúc của đối tượng.** Dây giằng và cánh máy bay (Hình 4.4), đường viền chim cánh cụt (Hình 4.3) vẫn sắc nét vì $\nabla f = \nabla g + \nabla h$ với ∇h biến thiên chậm.
- **Dung thứ mặt nạ vẽ lỏng.** Ở cả ba cặp, mặt nạ bao thêm một vùng nền của nguồn quanh đối tượng nhưng kết quả vẫn liền mạch: phần nền thừa đó bị "hấp thụ" bởi h nên mang màu của nền mới (nước xanh sẫm ở cặp `01`, trời xanh tím ở cặp `03`). Người dùng không phải chọn độ rộng chuyển tiếp như với Laplacian blending.
- **Nền tảng toán rõ ràng và chi phí chấp nhận được.** Hệ thưa xác định dương có nghiệm duy nhất, giải chính xác (Bảng 4.5) và dưới một giây với ảnh cỡ này.

5.1.2 Trường hợp làm tốt và chưa tốt

Bảng 5.1: Tóm tắt các trường hợp làm tốt và chưa tốt của Poisson editing và số liệu đi kèm.

Cặp	Làm tốt	Chưa tốt	Số liệu
`03`	Biên liền mạch, giữ dây giằng và cánh	Máy bay ngả nhẹ sang vàng lục	S: 29.5 → 4.1; 3.6% giá trị ngoài $[0, 1]$
`02`	Hết quầng trắng quanh chim cánh cụt	Thân chim đen phẳng, mất vân lông	S: 29.2 → 8.0; 75% điểm tối xuống < 0
`01`	Hết mảng nước tối quanh vịt	Thân vịt cháy sáng; còn vệt xanh và bóng phản chiếu của nguồn; gradient trộn gây mờ ảo	S: 83.5 → 19.2; 97% điểm sáng vượt 1

Trong ba cặp thử, lỗi về chi tiết chỉ xuất hiện ở hai cặp có độ chênh $|h|$ lớn nhất ($+0.68$ ở cặp `01`, -0.78 ở cặp `02`; Bảng 4.8), còn cặp `03` với $|h| \leq 0.27$ chỉ bị ngả màu

nhẹ. Nghĩa là độ khó của bài toán phụ thuộc chủ yếu vào mức chênh lệch màu *ở biên của mặt nạ*, không phải vào kích thước vùng ghép (cặp `02` có $|\Omega|$ nhỏ nhất nhưng vẫn gặp lỗi). Với chỉ ba cặp ảnh, nhận định này mang tính quan sát hơn là kết luận thống kê.

5.2 Nguyên nhân của các lỗi

5.2.1 Lan màu và bão hòa

Do $f = g + h$ và h là hàm điều hòa nội suy $f^* - g$ từ biên, nguyên lý cực đại cho biết h nằm trong khoảng giá trị của $f^* - g$ trên $\partial\Omega$. Khi nền ở biên khác xa màu nguồn thì h lớn và *toàn bộ* đối tượng bị cộng cùng một độ lệch trơn, kể cả những vùng vốn đã sáng hoặc tối. Phần bị đẩy ra ngoài $[0, 1]$ bị `np.clip` cắt phẳng, nên mất chi tiết: 97% điểm sáng của thân vịt bị đẩy vượt 1 (cặp `01`) và 75% điểm tối của thân chim cánh cụt bị đẩy xuống dưới 0 (cặp `02`; Bảng 4.8). Vì mỗi kênh màu được giải riêng và có độ lệch h_c riêng, ba kênh bị dịch khác nhau nên màu sắc (hue) của đối tượng cũng bị đổi, như máy bay ngả vàng lục ở cặp `03`.

5.2.2 Cấu trúc thừa của nguồn nằm trong mặt nạ

Phương pháp nhập *toàn bộ* gradient của nguồn trong Ω, nên mọi cấu trúc nằm trong mặt nạ (gợn sóng, bóng phản chiếu, vệt sáng) đều được mang sang chỉ khác màu. Ở cặp `01`, mặt nạ chứa cả bóng phản chiếu của con vịt trong nước nguồn nên bóng đó vẫn còn trong kết quả, không ăn khớp với mặt nước gợn sóng của nền. Đây là sự đánh đổi trong cách vẽ mặt nạ: vẽ rộng giúp biên nằm trong vùng nền mượt của nguồn (Poisson xử lý tốt) nhưng kéo theo cấu trúc thừa; vẽ sát đối tượng ít cấu trúc thừa hơn nhưng đường biên có thể cắt qua cạnh thật. Về sau là suy luận từ lý thuyết, báo cáo chưa thử nghiệm riêng với các mặt nạ khác nhau.

5.2.3 Gradient trộn làm đối tượng đặc bị mờ ảo

Gradient trộn (2.4) chọn gradient mạnh hơn tại từng điểm, nên ở nền nhiều kết cấu (cặp `01`) các gợn sóng của nền thường thắng gradient yếu của thân vịt và xuyên qua nó (Hình 4.6). Với đối tượng đặc, đó là lỗi chứ không phải ưu điểm; gradient trộn phù hợp hơn với đối tượng trong suốt hoặc có lỗ hổng. Ở nền mượt (cặp `03`) hai cách gần như không khác nhau.

5.2.4 Laplacian blending chỉ che mờ đường ghép

Laplacian blending làm mượt *chuyển tiếp* giữa hai ảnh nhưng không thay đổi mức màu bên trong mặt nạ. Mảng nền của nguồn vì thế vẫn còn, chỉ có viền mờ hơn (Hình 4.2, 4.4). Tăng số mức làm vùng chuyển tiếp rộng hơn và S giảm dần ở hai cặp `01`, `03` nhưng vẫn cao gấp 2.4 và 3.7 lần Poisson ở $L = 5$ (Bảng 4.6).

5.3 Ảnh hưởng của tham số

- **Số mức L của Laplacian pyramid:** L lớn làm quầng sáng mờ hơn nhưng rộng hơn; hiệu quả không đơn điệu (cặp `02` tốt nhất ở $L = 2$–3). Không có giá trị L nào đưa chỉ số về mức của Poisson.
- **Nhập hay trộn gradient:** chỉ khác biệt khi nền có gradient đủ mạnh; với đối tượng đặc nên dùng nhập gradient (Mục 4.4.2).
- **Vị trí và tỉ lệ đặt:** vị trí quyết định giá trị nền trên biên $f^*|_{\partial\Omega}$ nên quyết định h theo (2.3); đặt đối tượng ở nơi màu nền gần màu nguồn cho $|h|$ nhỏ và ít bão hòa. Đây là suy luận từ lý thuyết, báo cáo chưa so sánh nhiều vị trí đặt.
- **Mặt nạ:** mặt nạ quyết định Ω và biên $\partial\Omega$; xem phân tích ở trên.

5.4 Hạn chế của giải pháp

- **Bão hòa:** giá trị ngoài $[0, 1]$ chỉ bị cắt bằng `np.clip`, không có cơ chế giữ chi tiết khi $|h|$ lớn (Mục 5.2.1).
- **Điều kiện biên của lưới:** hiện thực giả định Ω không chạm biên vùng đặt; ở mép lưới, giá trị láng giềng ngoài lưới được xem là 0, nên mặt nạ chạm mép sẽ cho kết quả sai.
- **Chuẩn hóa min–max:** `cv2.normalize` với `NORM_MINMAX` kéo giãn từng ảnh để giá trị nhỏ nhất thành 0 và lớn nhất thành 1, thay vì chia cho 255. Với ảnh phủ đủ dải 0–255 hai cách như nhau; nhưng ảnh nền của cặp `03` có giá trị lớn nhất là 253 nên bị kéo giãn khoảng 0.8%, làm lệch nhẹ tỉ lệ giữa nguồn và nền.
- **Xử lý từng kênh độc lập:** có thể làm đổi sắc độ của đối tượng (Mục 5.2.1).
- **Chi phí:** dựng và giải lại ba hệ cho ba kênh dù ma trận giống nhau; với ảnh lớn ($n \sim 10^6$) bộ giải trực tiếp sẽ tốn bộ nhớ và thời gian (Bảng 2.1).
- **Phạm vi:** không xử lý khác biệt về hình học, phối cảnh, hướng chiếu sáng hay độ phân giải giữa nguồn và nền.
- **Đánh giá:** chỉ có ba cặp ảnh; chỉ số S do báo cáo tự đặt và chưa được đối chiếu với đánh giá của người xem.

5.5 Phần 2: So sánh các bộ trích xuất đặc trưng

Đề bài yêu cầu so sánh các bộ trích xuất trên ba tiêu chí: độ chính xác căn chỉnh, độ liền mạch và chất lượng hình ảnh. Bằng chứng ở Chương 4 xếp theo ba tiêu chí đó như sau.

- **Độ chính xác căn chỉnh**: đo bằng RMSE trên ảnh thật (Bảng 4.14–4.15) và bằng sai số so với đáp án trong nghiên cứu có kiểm soát (Hình 4.16, Bảng 4.17). SIFT dẫn đầu (0.65 px trung bình trên ảnh thật; sai số trung vị 0.03 px ở mức dễ), Harris + MOPS thứ hai (0.83 px; 0.03–0.04 px), ORB cuối cùng (1.08 px; 0.11–0.15 px).

- **Độ liền mạch**: đo bằng NCC thông cao và MAE trong vùng chồng lấn (Bảng 4.14) và bằng độ nét tại đường nối khi có sai số căn chỉnh (Bảng 4.22). Ba bộ gần như không phân biệt được: trong mỗi cảnh NCC chênh nhau không quá 0.05 và MAE chênh dưới một mức xám, vì độ liền mạch do cảnh và bộ hòa trộn quyết định nhiều hơn bộ trích xuất. Một sai số căn chỉnh 3.6 px, lớn hơn mọi RMSE của Bảng 4.14 (tối đa 1.29 px), chỉ làm hòa trộn đa băng tần mất 1.6% độ nét; vì vậy chênh lệch cỡ vài phần mười pixel giữa các bộ trích xuất không thể nhìn thấy trên panorama.
- **Chất lượng hình ảnh**: ba panorama của cùng một bộ ảnh gần như giống hệt nhau (Hình 4.12, Hình 4.14); canvas của `boat` có kích thước 3317×1404, 3310×1401 và 3351×1412 px, chênh nhau không quá 1.2%.

Sự khác nhau thật sự giữa ba bộ nằm ở chỗ khác: hình học chúng cung cấp, mức khó mà chúng chịu được, và chi phí.

5.5.1 SIFT (OpenCV)

Điểm mạnh. Hình học chính xác nhất ở cả năm cảnh thật (0.65 px trung bình) và trong các thí nghiệm chồng lấn, xoay, zoom và phơi sáng của nghiên cứu có kiểm soát (sai số trung vị 0.03–0.14 px). Dải làm việc rộng nhất: giải được cặp chỉ chung 7% diện tích và mọi mức zoom đến 4×, với số nội điểm giảm trơn theo zoom vì không gian tỉ lệ có ba khoảng mỗi octave (Bảng 4.21).

Điểm yếu. `contrastThreshold` là một giới hạn **tuyệt đối** về độ tương phản, nên SIFT không tìm được điểm nào trong ảnh tối (0 điểm ở gain 0.15) trừ khi ngưỡng được hạ theo độ phơi sáng; bộ mô tả của nó thì không có lỗi (Bảng 4.19–4.20). SIFT cũng là bộ chịu nhiễu kém nhất (22/30 cặp giải được ở $\sigma = 60$ mức xám, so với 28 của ORB và 27 của Harris + MOPS), và bộ mô tả 128 số thực chiếm nhiều bộ nhớ nhất (512 byte, so với 32 byte của ORB và 256 byte của MOPS).

5.5.2 ORB (OpenCV)

Điểm mạnh. Trích xuất rẻ nhất (18 ms mỗi ảnh, nhanh hơn SIFT khoảng 8 lần; Bảng 4.23) với bộ mô tả nhị phân gọn 32 byte; cùng với Harris + MOPS là bộ chịu nhiễu tốt nhất (28/30 ở $\sigma = 60$).

Điểm yếu. Hình học kém chính xác nhất: 1.08 px trung bình trên ảnh thật, và 0.11–0.15 px trung vị ở mức dễ của nghiên cứu có kiểm soát, gấp 3–5 lần hai bộ còn lại, vì góc FAST nằm ở vị trí *nguyên* của một bậc kim tự tháp hệ số 1.2. Dải làm việc hẹp nhất: mất cặp ảnh đầu tiên khi chồng lấn thu hẹp (22% diện tích so với 7% của SIFT), khi zoom vượt 2.5× và khi nhòe vượt $\sigma = 1.5$ px. Số lỗi thầm lặng nhiều nhất (77 trên 1050 lần thử, Bảng 4.18). Ngưỡng FAST cũng tuyệt đối nên ORB hỏng ở ảnh tối, dù cầm cự lâu hơn SIFT một chút (đến gain 0.25). Thời gian so khớp mỗi cặp của ORB dài nhất (151 ms) chỉ vì nó giữ nhiều điểm nhất (3900 mỗi ảnh); tính trên một triệu cặp bộ mô tả thì ba bộ tốn như nhau, 8–10 ms trong hiện thực NumPy này (một bộ so khớp

Hamming dùng popcount sẽ rẻ hơn nhiều cho ORB). Các hàng thời gian mô tả hiện thực chứ không phải thuật toán, và dao động hàng chục phần trăm giữa các lần chạy.

5.5.3 Harris + MOPS (tự viết)

Điểm mạnh. Các cặp khớp sạch nhất (tỉ lệ nội điểm trung bình 0.94). Chịu phơi sáng tốt nhất: giải được mọi mức đến gain 0.15, vì ngưỡng góc là *tương đối* so với đáp ứng mạnh nhất và patch được chuẩn hóa về trung bình 0, phương sai 1 (Mục 2.5). Chịu nhòe tốt nhất (σ đến 4.5 px), có lẽ vì patch được lấy mẫu từ một bậc kim tự tháp đã bị làm mờ; đây là giả thuyết, chưa được kiểm chứng riêng. Độ chính xác thứ hai (0.83 px). Vì tự cài đặt, mọi hành vi ở trên đều được giải thích bằng cơ chế (Mục 2.5, 4.10) chứ không chỉ quan sát.

Điểm yếu. Trích xuất chậm nhất (297 ms mỗi ảnh, NumPy thuần). Kim tự tháp hệ số 2 lấy mẫu tỉ lệ một lần mỗi octave, nên số nội điểm có dạng răng cưa theo hệ số zoom và bộ này hỏng *giữa* hai octave (zoom 3.2×) dù vẫn giải được ở 4× (Hình 4.17).

5.5.4 Bảng tổng hợp và khuyến nghị

Bảng 5.2: Tổng hợp so sánh ba bộ trích xuất (in đậm: tốt nhất; ở hàng "xoay" cả ba như nhau). Mọi số liệu lấy từ các bảng của Chương 4.

	SIFT (OpenCV)	**ORB (OpenCV)**	**Harris + MOPS (tự viết)**
Ảnh thật (5 bộ ảnh, 15 panorama)			
Sai số tái chiếu RMS của nội điểm, trung bình (px)	**0.65**	1.08	0.83
Tỉ lệ nội điểm, trung bình	0.89	0.92	**0.94**
Nghiên cứu có kiểm soát: mức khó nhất vẫn giải được ít nhất 90% trong 30 lần thử			
Góc giữa hai góc nhìn (diện tích chồng lấn)	**22°** (**7%**)	18° (22%)	21° (10%)
Xoay trong mặt phẳng	180°	180°	180°
Zoom của ảnh 2	**4×**	2.5×	2.5×
Nhiễu σ (mức xám)	40	**60**	**60**
Gain của ảnh 2 (nhỏ hơn = tối hơn)	0.4	0.25	**0.15**
Nhòe σ (px)	3	1.5	**4.5**
Cặp được nhận nhưng homography sai (trên 1050 lần thử)	**25**	77	46
Thời gian (hiện thực này, một lần chạy)			
Trích xuất đặc trưng, mỗi ảnh (ms)	144	**18**	297
So khớp, mỗi cặp ảnh (ms)	91	151	**70**
RANSAC, mỗi cặp ảnh (ms)	43	**29**	30

Khuyến nghị. Dùng SIFT khi hình học phải chính xác và điều kiện chụp không kiểm soát được (chồng lấn nhỏ, zoom, nhòe). Dùng ORB khi tốc độ quan trọng và ảnh sắc nét, phơi sáng tốt, chồng lấn rộng. Harris + MOPS là lựa chọn thích hợp để hiểu cơ chế (và, trong thí nghiệm này, cho ảnh có phơi sáng hoặc độ nhòe thay đổi mạnh) nhưng cần một kim tự tháp mịn hơn nếu tỉ lệ giữa các ảnh thay đổi nhiều. Dù dùng bộ nào, nên chuẩn hóa độ phơi sáng trước khi ảnh vào một bộ phát hiện có ngưỡng tuyệt đối, hoặc để ngưỡng bám theo độ tương phản của ảnh (Bảng 4.19–4.20).

5.6 Phần 2: Thành công và các trường hợp khó

5.6.1 Những gì đã hoạt động

- Cả 15 tổ hợp (5 bộ ảnh × 3 bộ trích xuất) đều được ghép tự động, không can thiệp thủ công, với sai số tái chiếu từ 0.24 đến 1.29 px.
- Mọi thành phần tự viết khớp với một tham chiếu độc lập: 41 bài kiểm thử (so khớp với `cv2.BFMatcher`, warp với `cv2.warpPerspective`, DLT và RANSAC với homography đã biết, Harris + MOPS với một phép xoay đã biết, toàn pipeline với các góc nhìn tổng hợp có đáp án chính xác).
- Nghiên cứu có kiểm soát biến các cảm nhận định tính thành giới hạn đo được và dẫn tới ba lời giải thích đã được *kiểm chứng* chứ không phỏng đoán: ảnh tối (ngưỡng của bộ phát hiện, Bảng 4.19–4.20), răng cưa của Harris + MOPS (lấy mẫu theo octave, Bảng 4.21), cặp giả (chữ lặp, Bảng 4.13). Trên ảnh thật, hai chẩn đoán bổ sung tách nội dung thay đổi khỏi căn chỉnh lệch (Mục 4.9) và đo sai số nối chuỗi (Mục 4.13).
- Hòa trộn đa băng tần giữ độ nét khi căn chỉnh lệch 3.6 px (mất 1.6%, so với 20–23% của average và feather) và cùng bù sáng xóa được vùng chuyển sáng tối (sai khác độ chói trong vùng chồng lấn giảm từ 37 xuống 7 mức xám).
- Các cặp không chồng lấn hoặc giả đều bị loại đúng: `newspaper` $(0, 2)$ và $(0, 3)$, `boat` $(0, 3)$ bị cả ba bộ loại, kể cả cặp $(0, 2)$ của `newspaper` vốn có 194 cặp khớp tạm thời nhờ chữ lặp.

5.6.2 Những gì không hoạt động, và vì sao

- **Cặp sai vẫn được chấp nhận.** Ba phép kiểm tra (số nội điểm, hạng của ma trận DLT, hình dạng vùng phủ) chặn được cặp giả của `newspaper`, nhưng chúng là lưới an toàn chứ không phải bảo đảm: nghiên cứu có kiểm soát có 25 (SIFT), 77 (ORB) và 46 (Harris + MOPS) cặp được chấp nhận mà homography lệch từ 3 px trở lên, phần lớn ở rìa dải chịu đựng của mỗi bộ (nhiễu mạnh; Bảng 4.18). Một kiểm tra nhất quán trên toàn đồ thị (tích các homography quanh một chu trình phải là ma trận đơn vị, mà Bảng 4.24 đã đo cho các cặp ngoài cây) có thể bắt thêm một phần; hiện chưa được dùng làm bộ lọc.

- **Cấu trúc lặp.** Các dòng chữ in tạo ra 42 "nội điểm" đều là bản sao của một điểm duy nhất; ratio test đơn thuần không đủ, và phép thử hạng của ma trận DLT mới phát hiện ra sự suy biến (Mục 2.7, Bảng 4.13).
- **Phép chiếu phẳng.** Mọi ảnh được ánh xạ lên một mặt phẳng nên các rìa bị kéo giãn khi trường nhìn rộng: panorama `boat` bị uốn cong và có nhiều vùng đen, panorama `cathedral` lõm hình chữ V ở mép trên. Panorama rộng cần phép chiếu trụ hoặc cầu; ở đây ảnh được ghép trên mặt phẳng, đúng như đề bài mô tả.
- **Thị sai, nội dung chuyển động và ngoại suy.** Homography chỉ chính xác cho phép quay quanh tâm quang học hoặc cho một mặt phẳng. Ở `boat`, mặt nước và băng trôi đổi chỗ giữa các lần chụp (không phép dịch nào khôi phục được tương quan), và ở cặp 2–3 homography bị ngoại suy lệch về phía bầu trời, nơi không có nội điểm (Mục 4.9). Hòa trộn đa băng tần che được đường nối nhưng không xóa được bóng ma, và không có bước tìm đường nối tối ưu (graph cut).
- **Phơi sáng và màu.** Một hệ số gain cho mỗi ảnh là toàn bộ mô hình quang học: không có vignetting, không cân bằng trắng. Ảnh xám ở đầu bộ `mountain` và `cathedral` cho thấy một gain không sửa được khác biệt về tông màu (MAE khoảng 20 mức xám của `mountain` chỉ giảm xuống 17.1 sau khi bù sáng).
- **Sai số tích lũy.** Các homography được nối chuỗi dọc cây khung, không có bundle adjustment hay đóng vòng. Sai số tích lũy đo được là 0.08–0.26 px trên cảnh phẳng và 0.6–3.7 px RMS (tối đa 12 px ở góc ảnh) trên hai cảnh ba chiều có đủ nội điểm (Bảng 4.24); chuỗi dài hơn sẽ lệch nhiều hơn.
- **Ít kết cấu.** Bộ `mountain` chỉ có 164–307 nội điểm dù chồng lấn tới 45% (tuyết ít kết cấu): vẫn đủ để xác định 8 bậc tự do của homography, nhưng gần với ngưỡng 20 nội điểm của phép kiểm tra chấp nhận cặp nếu chồng lấn nhỏ đi.
- **Tốc độ.** So khớp và hòa trộn là mã NumPy viết cho dễ hiểu chứ không cho nhanh: bộ `boat` bốn ảnh mất khoảng 13 s với bất kỳ bộ trích xuất nào, trong đó 10 s là hòa trộn đa băng tần trên canvas rộng 3300 px.

Chương 6 Kết luận và hướng phát triển

6.1 Tóm tắt kết quả đạt được

6.1.1 Phần 1: Gradient Domain Editing

Báo cáo đã hiện thực từ đầu Gradient Domain Editing (Poisson image editing) cho bài toán ghép ảnh: dựng hệ phương trình Poisson rời rạc, giải bằng bộ giải thưa cho từng kênh màu và hỗ trợ cả gradient trộn; đồng thời hiện thực hai phương pháp đối chứng là dán trực tiếp và Laplacian pyramid blending. Trên ba cặp ảnh:

- Poisson loại bỏ đường ghép: chỉ số S giảm 73–86% so với dán trực tiếp và chỉ bằng khoảng 1/5–1/3 so với Laplacian blending (Bảng 4.4).
- Thí nghiệm có đáp án xác nhận lý thuyết: khi nguồn chỉ lệch nền một hàm điều hòa (hằng số, tuyến tính), Poisson khôi phục chính xác ảnh gốc, còn hai phương pháp kia chỉ đạt PSNR $\approx$ 21–22 dB (Bảng 4.7).
- Cái giá phải trả là màu nền lan vào đối tượng và hiện tượng bão hòa khi chênh lệch màu ở biên lớn ($\approx$ 30% giá trị bị cắt ở các cặp `01`, `02`), cùng nguy cơ mờ ảo khi dùng gradient trộn cho đối tượng đặc trên nền nhiều kết cấu.

6.1.2 Phần 2: Ghép ảnh toàn cảnh

Báo cáo đã cài đặt đầy đủ pipeline ghép ảnh toàn cảnh theo hướng dẫn của đề bài (tiền xử lý, đặc trưng, so khớp, homography, căn chỉnh, hòa trộn) và chạy cùng một pipeline với ba bộ trích xuất: SIFT và ORB của OpenCV, và Harris + MOPS tự viết. Các kết luận chính:

1. Cả ba bộ đều ghép được cả năm bộ ảnh thật với sai số tái chiếu dưới 1.3 px, và panorama cuối cùng gần như không phân biệt được bằng mắt; yếu tố hạn chế chất lượng là cảnh (thị sai, nội dung chuyển động, phơi sáng) và mô hình (chiếu phẳng, một gain mỗi ảnh), không phải bộ trích xuất.
2. Các bộ khác nhau ở hình học và ở mức khó chịu được: SIFT chính xác nhất và rộng nhất (chồng lấn 7%, zoom 4×), ORB rẻ nhất nhưng kém chính xác và hẹp nhất, Harris + MOPS cho các cặp khớp sạch nhất và chịu phơi sáng, nhòe tốt nhất nhưng có răng cưa theo tỉ lệ.
3. Hai trong các hiện tượng khó hiểu nhất đều có nguyên nhân cơ chế đã được kiểm chứng: SIFT và ORB hỏng ở ảnh tối vì ngưỡng tuyệt đối của bộ phát hiện chứ không phải do bộ mô tả, và Harris + MOPS hỏng giữa hai octave vì kim tự tháp

hệ số 2.

4. Một con số trung bình về độ khớp của vùng chồng lấn có thể che hai nguyên nhân khác nhau (nội dung thay đổi và căn chỉnh lệch); phép dịch thử tách được chúng.

6.2 Hướng cải thiện (Phần 1)

Các đề xuất dưới đây xuất phát từ những hạn chế đã quan sát ở Phần 1; báo cáo chưa thử nghiệm chúng. Hướng mở rộng cho Phần 2 nằm ở Mục 6.3.2.

1. **Giảm bão hòa:** đối sánh màu giữa nguồn và nền trước khi giải (ví dụ histogram matching) để giảm $|h|$; hoặc thay phép cắt `np.clip` bằng một ánh xạ tông (tone mapping) hay bài toán bình phương tối thiểu có ràng buộc hộp $0 \le f \le 1$.
2. **Tăng tốc:** phân rã LU một lần rồi dùng lại cho ba vế phải (ma trận giống nhau ở ba kênh); với ảnh lớn, dùng conjugate gradient có tiền điều kiện hoặc multigrid (Bảng 2.1).
3. **Sửa chuẩn hóa:** chia cho 255 thay cho `NORM_MINMAX` để nguồn và nền luôn cùng thang.
4. **Chọn biên tốt hơn:** tối ưu đường biên của mặt nạ để cắt qua vùng nguồn mượt và khớp nền, như Drag-and-Drop Pasting [22].
5. **Đánh giá khách quan hơn:** thêm nhiều cặp ảnh, đối chiếu với hiện thực có sẵn như `cv2.seamlessClone`, và thu thập đánh giá của người xem để kiểm chứng chỉ số S.

6.3 Hướng mở rộng

6.3.1 Phần 1: Gradient Domain Editing

- Cùng khung Poisson có thể dùng cho làm phẳng kết cấu, đổi màu hoặc ánh sáng cục bộ [3] và nén dải động ảnh HDR [5]; chỉ cần đổi trường dẫn hướng $\mathbf{v}$.
- Với yêu cầu thời gian thực, có thể thay việc giải hệ bằng nội suy dùng tọa độ (mean-value coordinates) [23].
- Blending trong miền gradient cũng có thể áp dụng cho vùng chồng lấn khi ghép ảnh toàn cảnh [4]; hướng này được nêu lại ở Phần 2.

6.3.2 Phần 2: Ghép ảnh toàn cảnh

- Phép chiếu trụ hoặc cầu và bundle adjustment cho góc quay và tiêu cự (Brown & Lowe, 2007) [18], đồng thời đóng vòng cho panorama 360°.
- Tìm đường nối bằng quy hoạch động hoặc graph cut để đi vòng qua vật chuyển động; dùng hòa trộn Poisson của Phần 1 trong vùng chồng lấn thay cho (hoặc cùng với) hòa trộn đa băng tần để triệt tiêu chênh lệch phơi sáng tại đường nối.

- Chuẩn hóa phơi sáng trước khi phát hiện, mô hình vignetting và gain theo từng kênh màu.
- Kim tự tháp mịn hơn (ví dụ ba mức mỗi octave như SIFT) cho Harris + MOPS để xóa răng cưa theo tỉ lệ; thêm bộ trích xuất khác (AKAZE và BRISK không có trong bản OpenCV 5.0.0 dùng ở đây) hoặc một bộ phát hiện kèm so khớp học được làm tầng đầu thay thế.
- Bộ ảnh mẫu của môn học khi có: đặt vào `data/panorama/input/<tên bộ>/` rồi chạy `python scripts/run_scenes.py`.

Tài liệu tham khảo

[1] Y. Saad, *Iterative Methods for Sparse Linear Systems*. SIAM, 2nd ed., 2003.

[2] A. George, "Nested dissection of a regular finite element mesh," *SIAM Journal on Numerical Analysis*, vol. 10, no. 2, pp. 345–363, 1973.

[3] P. Pérez, M. Gangnet, and A. Blake, "Poisson image editing," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 22, no. 3, pp. 313–318, 2003.

[4] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2nd ed., 2022.

[5] R. Fattal, D. Lischinski, and M. Werman, "Gradient domain high dynamic range compression," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 21, no. 3, pp. 249–256, 2002.

[6] X. S. Li, "An overview of SuperLU: Algorithms, implementation, and user interface," *ACM Transactions on Mathematical Software*, vol. 31, no. 3, pp. 302–325, 2005.

[7] P. Virtanen, R. Gommers, *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[8] P. J. Burt and E. H. Adelson, "A multiresolution spline with application to image mosaics," *ACM Transactions on Graphics*, vol. 2, no. 4, pp. 217–236, 1983.

[9] P. J. Burt and E. H. Adelson, "The Laplacian pyramid as a compact image code," *IEEE Transactions on Communications*, vol. 31, no. 4, pp. 532–540, 1983.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[11] S. van der Walt, J. L. Schönberger, *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.

[12] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proc. Alvey Vision Conference*, pp. 147–151, 1988.

[13] M. Brown, R. Szeliski, and S. Winder, "Multi-image matching using multi-scale oriented patches," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 510–517, 2005.

[14] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[15] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, pp. 2564–2571, 2011.

[16] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd ed., 2004.

[17] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[18] M. Brown and D. G. Lowe, "Automatic panoramic image stitching using invariant features," *International Journal of Computer Vision*, vol. 74, no. 1, pp. 59–73, 2007.

[19] C. R. Harris, K. J. Millman, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[20] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[21] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[22] J. Jia, J. Sun, C.-K. Tang, and H.-Y. Shum, "Drag-and-drop pasting," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 25, no. 3, pp. 631–637, 2006.

[23] Z. Farbman, G. Hoffer, Y. Lipman, D. Cohen-Or, and D. Lischinski, "Coordinates for instant image cloning," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 28, no. 3, pp. 67:1–67:9, 2009.

Phụ lục A Phân công công việc

Bảng A.1: Phân công công việc và mức đóng góp.

Thành viên	Công việc phụ trách	Đóng góp
Trần Gia Hiển	Code: hiện thực Phần 1 (image blending) và Phần 2 (ghép ảnh toàn cảnh)	100%
Nguyễn Chí Tân	Viết báo cáo: tổng hợp, rà soát và đối chiếu báo cáo với code	100%
Võ Thanh	Viết báo cáo: lý thuyết, hiện thực, kết quả, thảo luận	100%
Mai Anh Khoa	Test: chạy thử, kiểm tra kết quả và số liệu thực nghiệm của hai phần	100%

Phụ lục B Nguồn tham khảo và công cụ hỗ trợ

Để bảo đảm tính trung thực học thuật, mục này ghi rõ nguồn của những thành phần không do sinh viên tự tạo.

- **Ảnh:** sáu ảnh của Phần 1 được lấy từ Internet (Bảng 4.1). Ảnh của Phần 2 là bộ ảnh kiểm thử ghép ảnh của dự án OpenCV (`opencv_extra/testdata/stitching`; nguồn gốc ghi trong `data/panorama/README.md`, Mục 1.3.4).
- **Mã nguồn Phần 1:** thuật toán dán trực tiếp, Laplacian pyramid blending và Poisson blending do sinh viên tự viết trong repo `Computer-Vision-2` (`src/image_blending/`) (NumPy, SciPy, OpenCV). Theo README của repo, phần Laplacian pyramid có tham khảo thêm hai tài liệu trực tuyến: E. Arnebäck, *Laplacian Pyramids*; J. Salazar Cavazos, *Image Blending* (README không ghi đường dẫn). Công thức dựa trên [3, 8, 9]. Script `scripts/make_figures_part1.py` chỉ gọi lại các hàm của repo để sinh hình và số liệu của Phần 1.
- **Mã nguồn Phần 2:** Harris + MOPS, so khớp, homography (DLT, RANSAC, Levenberg–Marquardt), warp, đồ thị chồng lấn và hòa trộn do sinh viên tự viết bằng NumPy/SciPy (Bảng 3.3); SIFT và ORB dùng bản của OpenCV. Công thức dựa trên [16, 13, 12, 14, 15, 17, 8, 18].
- **Công cụ hỗ trợ:** nhóm sử dụng AI hỗ trợ cho một số tác vụ sau: xây dựng sườn bài báo cáo, hỗ trợ định dạng mã nguồn LaTeX, hỗ trợ rà soát lỗi chính tả và cải thiện văn phong báo cáo.

Phụ lục C Cách chạy lại Phần 2

Cài đặt (Python 3.12 trở lên; kết quả trong báo cáo được tạo với Python 3.13, OpenCV 5.0.0, NumPy 2.5, SciPy 1.18, Matplotlib 3.11):

```
pip install -e ".[dev]"
```

Ghép một bộ ảnh hoặc ảnh của riêng bạn:

```
python run_panorama.py --scene boat --method sift                # một bộ ảnh mẫu
python run_panorama.py --folder đường/dẫn/thư/mục --compare      # cả ba bộ trích
↪  xuất, một bảng so sánh
python run_panorama.py --images a.jpg b.jpg c.jpg --gain --out pano.jpg
```

Tái tạo mọi số liệu và hình của báo cáo (thời gian là ước lượng trên máy dùng chung):

```
python scripts/run_scenes.py              # 5 bộ ảnh x 3 bộ trích xuất, ~2 phút
python scripts/run_synthetic.py           # nghiên cứu có kiểm soát, ~10 phút
python scripts/make_figures.py            # biểu đồ và bảng markdown từ các file CSV
↪  ở trên
python scripts/describe_scenes.py         # kích thước ảnh, ảnh xám, chồng lấn từng
↪  cặp, ~15 giây
python scripts/analyse_overlap_bands.py   # NCC theo dải và phép dịch thử, ~40 giây
python scripts/analyse_chain_drift.py     # sai số nối chuỗi, ~40 giây
python -m pytest tests -q                 # 41 bài kiểm thử, ~20 giây
jupyter notebook notebooks/demo_panorama.ipynb   # các suy diễn và thí nghiệm, đã
↪  chạy sẵn
```

Thêm ảnh của môn học (hoặc của riêng bạn): tạo thư mục `data/panorama/input/<tên bộ>/` chứa từ hai ảnh trở lên (thứ tự tùy ý, các ảnh được chụp từ một vị trí và chỉ xoay máy), rồi chạy `python run_panorama.py --scene <tên bộ> --compare` hoặc `python scripts/run_scenes.py --scenes <tên bộ>`. Không cần sửa mã.

Bảng C.1: Nguồn của các bảng và hình có số liệu của Phần 2 (các bảng mô tả 3.3, 3.2, 2.4, 5.2 và D.1 không có trong danh sách).

Bảng / hình	**Được tạo bởi**
Hình 3.4	draw.io
Hình 4.9, 4.13, 4.14, 4.19	`scripts/run_scenes.py` (`figures/keypoints_*`, `matches_*`, `panorama_*`, `blend_*`)
Hình 2.3, 4.10, 4.11, 4.12, 4.17, 4.18	`notebooks/demo_panorama.ipynb`
Hình 4.15, 4.16	`scripts/make_figures.py`
Bảng 2.3, 3.4, 4.10, 4.11, 4.12, 4.13, 4.19, 4.20, 4.21, 4.22	`notebooks/demo_panorama.ipynb`
Bảng 4.9	`scripts/describe_scenes.py` (`scenes_data.md`)
Bảng 4.14, 4.15, 4.23	`scripts/run_scenes.py`, rồi `scripts/make_figures.py` (`scenes.csv`, `pairs.csv`, `scenes_summary.md`)
Bảng 4.16	`scripts/analyse_overlap_bands.py` (`overlap_bands_boat_sift.md`)
Bảng 4.17, 4.18	`scripts/run_synthetic.py`, rồi `scripts/make_figures.py` (`synthetic/summary.md`)
Bảng 4.24	`scripts/analyse_chain_drift.py` (`chain_drift.md`)

Phụ lục D Tham số mặc định của Phần 2

Bảng D.1: Tham số mặc định của `StitchConfig` (`src/panorama/config.py`) dùng cho mọi kết quả trên ảnh thật.

Tham số	Giá trị	Ý nghĩa
`max_side`	1000	thu nhỏ để cạnh dài nhất không quá 1000 px
`method`	`"sift"`	`"sift"`, `"orb"` hoặc `"harris"`
`max_keypoints`	4000	cùng ngân sách điểm cho mọi bộ trích xuất
`fix_coords`	`True`	sửa lệch nửa pixel của tọa độ SIFT/ORB trong OpenCV
`ratio`	0.75	ngưỡng ratio test của Lowe
`mutual`	`False`	kiểm tra chéo hai chiều (không dùng)
`unique_matches`	`True`	lọc một–một
`ransac_thresh`	3.0 px	ngưỡng nội điểm
`ransac_conf`	0.995	xác suất mong muốn rút được một mẫu sạch
`ransac_max_iters`	3000	số mẫu tối đa
`min_inliers`	20	số nội điểm tối thiểu để nhận một cặp
`min_rank_ratio`	10^{-3}	σ_8/σ_1 tối thiểu của ma trận DLT
`max_area_ratio`	25	tỉ lệ diện tích tối đa của vùng phủ ảnh sau warp
`seed`	0	hạt giống của RANSAC
`max_canvas_side`	6000	thu nhỏ panorama nếu canvas lớn hơn
`blend`	`"multiband"`	`"overwrite"`, `"average"`, `"feather"` hoặc `"multiband"`
`num_levels`	5	số bậc kim tự tháp của bộ hòa trộn đa băng tần
`gain_compensation`	`False`	bù sáng (tắt mặc định)
`crop`	`True`	cắt panorama theo hộp bao của các điểm ảnh hợp lệ

Tham số riêng của từng bộ trích xuất: SIFT `contrastThreshold` = 0.04 (mặc định của OpenCV); ORB `fastThreshold` = 20, 8 bậc, hệ số 1.2; Harris + MOPS $\sigma_d = 1$, $\sigma_i = 1.5$, 4 bậc hệ số 2, ngưỡng tương đối 1%, ANMS với $c = 0.9$, patch 8×8 cách nhau 5 px lấy từ cửa sổ có hướng 40×40 px (Mục 2.5).